



Data Science and Analytics

Insights and Applications

Er.Rudra Nepal

Assistant Professor Nepal College of Information Technology

17th JULY, 2025

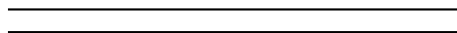


TABLE OF CONTENTS

TABLE OF CONTENTS	i
CHAPTER ONE: INTRODUCTION	1
1.1 Knowledge Discovery in Databases (KDD)	1
1.2 Structured and Unstructured Data and Their Examples	7
1.3 Overview of Data Preprocessing	10
1.3.1 Data Cleaning	10
1.3.2 Data Integration	12
1.3.3 Data Transformation and Discretization	13
CHAPTER TWO: Statistical Foundation	16
2.1 Types of variable Numerical and Categorical	16
2.2 Empirical Distribution	20
2.2.1 Numeric Data	20
2.2.2 Categorical Data	30
2.3 Correlation Analysis	37
2.3.1 Pearson Correlation Coefficient (r)	37
2.3.2 Cross-Tabulation for Categorical Variables	40
2.3.3 Assessing Correlation between numeric and categorical variable	44
2.4 Statistical Significance	49
CHAPTER THREE: Numeric Data	58
3.1 Introduction to Linear Regression	58
3.2 Types of Linear Regression	59
3.2.1 Multivariate Linear Regression	65
3.2.2 Ordinary Least Squares (OLS) Estimation	67
3.2.3 R-squared and Adjusted R-squared	70
3.2.4 Definition of Multicollinearity	76
3.3 Non-Parametric Regression: Nadaraya-Watson Kernel Regression . .	78
3.4 Principal Component Analysis (PCA)	80
3.5 Numericals	94
CHAPTER FOUR: Categorical and Mixed-type Data	105
4.1 What is a Decision Tree?	105
4.1.1 What ID3 Does	108

4.1.2	CART (Gini Index)	114
4.2	Variable Importance: Permutation Tests and Partial Dependence Plots	119
4.3	K means clustering	121
4.4	Clustering Mixed-Type Data: K-Means and Distance Metrics	128
4.4.1	Gower Distance	129
4.4.2	K-Prototypes	129
4.4.3	Preprocessing	130
4.5	Logistic Regression	132

CHAPTER ONE: INTRODUCTION

1.1 Knowledge Discovery in Databases (KDD)

Knowledge Discovery in Databases (KDD) refers to the complete process of uncovering valuable knowledge from large datasets. It starts with the selection of relevant data, followed by preprocessing to clean and organize it, transformation to prepare it for analysis, data mining to uncover patterns and relationships, and concludes with the evaluation and interpretation of results, ultimately producing valuable knowledge or insights. KDD is widely utilized in fields like machine learning, pattern recognition, statistics, artificial intelligence, and data visualization.

The KDD process is iterative, involving repeated refinements to ensure the accuracy and reliability of the knowledge extracted. The whole process consists of the following steps:

- Data Selection
- Data Cleaning and Preprocessing
- Data Transformation and Reduction
- Data Mining
- Evaluation and Interpretation of Results

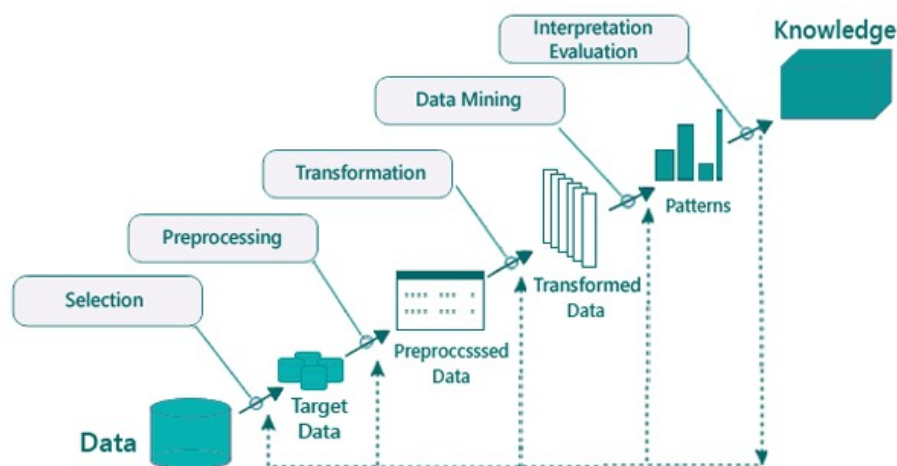


Figure 1.1: Knowledge Discovery in Database.

Data Selection

Data Selection is the initial step in the Knowledge Discovery in Databases (KDD) process, where relevant data is identified and chosen for analysis. It involves selecting a dataset or focusing on specific variables, samples, or subsets of data that will be used to extract meaningful insights.

- It ensures that only the most relevant data is used for analysis, improving efficiency and accuracy.
- It involves selecting the entire dataset or narrowing it down to particular features or subsets based on the task's goals.
- Data is selected after thoroughly understanding the application domain.
- By carefully selecting data, we ensure that the KDD process delivers accurate, relevant, and actionable insights.

Data Cleaning and Preprocessing

In the KDD process, Data Cleaning is essential for ensuring that the dataset is accurate and reliable by correcting errors, handling missing values, removing duplicates, and addressing noisy or outlier data.

- **Missing Values:** Gaps in data are filled with the mean or most probable value to maintain dataset completeness.
- **Noisy Data:** Noise is reduced using techniques like binning, regression, or clustering to smooth or group the data.
- **Removing Duplicates:** Duplicate records are removed to maintain consistency and avoid errors in analysis.

Data cleaning is crucial in KDD to enhance the quality of the data and improve the effectiveness of data mining.

Data Transformation and Reduction

Data Transformation in KDD involves converting data into a format that is more suitable for analysis.

- **Normalization:** Scaling data to a common range for consistency across variables.
- **Discretization:** Converting continuous data into discrete categories for simpler analysis.
- **Data Aggregation:** Summarizing multiple data points (e.g., averages or totals) to simplify analysis.
- **Concept Hierarchy Generation:** Organizing data into hierarchies for a clearer, higher-level view.

Data Reduction helps simplify the dataset while preserving key information.

- **Dimensionality Reduction (e.g., PCA):** Reducing the number of variables while keeping essential data.
- **Numerosity Reduction:** Reducing data points using methods like sampling to maintain critical patterns.
- **Data Compression:** Compacting data for easier storage and processing.

Together, these techniques ensure that the data is ready for deeper analysis and mining.

Data Mining

Data Mining is the process of discovering valuable, previously unknown patterns from large datasets through automatic or semi-automatic means. It involves exploring vast amounts of data to extract useful information that can drive decision-making.

Key characteristics of data mining patterns include:

- **Validity:** Patterns that hold true even with new data.
- **Novelty:** Insights that are non-obvious and surprising.
- **Usefulness:** Information that can be acted upon for practical outcomes.

- **Understandability:** Patterns that are interpretable and meaningful to humans.

In the KDD process, choosing the data mining task is critical. Depending on the objective, the task could involve classification, regression, clustering, or association rule mining. After determining the task, selecting the appropriate data mining algorithms is essential. These algorithms are chosen based on their ability to efficiently and accurately identify patterns that align with the goals of the analysis.

Evaluation and Interpretation of Results

Evaluation in KDD involves assessing the patterns identified during data mining to determine their relevance and usefulness. It includes calculating the “interestingness score” for each pattern, which helps to identify valuable insights. Visualization and summarization techniques are then applied to make the data more understandable and accessible for the user.

Interpretation of Results focuses on presenting these insights in a way that is meaningful and actionable. By effectively communicating the findings, decision-makers can use the results to drive informed actions and strategies.

Practical Example of KDD

Let’s assume a scenario where a fitness center wants to improve member retention by analyzing usage patterns.

- **Data Selection:** The fitness center gathers data from its membership system, focusing on the past six months of activity. They filter out inactive members and focus on those with regular usage.
- **Data Cleaning and Preprocessing:** The fitness center cleans the data by eliminating duplicates and correcting missing information, such as incomplete workout records or member details. They also handle any gaps in data by filling in missing values based on previous patterns.
- **Data Transformation and Reduction:** The data is transformed to highlight important metrics, such as the average number of visits per week per member and their most frequently chosen workout types. Dimensionality reduction is applied to focus on the most significant factors like membership duration and gym attendance frequency.

- **Data Mining:** By applying clustering algorithms, the fitness center segments members into groups based on their usage patterns. These segments include frequent visitors, occasional users, and those with minimal attendance.
- **Evaluation and Interpretation of Results:** The fitness center evaluates the groups by examining their retention rates. They find that occasional users are more likely to cancel their memberships. The interpretation reveals that members who visit the gym less than once a week are at a higher risk of discontinuing their membership.

This analysis helps the fitness center implement effective retention strategies, such as offering tailored incentives and creating engagement programs aimed at boosting the activity of occasional users.

Difference between KDD and Data Mining

Parameter	KDD	Data Mining
Definition	The overall process of discovering valid, novel, potentially useful, and ultimately understandable patterns and relationships in large datasets.	A subset of KDD, focused on the extraction of useful patterns and insights from large datasets.
Objective	To extract valuable knowledge and insights from data to support decision-making and understanding.	To identify patterns, relationships, and trends within data to generate useful insights.
Techniques Used	Involves multiple steps such as data cleaning, data integration, data selection, data transformation, data mining, pattern evaluation, and knowledge representation.	Includes techniques like association rules, classification, clustering, regression, decision trees, neural networks, and dimensionality reduction.
Output	Structured knowledge in the form of rules, models, and insights that can aid in decision-making or predictions.	Patterns, relationships, or associations that can improve understanding or decision-making.
Focus	Focuses on the discovery of useful knowledge, with an emphasis on interpreting and validating the findings.	Focuses on discovering patterns, relationships, and trends within data without necessarily considering the broader context.
Role of Domain Expertise	Important for defining goals, selecting data, and interpreting results.	Less critical, as focus is on algorithmic pattern detection.

Table 1.1: Comparison between KDD and Data Mining

1.2 Structured and Unstructured Data and Their Examples

Data Types in the Real World

Understanding the type of data is essential for choosing the appropriate tools and techniques.

Structured Data

Structured data is highly organized information that fits into predefined formats, such as tables with rows and columns, making it easy to store, search, and analyze using tools like SQL or spreadsheet software. It typically includes both numeric and short text fields and is often used for transactional, operational, and analytical purposes.

Characteristics:

- Fixed schema (predefined structure)
- Stored in tabular formats (databases, spreadsheets)
- Easily searchable and analyzable
- Quantitative or categorical in nature

Examples of Structured Data:

- Dates and times (e.g., transaction timestamps)
- Customer names, phone numbers, addresses, email addresses
- Product prices, serial numbers, inventory counts
- Social security numbers, banking/transaction information
- Employee records, ZIP codes, credit card numbers

Common Sources:

- Hotel reservation systems
- Point-of-sale (POS) and online transaction processing (OLTP) systems

- Customer Relationship Management (CRM) and Enterprise Resource Planning (ERP) systems
- Financial data warehouses
- Online forms and surveys
- Medical devices that generate tabular logs

Unstructured Data

Unstructured data lacks a predefined model or organization and does not fit neatly into relational tables. It is often text-heavy, multimedia, or complex, requiring advanced analytics techniques such as natural language processing or machine learning to extract insights.

Characteristics:

- No fixed schema or structure
- Stored in native or original formats (text, images, audio, video)
- Difficult to search, categorize, or analyze with traditional tools
- Qualitative and often subjective in nature

Examples of Unstructured Data:

- Text documents and files (Word, PDF)
- Email messages (the content/body)
- Social media posts, comments, and chat transcripts
- Audio and video files (recordings, surveillance, podcasts)
- Images and digital photographs
- Web pages, HTML content, blogs, product reviews
- Sensor data and computer logs in raw format

Common Sources:

- Email clients and messaging systems (e.g., WhatsApp, SMS)
- Social media platforms
- Websites and file logs
- Word processing and presentation software
- Media viewing and editing tools
- GPS and weather satellite feeds

Comparison Table

Feature	Structured Data	Unstructured Data
Format	Predefined schema (tables, rows, columns)	No predefined format; native file formats
Storage	Databases, spreadsheets	Data lakes, file systems
Searchability	Easily searchable (SQL, queries)	Hard to search; requires advanced tools
Examples	Names, dates, prices, IDs, transactions	Emails, images, videos, social posts
Sources	POS, CRM, ERP, forms, sensors	Email, social media, web, multimedia files
Nature	Quantitative, categorical	Qualitative, subjective
Analysis	Traditional analytics, BI tools	NLP, ML, AI, advanced analytics

Table 1.2: Comparison of Structured and Unstructured Data

Real-World Example

In an email, the sender, recipient, and timestamp are structured data, while the body of the email is unstructured data.

Summary

Structured data is ideal for transactional and operational tasks due to its organization and ease of analysis. Unstructured data, while harder to manage, holds valuable insights—especially when analyzed with modern AI and machine learning techniques. Most organizations today leverage both types to gain a comprehensive understanding of their operations and customer behavior.

1.3 Overview of Data Preprocessing

Data preprocessing is a foundational step in the data analysis pipeline that ensures raw, often chaotic datasets are converted into structured, high-quality data suitable for analysis and machine learning. Without proper preprocessing, even the most advanced analytical models may yield inaccurate, biased, or misleading results. The goal of preprocessing is to enhance data quality and compatibility, improve model performance, and reduce computational overhead.

Data preprocessing typically includes four key processes:

- **Data Cleaning**
- **Data Integration**
- **Data Transformation**
- **Data Discretization**

Each of these processes plays a specific role in preparing the data for robust and accurate analysis.

1.3.1 Data Cleaning

Data Cleaning is the process of identifying and rectifying errors, inconsistencies, and inaccuracies in a dataset. It is the first step in preprocessing because analysis performed on flawed data will lead to flawed conclusions.

1. Missing Data Handling

Missing values can occur due to equipment malfunctions, user omissions, data corruption, or differences in data collection policies. These gaps can distort statistical metrics and machine learning model training.

Techniques for handling missing data:

- **Deletion Methods:**
 - *Listwise Deletion:* Remove entire records with missing values (can reduce dataset size drastically).

- *Pairwise Deletion*: Only exclude data points when they affect the current computation.

- **Imputation Methods:**

- *Mean/Median/Mode Substitution*: Simple, but can bias data distributions.
- *Regression Imputation*: Predict the missing value using linear or logistic regression.
- *K-Nearest Neighbors (KNN) Imputation*: Use proximity in feature space to estimate missing values.
- *Multiple Imputation*: Creates several imputed datasets and combines their results for better statistical integrity.

2. Noise and Outlier Detection

Noisy data includes random errors or variances that don't reflect true values, while outliers are extreme deviations that can skew models.

Approaches to handle noise:

- *Binning*: Grouping values into intervals and smoothing within bins.
- *Regression*: Fit a function (linear/non-linear) and smooth out deviations.
- *Moving Average Smoothing*: Useful in time-series data to reduce variance.

Outlier detection methods:

- Statistical tests (e.g., Z-score, IQR)
- Clustering techniques (e.g., DBSCAN)
- Distance-based methods

3. Inconsistent Data

Inconsistencies arise due to different formats, units, or naming conventions across data entries.

Fixing strategies:

- Standardizing units (e.g., converting currencies or measurement units)
- Harmonizing formats (e.g., DD-MM-YYYY to YYYY-MM-DD)
- Resolving naming conflicts using dictionaries or metadata (e.g., “NY” = “New York”)

4. Duplicate Detection

Duplicates may arise during data entry or collection. They can lead to overrepresentation of certain entities.

Deduplication techniques:

- Exact matching of fields (IDs, names)
- Fuzzy string matching (Levenshtein distance, Soundex)
- Entity Resolution using supervised or unsupervised ML algorithms

1.3.2 Data Integration

Data Integration involves merging data from multiple heterogeneous sources (e.g., databases, sensors, APIs, surveys) into a unified, consistent dataset. This process is especially critical in big data environments where data silos are common.

1. Schema Integration

When data from different sources have different schemas (structure, field names), aligning them becomes necessary.

Techniques include:

- Schema Matching: Identifying semantically similar fields (e.g., “CustomerID” and “ClientID”)

- Ontology-based mapping: Using a shared vocabulary or metadata standard to align schemas
- Use of tools like Apache Nifi, Talend, or custom SQL/ETL scripts

2. Entity Resolution

Also called record linkage or deduplication across sources.

Techniques:

- Rule-based matching (manual rules to compare key attributes)
- Machine learning models trained on labeled pairs (supervised)
- Unsupervised clustering (e.g., hierarchical clustering)

3. Data Redundancy and Conflict Resolution

Redundancies (repeated information) and conflicts (e.g., different addresses for the same customer) are resolved through:

- Priority rules (e.g., trust data from source A over B)
- Data fusion (e.g., taking an average or latest value)
- Source reliability scoring

4. Data Warehousing and ETL

- **ETL (Extract, Transform, Load)** is a key method in data integration pipelines. Data is extracted from sources, transformed for consistency, and loaded into centralized storage (data warehouse or lake).
- Integration frameworks like **Apache Kafka**, **Apache Airflow**, or **Informatica** help automate this.

1.3.3 Data Transformation and Discretization

Data transformation reshapes data into forms that are optimal for analysis, while discretization converts continuous attributes into categorical values.

1. Data Transformation

Data transformation may involve:

- **Normalization:** Rescaling data to fit within a specific range, such as $[0, 1]$.

- *Min-Max Normalization:*

$$x' = \frac{x - \min(X)}{\max(X) - \min(X)}$$

- *Z-score Normalization:*

$$x' = \frac{x - \mu}{\sigma}$$

- **Standardization:** Ensures features have zero mean and unit variance, especially useful for algorithms sensitive to scale (e.g., SVM, PCA).
- **Log Transformations:** Reduce skewness in data (e.g., log-income).
- **Encoding Categorical Variables:**
 - *One-hot encoding:* Creates binary variables for each category.
 - *Label encoding:* Assigns integers to categories (ordinal features).
 - *Binary or hash encoding:* Compact encoding for high-cardinality variables.
- **Feature Construction:** Creating new variables from existing ones.
 - Example: From date-of-birth, derive “Age”; from GPS data, derive “Distance”
- **Aggregation:** Summarizing data (e.g., average sales per region per month).

2. Data Discretization

Discretization is the process of dividing continuous attributes into distinct intervals or "bins." It is useful when:

- Building decision trees (which split on categorical conditions)

- Reducing model complexity
- Improving interpretability

Techniques include:

- *Equal-width binning*: All intervals have the same range.
 - May produce empty bins if data is skewed.
- *Equal-frequency binning*: Each bin has the same number of records.
 - Maintains balance across bins.
- *Cluster-based binning*: Use clustering (e.g., k-means) to define bins based on similarity.
- *Entropy-based methods*:
 - Used in algorithms like ID3, C4.5
 - Splits the data at points that minimize entropy (maximize information gain)

Supervised vs. Unsupervised Discretization:

- *Supervised*: Considers class labels (target variable) while discretizing
- *Unsupervised*: Only considers distribution of feature values

CHAPTER TWO: STATISTICAL FOUNDATION

2.1 Types of variable Numerical and Categorical

In data analysis, understanding the types of variables is fundamental because it determines the kinds of analytical techniques and statistical methods that can be applied.

Two major types:

- **Numeric Variables**
- **Categorical Variables**

Numeric Variables

Definition: Numeric variables, sometimes referred to as quantitative variables, represent measurable quantities. They are expressed with numbers and can be subjected to meaningful mathematical operations such as addition, subtraction, averaging, or finding variance.

Subtypes:

- Continuous Variables
- Discrete Variables

Continuous Variables

Definition: Variables that can take any value within a range, with infinite possible values between two numbers.

Examples:

- Height (e.g., 170.5 cm)
- Weight (e.g., 65.8 kg)
- Temperature (e.g., 36.6°C)
- Time (e.g., 2.75 seconds)

Characteristics:

- Infinite number of values within a range
- Measured on scales such as interval or ratio
- Precision depends on the measurement tool

Operations: All arithmetic operations are meaningful (sum, mean, variance, etc.)

Discrete Variables

Definition: Discrete variables can take only specific, distinct values, typically representing counts.

Examples:

- Number of students in a class (e.g., 30)
- Number of cars in a parking lot (e.g., 15)
- Number of defects on a manufactured item (e.g., 2)

Characteristics:

- Values are countable and finite (or countably infinite)
- Cannot take partial values

Operations: Counting and summing are meaningful. Averaging may sometimes produce non-intuitive results.

Categorical Variables

Definition: Also called qualitative variables, they represent characteristics, attributes, or labels rather than measurements. They describe qualities and group observations into categories.

Subtypes:

- Nominal Variables

- Ordinal Variables

Nominal Variables

Definition: Classify data into distinct categories where no order or ranking is implied.

Examples:

- Gender (Male, Female, Other)
- Nationality (Nepalese, Indian, American)
- Blood Type (A, B, AB, O)

Characteristics:

- Categories are mutually exclusive
- No intrinsic ordering between categories
- Often encoded with labels or numbers (e.g., 0 for Male, 1 for Female) without quantitative meaning

Operations: Only equality or inequality comparisons are valid.

Ordinal Variables

Definition: Classify data into categories that can be ranked or ordered.

Examples:

- Educational Level (Primary, Secondary, Tertiary)
- Customer Satisfaction (Very Dissatisfied to Very Satisfied)
- Socio-economic Status (Low, Middle, High)

Characteristics:

- Categories have a logical order

- Distance between categories is not uniform

Operations: Comparisons (greater than, less than) are meaningful, but exact arithmetic operations are not always valid.

Visual Summary

Type	Subtype	Example	Characteristics
Numeric	Continuous	Temperature	Infinite values in a range
Numeric	Discrete	Number of cars	Specific countable values
Categorical	Nominal	Blood Type	No inherent order
Categorical	Ordinal	Education Level	Ordered categories

Why Does Variable Type Matter?

The type of variable dictates:

- **Choice of Statistical Methods**
 - Use mean and standard deviation for continuous numeric data.
 - Use mode and frequency for nominal data.
- **Preprocessing Techniques**
 - Normalization for continuous variables.
 - One-hot encoding for nominal variables.
- **Visualization Methods**
 - Histograms for numeric data.
 - Bar charts for categorical data.
- **Model Selection**
 - Logistic regression handles categorical outcomes.
 - Linear regression requires numeric outcomes.

Beware of Numeric-Looking Categorical Variables

Some variables can appear numeric but are actually categorical:

- **Zip Codes** (e.g., 44600, 10001): Numbers without arithmetic meaning; they are nominal categorical variables.
- **Rating Scales** (e.g., 1 to 5 stars): Represent ordinal variables; use with caution in numeric analysis.

Understanding the nature of a variable—not just its format—is essential for proper data analysis.

2.2 Empirical Distribution

Empirical distributions describe how observed data is distributed, without assuming an underlying theoretical distribution. Understanding empirical distributions is crucial for visualizing and analyzing the behavior of both numeric and categorical data.

2.2.1 Numeric Data

Empirical distribution of numeric data can be visualized and analyzed using various statistical tools and probability models:

- **Histograms:**
 - A graphical representation of the frequency distribution of continuous numeric data.
 - Data is divided into intervals (bins), and the frequency of data points in each bin is plotted.

When to Use a Histogram

A histogram is a powerful graphical tool used to summarize and understand the distribution of numerical data. It is particularly useful in the following situations:

- **The data are numerical:** Histograms are suitable only for quantitative (numeric) data.
- **You want to see the shape of the data's distribution:** Especially useful to determine whether the data follow a normal distribution or are skewed.
- **Analyzing process capability:** Helpful in evaluating whether a process can consistently meet customer requirements.
- **Analyzing supplier output:** Used to understand what the output from a supplier's process looks like.
- **Monitoring process changes over time:** Histograms can show whether a process has changed from one time period to another.
- **Comparing multiple processes:** Useful in determining whether the outputs of two or more processes are significantly different.
- **Effective communication:** Allows for quick and clear communication of the distribution of data to others.

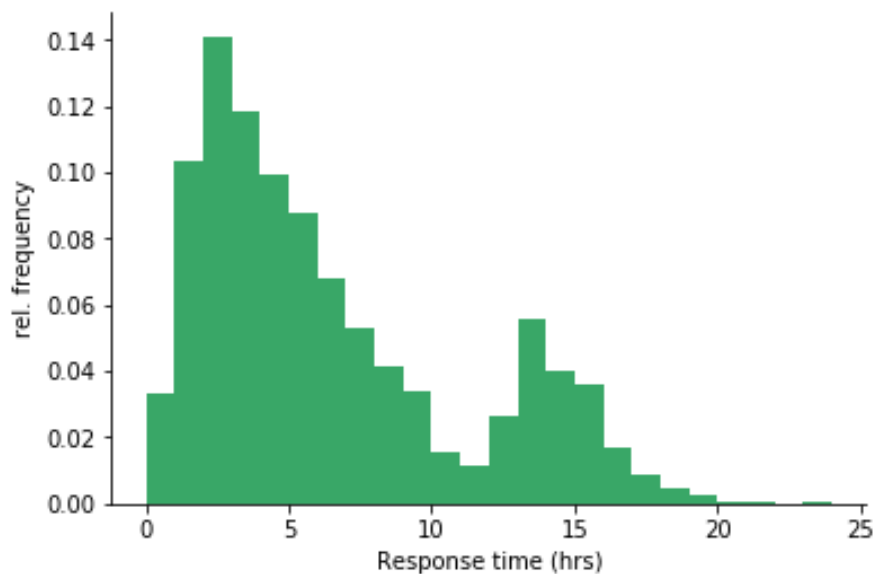


Figure 2.1: Histogram plot

- **Normal Distribution:**

What is Normal Distribution?

The **Normal Distribution**, also known as the **Gaussian distribution**, is a probability distribution that is widely used to model real-world numerical data. It has the following characteristics:

- Bell-shaped curve
- Symmetrical around the mean
- Mean (μ), median, and mode are equal
- Defined by two parameters:
 - * Mean (μ): the center of the distribution
 - * Standard deviation (σ): the spread of the data

Mathematical Formula

The probability density function (PDF) of the normal distribution is given by:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where:

- x = value of the random variable
- μ = mean
- σ = standard deviation
- $e \approx 2.718$

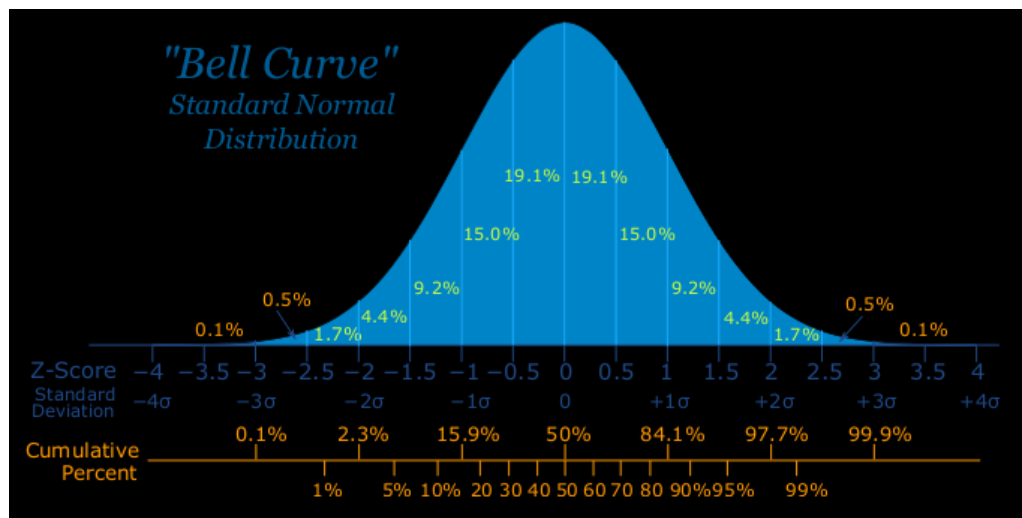
Why is it Important?

Many types of numerical data in nature, economics, engineering, and social sciences follow or approximate a normal distribution, such as:

- Heights of people
- Test scores
- Measurement errors
- Blood pressure

Key Properties

Property	Description
Shape	Bell-curve, symmetrical
Central tendency	Mean = Median = Mode
Area under the curve	Always equals 1
Empirical Rule	$\sim 68\%$ within 1σ $\sim 95\%$ within 2σ $\sim 99.7\%$ within 3σ



How to Check for Normality?

1. Histogram
2. Q-Q Plot (Quantile-Quantile Plot)
3. Shapiro-Wilk Test
4. Kolmogorov-Smirnov Test

5. Skewness and Kurtosis

- **Empirical Probability Distribution:**

- Assigns probabilities based on observed relative frequencies in data.
- For a value x_i , empirical probability $P(x_i) = \frac{\text{frequency of } x_i}{\text{total number of observations}}$.

- **Exponential Distribution:**

What is the Exponential Distribution?

The **Exponential Distribution** is a continuous probability distribution used to model the **time between independent events** that happen at a **constant average rate**.

It is commonly used in fields such as:

- Reliability engineering (time until failure)
- Queueing theory (time between arrivals)
- Survival analysis (time until death or failure)
- Telecommunications (time between packet arrivals)

When to Use It?

Use the Exponential Distribution when:

- Events occur continuously and independently
- Events follow a Poisson process (i.e., fixed average rate)
- You want to model waiting time or time until an event occurs

Probability Density Function (PDF)

Let X be a continuous random variable following an exponential distribution with rate parameter $\lambda > 0$. The probability density function (PDF) is:

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x \geq 0 \\ 0 & \text{for } x < 0 \end{cases}$$

Where:

- λ is the **rate parameter** (average number of events per time unit)
- x is the **time or distance between events**
- $e \approx 2.718$

Cumulative Distribution Function (CDF)

$$F(x) = \begin{cases} 1 - e^{-\lambda x} & \text{for } x \geq 0 \\ 0 & \text{for } x < 0 \end{cases}$$

It gives the **probability that the event occurs within time x** .

Key Properties

Property	Value
Mean	$\mu = \frac{1}{\lambda}$
Variance	$\sigma^2 = \frac{1}{\lambda^2}$
Memoryless Property	$P(X > s + t \mid X > s) = P(X > t)$
Skewness	2
Kurtosis	6

Memoryless Property

The exponential distribution is the **only continuous distribution that is memoryless**. This means:

The probability of an event occurring in the next t time units does not depend on how much time has already elapsed.

Mathematically:

$$P(X > s + t \mid X > s) = P(X > t)$$

Example

Suppose a server receives requests at an average rate of 2 requests per minute (i.e., $\lambda = 2$).

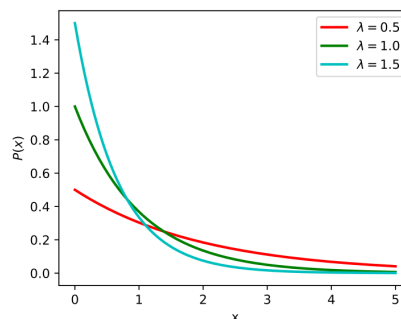
- The mean time between requests is:

$$\frac{1}{\lambda} = \frac{1}{2} = 0.5 \text{ minutes}$$

- The probability that the next request arrives within 30 seconds (0.5 minutes) is:

$$P(X \leq 0.5) = 1 - e^{-\lambda x} = 1 - e^{-2 \cdot 0.5} = 1 - e^{-1} \approx 0.6321$$

So, there is about a **63.21% chance** that a request arrives within 30 seconds.



Plot Shape

- Right-skewed
- Starts at a maximum at $x = 0$
- Decreases exponentially

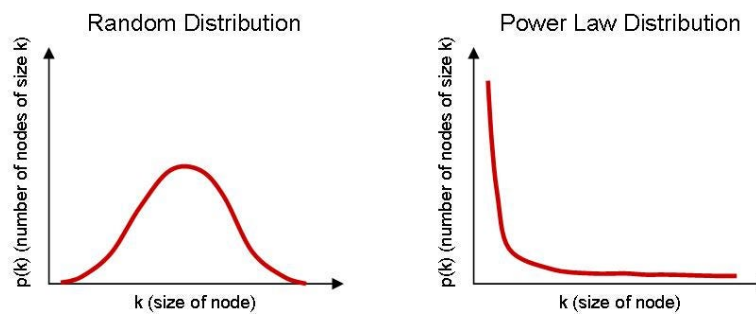
Real-Life Applications

- Time until a radioactive particle decays

- Life span of a machine part (without aging)
- Time between arrivals at a hospital
- Time until the next phone call

- **Power Law Distribution:**

- Heavy-tailed distribution, often used to describe phenomena where a few occurrences are very common, but many are rare (e.g., earthquake magnitudes, wealth distribution).
- Follows $P(x) \propto x^{-\alpha}$, where $\alpha > 1$.



What is a Power Law?

A **Power Law** is a functional relationship where a quantity varies as a power of another. Mathematically, a power-law distribution has the form:

$$P(x) = Cx^{-\alpha}$$

Where:

- x is the variable (e.g., size, frequency)
- $\alpha > 1$ is the **scaling exponent**
- C is the **normalization constant**

Key Characteristics

Property	Description
Heavy tail	High values (rare events) are more probable than in exponential or normal distributions
Scale-invariant	Multiplying x by a constant changes the function by a constant factor
Long-tailed	The distribution decays slowly
No characteristic scale	Power law lacks a typical size — small and large events coexist

When Does It Arise?

Power-law distributions appear in systems with self-organizing behavior or preferential attachment. Examples include:

- Earthquakes (Richter scale)
- City population sizes
- Wealth distribution (Pareto law)
- Word frequencies in languages (Zipf's law)
- Internet (number of links per website)
- Social networks (followers per user)

Comparison with Other Distributions

Distribution	Tail Behavior	Common Use Case
Normal	Decays rapidly	Heights, IQ, test scores
Exponential	Decays moderately fast	Time between Poisson events
Power Law	Decays slowly	Rare large events (e.g., blackouts, floods)

Mathematical Form

The probability density function (PDF) for the continuous power-law distribution is:

$$f(x) = \frac{\alpha - 1}{x_{\min}} \left(\frac{x}{x_{\min}} \right)^{-\alpha} \quad \text{for } x \geq x_{\min}$$

Where:

- α is the exponent (usually $\alpha > 1$)
- $x_{\min}$ is the lower bound beyond which the power law applies

Interpretation of α

- $\alpha \approx 2$: moderate inequality
- $\alpha \rightarrow 1$: extremely unequal, heavy tail
- The **lower** the α , the **heavier** the tail of the distribution

Log-Log Plot

If data follows a power law, a log-log plot of frequency versus value yields a straight line, since:

$$\log P(x) = \log C - \alpha \log x$$

This helps visually verify whether a dataset follows power-law behavior.

Cautions in Usage

- Not all heavy-tailed distributions are power laws.
- Large datasets are needed for accurate fitting.
- Proper estimation of α is essential (e.g., using *Maximum Likelihood Estimation*).
- Visual inspection (e.g., log-log plots) is not sufficient alone.
- Use statistical tests (e.g., Kolmogorov-Smirnov test) to confirm.

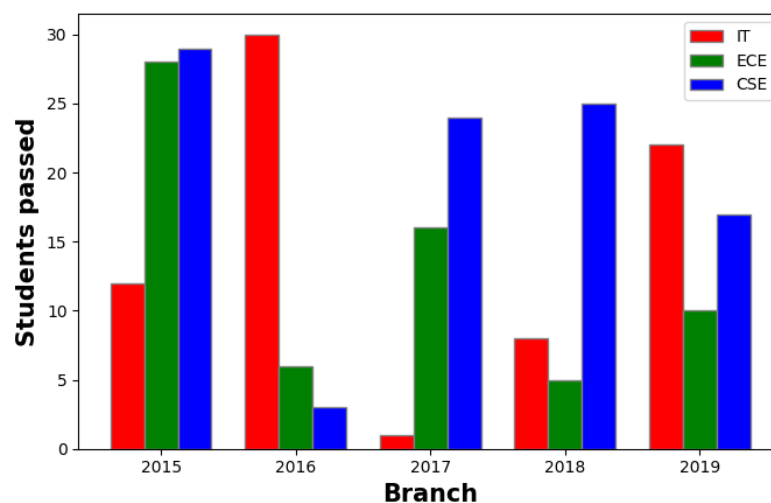
Summary

- Power laws model extreme inequality and rare, high-impact events.
- They describe many natural and man-made systems.
- Mathematically simple but statistically subtle.

2.2.2 Categorical Data

Empirical analysis of categorical data involves understanding the frequency and distribution of categories:

- **Bar Plots:**
 - Visual representation of frequency or proportion of categories.
 - Each bar represents a category, and its height shows the count or relative frequency.



What is Bar Plot

Bar plots are one of the most common and effective ways to visualize **categorical data**. They display **counts or frequencies** (or any summary metric) for each category using bars.

What is Categorical Data?

Categorical data represents **discrete groups or labels**, such as:

- Gender: Male, Female
- Colors: Red, Green, Blue
- Grades: A, B, C, D, F

Unlike numerical data, categorical values do **not have inherent numerical meaning**, though they may be numerically coded.

Bar Plot Basics

A bar plot visualizes categorical data by showing:

- **x-axis:** distinct categories
- **y-axis:** value for each category (usually count or percentage)
- **Bars:** height (or length) represents the magnitude for each category

When to Use Bar Plots?

Bar plots are useful:

- For categorical or ordinal data
- To compare frequencies or summary statistics (e.g., mean, sum) across groups
- When the number of categories is **limited and manageable**

Do Not Use Bar Plots For:

Avoid using bar plots in the following cases:

- For **continuous numerical data** (use histograms or box plots instead)

- For **too many categories** (can be overcrowded; use pie charts, treemaps, or summarized plots instead)
- **Binomial Variables:**
 - A special case of categorical data with two possible outcomes (e.g., success/failure).
 - Follows a binomial distribution: $P(k; n, p) = \binom{n}{k} p^k (1-p)^{n-k}$.
 - Used in experiments with a fixed number of independent trials.

What is the Binomial Distribution?

The **Binomial Distribution** is a discrete probability distribution that models the number of **successes** in a fixed number of independent trials, where each trial has only two possible outcomes: **success or failure**.

Conditions for Binomial Distribution

A random experiment follows a Binomial Distribution if:

- The number of trials, n , is fixed.
- Each trial is independent.
- Each trial has only two outcomes: Success (S) or Failure (F).
- The probability of success p remains constant in all trials.

Probability Mass Function (PMF)

The probability of getting exactly k successes in n independent trials is given by:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Where:

- n : number of trials
- k : number of successes, where $0 \leq k \leq n$
- p : probability of success
- $1 - p$: probability of failure
- $\binom{n}{k} = \frac{n!}{k!(n-k)!}$: number of combinations

Mean, Variance, and Standard Deviation

Measure	Formula
Mean	$\mu = np$
Variance	$\sigma^2 = np(1-p)$
Standard Deviation	$\sigma = \sqrt{np(1-p)}$

Example

Let's say:

- $n = 10$: number of coin tosses
- $p = 0.5$: probability of getting heads (success)

Then the probability of getting exactly $k = 6$ heads is:

$$P(X = 6) = \binom{10}{6} (0.5)^6 (0.5)^4 = \frac{10!}{6!4!} \cdot (0.5)^{10} \approx 0.205$$

So there is about a 20.5% chance of getting exactly 6 heads.

Graph Shape

The shape of the binomial distribution depends on p :

- Symmetrical if $p = 0.5$
- Skewed right if $p < 0.5$

- Skewed left if $p > 0.5$

Common Use Cases

- Number of heads in coin tosses
- Number of defective items in a batch
- Number of correct answers in a multiple-choice quiz
- Success/failure modeling in experiments

Notes

- The binomial distribution is only applicable for discrete trials.
 - If n is large and p is not too close to 0 or 1, the **normal distribution** can be used as an approximation.
- **Zipf's Law:**
 - A type of power law commonly found in language, city populations, and website visits.
 - Rank-frequency distribution: the frequency of an item is inversely proportional to its rank in a frequency table.
 - For rank r : $f(r) \propto \frac{1}{r^s}$, typically with $s \approx 1$.

What is Zipf's Law?

Zipf's Law is an empirical law that describes the frequency distribution of words in a natural language, or more generally, the **rank-frequency relation** in many types of datasets.

Mathematically, Zipf's Law states that:

$$f(r) \propto \frac{1}{r^s}$$

Where:

- $f(r)$: frequency of the item ranked r
- r : rank of the item (with 1 being the most frequent)
- s : exponent close to 1 (usually $s \approx 1$)

The symbol $\propto$ means “proportional to.”

Interpretation

Zipf’s Law implies that:

- The 2nd most frequent item appears about $\frac{1}{2}$ as often as the most frequent.
- The 3rd most frequent item appears about $\frac{1}{3}$ as often.
- And so on.

This leads to a **long-tailed distribution** — a few items are very common, while most are rare.

Where Zipf’s Law Appears

Zipf’s Law appears in many real-world datasets, including:

- Word frequencies in natural languages (English, Nepali, etc.)
- City population sizes
- Web page visits
- Book sales (bestsellers)
- Social media followers

Zipf’s Law as a Power Law

Zipf’s Law is a special case of a power-law distribution, where the variable of interest is **ranked**:

$$f(r) = \frac{C}{r^s}$$

Where:

- C : normalization constant

If you plot $f(r)$ against r on a **log-log scale**, the data will appear approximately as a **straight line** with slope $-s$.

Example (Word Frequencies)

In a large English corpus:

- **the** is the most common word (rank 1)
- **of** is second (rank 2)
- **and** is third (rank 3)

If **the** appears 10,000 times:

- **of** will appear around 5,000 times
- **and** will appear around 3,333 times

Applications

Zipf's Law is useful in:

- **Linguistics**: understanding language structure
- **Data science**: predicting frequencies in ranked data
- **Information retrieval**: keyword weighting (e.g., TF-IDF)
- **Economics**: analyzing inequality and distributions
- **AI and NLP**: language modeling and token frequency analysis

Limitations

- Zipf's Law is empirical: it fits real-world data approximately, not exactly.
- It performs better on **large datasets**.
- The exponent s may vary depending on the dataset and domain.

2.3 Correlation Analysis

2.3.1 Pearson Correlation Coefficient (r)

Definition: Measures the linear relationship between two numeric variables. Range: -1 (perfect negative) to $+1$ (perfect positive).

Key Formulas

Standard Formula:

$$r = \frac{\text{cov}(x, y)}{\sigma_x \sigma_y}$$

where:

- $\text{cov}(x, y)$: Covariance of x and y
- σ_x, σ_y : Standard deviations of x and y

Alternative Formula:

$$r = \frac{n \sum xy - \sum x \sum y}{\sqrt{[n \sum x^2 - (\sum x)^2][n \sum y^2 - (\sum y)^2]}}$$

Assumptions

- **Normality:** Both variables should be approximately normally distributed
- **Linearity:** Relationship should be linear (check via scatterplot)
- **Homoscedasticity:** Equal variance across data points
- **No outliers:** Extreme values can distort correlation

Interpretation

Strength of Correlation:

- $|r| \geq 0.7$: Very strong
- $0.5 \leq |r| < 0.7$: Strong
- $0.3 \leq |r| < 0.5$: Moderate
- $|r| < 0.3$: Weak or no correlation

Direction:

- **Positive** ($r > 0$): As x increases, y increases
- **Negative** ($r < 0$): As x increases, y decreases

Example Calculation

Data (5 individuals):

Age (x)	Salary (y, in \$1,000s)
25	30
32	45
40	55
48	65
55	75

Step 1: Calculate Means

$$\bar{x} = \frac{25 + 32 + 40 + 48 + 55}{5} = 40$$
$$\bar{y} = \frac{30 + 45 + 55 + 65 + 75}{5} = 54$$

Step 2: Deviations and Products

For $(x_1 = 25, y_1 = 30)$:

$$x_1 - \bar{x} = -15$$

$$y_1 - \bar{y} = -24$$

$$(x_1 - \bar{x})(y_1 - \bar{y}) = 360$$

Sum of products: $360 + 24 + 0 + 72 + 315 = 771$

Step 3: Covariance and Standard Deviations

$$\text{cov}(x, y) = \frac{771}{4} = 192.75$$

$$\sigma_x = \sqrt{\frac{618}{4}} \approx 12.45$$

$$\sigma_y = \sqrt{\frac{1204}{4}} \approx 17.36$$

Step 4: Calculate Pearson's r

$$r = \frac{192.75}{12.45 \times 17.36} \approx 0.89$$

Interpretation

$r = 0.89$ indicates very strong positive correlation between age and salary.

Visualization

Scatterplots in Python

```
import matplotlib.pyplot as plt
import pandas as pd
```

```
data = pd.DataFrame({
    'Age': [25, 32, 40, 48, 55],
    'Salary': [30, 45, 55, 65, 75]
})
```

```
plt.scatter(data['Age'], data['Salary'])
plt.xlabel('Age')
plt.ylabel('Salary ($1000s)')
plt.title('Scatterplot of Age vs. Salary')
plt.show()
```

2.3.2 Cross-Tabulation for Categorical Variables

Cross-tabulation (or **crosstab**) is a statistical method used to analyze the relationship between two or more categorical variables. It presents the data in a **contingency table**, showing the frequency distribution of variables.

When to Use Cross-Tabulation

- Compare proportions between groups.
- Examine associations between categorical variables.
- Identify patterns or dependencies in survey data.

How to Create a Cross-Tabulation Table

Suppose we have two categorical variables:

- **Gender** (Male, Female)
- **Preference** (Yes, No)

Table 2.1: Raw Data

Gender	Preference	Count
Male	Yes	30
Male	No	20
Female	Yes	40
Female	No	10

Contingency Table (Crosstab)

Table 2.2: Cross-Tabulation of Gender and Preference

	Yes	No	Total
Male	30	20	50
Female	40	10	50
Total	70	30	100

Interpreting Cross-Tabulation

1. **Frequency Distribution:** Shows how many cases fall into each combination.

- Example: 30 males answered "Yes."

2. Row/Column Percentages:

- **Row %:** % of each row (e.g., 60% of males said "Yes").
- **Column %:** % of each column (e.g., 42.9% of "Yes" responses were from males).

3. Chi-Square Test: Determines if the variables are independent.

- If $p < 0.05$, there's a significant association.

How to Perform Cross-Tabulation in Software

In Python (Pandas)

```
import pandas as pd

data = pd.DataFrame({
    'Gender': ['Male', 'Male', 'Female', 'Female'],
    'Preference': ['Yes', 'No', 'Yes', 'No'],
    'Count': [30, 20, 40, 10]
})

crosstab = pd.crosstab(data['Gender'], data['Preference'],
                        values=data['Count'], aggfunc='sum')
print(crosstab)
```

Common Statistics from Cross-Tabulation

- **Chi-Square Test** (tests independence between variables)
- **Cramer's V** (measures association strength, 0 = no association, 1 = strong)
- **Odds Ratio** (for 2×2 tables, compares likelihood between groups)

Limitations

- Only works for **categorical** (nominal/ordinal) variables.

- Large tables can be hard to interpret.
- Does not imply causation.

Conclusion

Cross-tabulation is a **simple yet powerful** way to explore relationships between categorical variables. It's widely used in **survey analysis, marketing research, and social sciences**.

Chi-Square Test of Independence

The **Chi-Square Test** (χ^2) evaluates whether two categorical variables are independent or associated in a contingency table.

Hypotheses

- **Null Hypothesis** (H_0): The variables are independent (no association).
- **Alternative Hypothesis** (H_1): The variables are dependent (association exists).

Test Statistic

The Chi-Square statistic is calculated as:

$$\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

where:

- O_{ij} = Observed frequency in cell (i, j)
- E_{ij} = Expected frequency under independence, computed as:

$$E_{ij} = \frac{(\text{Row Total}_i) \times (\text{Column Total}_j)}{\text{Grand Total}}$$

Degrees of Freedom

$$df = (r - 1)(c - 1)$$

where r = number of rows, c = number of columns.

Example

Given the cross-tabulation of *Gender* and *Preference*:

	Yes	No	Total
Male	30	20	50
Female	40	10	50
Total	70	30	100

Expected Frequencies

For Male/Yes:

$$E_{\text{Male, Yes}} = \frac{50 \times 70}{100} = 35$$

Similarly:

$$E_{\text{Male, No}} = 15, \quad E_{\text{Female, Yes}} = 35, \quad E_{\text{Female, No}} = 15$$

Chi-Square Calculation

$$\chi^2 = \frac{(30 - 35)^2}{35} + \frac{(20 - 15)^2}{15} + \frac{(40 - 35)^2}{35} + \frac{(10 - 15)^2}{15} = 9.52$$

With $df = 1$ and $\alpha = 0.05$, the critical value from χ^2 tables is **3.84**. Since $9.52 > 3.84$, we reject H_0 .

Assumptions

- Observations are independent.
- Expected frequency in each cell ≥ 5 (otherwise, use Fisher's Exact Test).

Effect Size

Measure association strength with:

- **Phi** (for 2×2 tables): $\phi = \sqrt{\frac{\chi^2}{n}}$
- **Cramér's V** (for larger tables): $V = \sqrt{\frac{\chi^2}{n \times \min(r-1, c-1)}}$

Conclusion

The Chi-Square Test determines if an observed association in a cross-tabulation is statistically significant. Always check assumptions and report effect sizes.

2.3.3 Assessing Correlation between numeric and categorical variable

Point-Biserial Correlation

The **Point-Biserial Correlation** (r_{pb}) is a measure of association between:

- A **continuous** variable (interval/ratio scale), and
- A **dichotomous** categorical variable (binary: 0/1, Yes/No, etc.).

When to Use

- Testing if a binary variable (e.g., Gender: Male/Female) relates to a continuous variable (e.g., Test Scores).
- An alternative to the independent samples t-test (provides equivalent statistical significance but also effect size).

Formula

The point-biserial correlation coefficient is calculated as:

$$r_{pb} = \frac{M_1 - M_0}{s} \sqrt{\frac{n_1 n_0}{n(n-1)}}$$

where:

- M_1 = Mean of the continuous variable for group 1 (coded as 1).
- M_0 = Mean of the continuous variable for group 0 (coded as 0).
- s = Standard deviation of the continuous variable (pooled).
- n_1, n_0 = Number of observations in groups 1 and 0.
- n = Total sample size ($n_1 + n_0$).

Interpretation

- Range: $-1 \leq r_{pb} \leq 1$.
- $r_{pb} = 0$: No association.

- $r_{pb} > 0$: Higher values of the continuous variable are associated with group 1.
- $r_{pb} < 0$: Higher values of the continuous variable are associated with group 0.
- Magnitude guidelines (Cohen, 1988):
 - Small: $|r_{pb}| \approx 0.1$
 - Medium: $|r_{pb}| \approx 0.3$
 - Large: $|r_{pb}| \approx 0.5$

Example

Suppose we have:

- Binary variable: *Pass* (1 = Pass, 0 = Fail).
- Continuous variable: *Study Hours*.

Table 2.3: Example Data

Pass (1/0)	Study Hours
1	12
1	15
1	18
0	5
0	8
0	6

Calculations

- $M_1 = \frac{12+15+18}{3} = 15$
- $M_0 = \frac{5+8+6}{3} = 6.33$
- $s = 5.12$ (pooled standard deviation)
- $n_1 = 3, n_0 = 3, n = 6$

$$r_{pb} = \frac{15 - 6.33}{5.12} \sqrt{\frac{3 \times 3}{6 \times 5}} = 0.82$$

Interpretation

A strong positive correlation ($r_{pb} = 0.82$) exists between passing and study hours.

Assumptions

- The continuous variable is normally distributed within each group (check via Shapiro-Wilk test).
- Homogeneity of variance across groups (check via Levene's test).

Relationship to Other Tests

- Equivalent to Pearson's r when one variable is dichotomous.
- Mathematically related to the independent samples t-test:

$$t = \frac{r_{pb}\sqrt{n-2}}{\sqrt{1-r_{pb}^2}}$$

Conclusion

The point-biserial correlation is useful for analyzing relationships between a binary variable and a continuous variable. It provides both effect size and direction of association.

ANOVA

Analysis of Variance (ANOVA) is a statistical method used to test differences between two or more means. It compares the variance between groups to the variance within groups to determine if the group means are statistically different.

Types of ANOVA

- **One-way ANOVA:** Compares means across one independent variable with three or more levels.
- **Two-way ANOVA:** Examines the effect of two independent variables.

- **Repeated Measures ANOVA:** Used when the same subjects are measured multiple times.

Assumptions

ANOVA requires:

- **Normality:** Data in each group should be normally distributed.
- **Homogeneity of variance:** Equal variances across groups.
- **Independence:** Observations are independent of each other.

Hypotheses

- **Null Hypothesis (H_0):** All group means are equal.

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$$

- **Alternative Hypothesis (H_1):** At least one group mean is different.

Calculations

The F-statistic is calculated as:

$$F = \frac{\text{Between-group variability}}{\text{Within-group variability}} = \frac{MS_{\text{between}}}{MS_{\text{within}}}$$

where:

- MS_{between} = Mean square between groups
- MS_{within} = Mean square within groups

Sum of Squares

$$SS_{\text{total}} = \sum_{i=1}^n (X_i - \bar{X})^2$$
$$SS_{\text{between}} = \sum_{j=1}^k n_j (\bar{X}_j - \bar{X})^2$$
$$SS_{\text{within}} = SS_{\text{total}} - SS_{\text{between}}$$

Degrees of Freedom

$$df_{\text{between}} = k - 1$$
$$df_{\text{within}} = N - k$$
$$df_{\text{total}} = N - 1$$

Example

Suppose we test three teaching methods (A, B, C) with the following test scores:

Table 2.4: Test Scores by Teaching Method

Method A	Method B	Method C
85	90	92
78	88	95
82	86	89
80	89	93

ANOVA Table

Source	SS	df	MS	F
Between	264.67	2	132.33	24.5
Within	48.75	9	5.42	
Total	313.42	11		

Interpretation

With $F(2,9) = 24.5$ and $p < 0.05$, we reject the null hypothesis, concluding that at least one teaching method differs significantly in effectiveness.

Post Hoc Tests

If ANOVA is significant, conduct post hoc tests (e.g., Tukey's HSD) to identify which specific groups differ.

Effect Size

Measure effect size using Eta-squared (η^2):

$$\eta^2 = \frac{SS_{\text{between}}}{SS_{\text{total}}}$$

Conclusion

ANOVA is a powerful tool for comparing means across multiple groups. Always check assumptions and consider effect sizes alongside significance tests.

2.4 Statistical Significance

What is Statistical Significance?

Statistical significance helps us determine whether an observed effect, difference, or relationship in data is likely due to a real effect rather than random chance.

In hypothesis testing, we start with a null hypothesis H_0 , which usually states that there is no effect or difference.

The goal is to use sample data to decide whether to reject H_0 or fail to reject H_0 .

A result is called statistically significant if it is unlikely to have happened under H_0 .

Significance Level α

The significance level α is the threshold probability for rejecting H_0 .

Common values: 0.05 (5%), 0.01 (1%).

If the probability of the observed data under H_0 is less than α , reject H_0 .

1. P-value

Definition

The p-value is the probability of obtaining test results at least as extreme as the observed results, assuming the null hypothesis is true.

Formally:

$$p = P(\text{data} \mid H_0)$$

Interpretation

- Small p-value (less than α): Strong evidence against H_0 , reject H_0 .
- Large p-value: Insufficient evidence to reject H_0 .

Misconceptions

- p-value is **not** the probability that H_0 is true.
- It does **not** measure the size or importance of an effect.
- It depends on sample size and test design.

2. T-value (in t-tests)

Purpose

The t-value is a test statistic used in the t-test, which compares the means of two groups or a sample mean against a known value when population variance is unknown.

Formula

For one-sample t-test:

$$t = \frac{\bar{x} - \mu_0}{s / \sqrt{n}}$$

Where:

- $\bar{x}$ = sample mean
- μ_0 = hypothesized population mean under H_0
- s = sample standard deviation
- n = sample size

Meaning

- The numerator measures the difference between observed and hypothesized means.
- The denominator (standard error) accounts for variability and sample size.
- Larger $|t|$ values indicate the observed mean is far from hypothesized mean relative to variability — evidence against H_0 .

Degrees of Freedom

Usually $df = n - 1$ for one-sample t-test.

Used to determine critical t-values and p-values from t-distribution tables.

3. Chi-Squared Test (χ^2 Test)

Purpose

Used to test hypotheses about:

- **Goodness-of-fit:** Does observed data fit a particular distribution?
- **Independence:** Are two categorical variables independent?

Formula for Test Statistic

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

Where:

- O_i = observed frequency count for category i

- E_i = expected frequency under H_0
- k = number of categories

Interpretation

- A large χ^2 value indicates a large difference between observed and expected data — evidence against H_0 .
- The distribution of χ^2 depends on degrees of freedom $df = k - 1$ (or more complex for independence tests).

Assumptions

- Samples are independent.
- Expected frequency in each category is sufficiently large (usually ≥ 5).

Summary Table

Concept	Description
Statistical Significance	Whether observed effect is unlikely by chance alone
P-value	Probability of observing data at least as extreme as observed under H_0
T-value	Standardized difference between sample mean and hypothesized mean
Chi-Squared Statistic	Measure of difference between observed and expected categorical frequencies

P-value Problems

Problem 1: One-Sample t-Test

Question: Given sample data [3.2, 3.5, 3.7, 3.9, 4.0] and population mean $\mu_0 = 3.0$, calculate the t-statistic and two-tailed p-value. Assume significance level $\alpha = 0.05$.

Solution:

1. Calculate sample mean ($\bar{x}$):

$$\bar{x} = \frac{\sum x_i}{n} = \frac{3.2 + 3.5 + 3.7 + 3.9 + 4.0}{5} = \frac{19.3}{5} = 3.86$$

2. Calculate sample standard deviation (s):

$$\begin{aligned} s &= \sqrt{\frac{\sum (x_i - \bar{x})^2}{n - 1}} \\ &= \sqrt{\frac{(3.2 - 3.86)^2 + (3.5 - 3.86)^2 + (3.7 - 3.86)^2 + (3.9 - 3.86)^2 + (4.0 - 3.86)^2}{4}} \\ &= \sqrt{\frac{0.4356 + 0.1296 + 0.0256 + 0.0016 + 0.0196}{4}} = \sqrt{\frac{0.612}{4}} = \sqrt{0.153} \approx 0.391 \end{aligned}$$

3. Calculate t-statistic:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} = \frac{3.86 - 3.0}{0.391/\sqrt{5}} = \frac{0.86}{0.1749} \approx 4.914$$

4. Degrees of freedom: $df = n - 1 = 4$
5. Two-tailed p-value (using t-distribution table/calculator):

$$p\text{-value} = 2 \times P(T > 4.914) \approx 0.008$$

Interpretation: Since $p < 0.05$, we reject H_0 at $\alpha = 0.05$.

Problem 2: Two-Sample t-Test

Question: Given Sample 1 [10, 12, 14, 15, 16] and Sample 2 [11, 12, 13, 14, 15], calculate the t-statistic and two-tailed p-value assuming equal variances ($\alpha = 0.05$).

Solution:

1. Calculate sample means:

$$\bar{x}_1 = \frac{10 + 12 + 14 + 15 + 16}{5} = 13.4, \quad \bar{x}_2 = \frac{11 + 12 + 13 + 14 + 15}{5} = 13.0$$

2. Calculate sample variances:

$$s_1^2 = \frac{\sum (x_i - \bar{x}_1)^2}{n_1 - 1} = \frac{(10 - 13.4)^2 + \dots + (16 - 13.4)^2}{4} = \frac{23.0}{4} = 5.75$$

$$s_2^2 = \frac{(11 - 13.0)^2 + \dots + (15 - 13.0)^2}{4} = \frac{10.0}{4} = 2.5$$

3. Calculate pooled variance:

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2} = \frac{4 \times 5.75 + 4 \times 2.5}{8} = \frac{33}{8} = 4.125$$

4. Calculate t-statistic:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_p^2 \left(\frac{1}{n_1} + \frac{1}{n_2} \right)}} = \frac{13.4 - 13.0}{\sqrt{4.125 \times \left(\frac{1}{5} + \frac{1}{5} \right)}} = \frac{0.4}{\sqrt{1.65}} \approx 0.312$$

5. Degrees of freedom: $df = n_1 + n_2 - 2 = 8$

6. Two-tailed p-value:

$$p\text{-value} \approx 0.76$$

Interpretation: Since $p > 0.05$, we fail to reject H_0 at $\alpha = 0.05$.

Problem 3: z-Test for Proportions

Question: A coin flipped 100 times shows 58 heads. Test fairness ($H_0 : p = 0.5$). Calculate z-statistic and two-tailed p-value ($\alpha = 0.05$).

Solution:

1. Sample proportion:

$$\hat{p} = \frac{58}{100} = 0.58$$

2. Calculate z-statistic:

$$z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}} = \frac{0.58 - 0.5}{\sqrt{\frac{0.5 \times 0.5}{100}}} = \frac{0.08}{0.05} = 1.6$$

3. Two-tailed p-value:

$$p\text{-value} = 2 \times P(Z > 1.6) = 2 \times 0.0548 = 0.1096$$

Interpretation: Since $p > 0.05$, we fail to reject H_0 at $\alpha = 0.05$.

T-value Problems

Problem 4: One-Sample t-Test

Question: Given $n = 25$, $\bar{x} = 8$, $s = 5$, and $\mu_0 = 10$, calculate the t-statistic.

Solution:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}} = \frac{8 - 10}{5/\sqrt{25}} = \frac{-2}{1} = -2$$

Problem 5: Two-Sample t-Test

Question: Given Sample 1 ($n_1 = 10$, $\bar{x}_1 = 50$, $s_1 = 5$) and Sample 2 ($n_2 = 12$, $\bar{x}_2 = 55$, $s_2 = 6$), calculate the t-statistic assuming equal variances.

Solution:

1. Calculate pooled variance:

$$s_p^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2} = \frac{9 \times 25 + 11 \times 36}{20} = \frac{225 + 396}{20} = 31.05$$

2. Calculate t-statistic:

$$t = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{s_p^2 \left(\frac{1}{n_1} + \frac{1}{n_2} \right)}} = \frac{50 - 55}{\sqrt{31.05 \left(\frac{1}{10} + \frac{1}{12} \right)}} = \frac{-5}{\sqrt{5.692}} \approx -2.095$$

χ^2 Problems

Problem 6: Goodness-of-Fit Test

Question: A die rolled 60 times shows frequencies [10, 8, 12, 15, 7, 8]. Test fairness (expected $E_i = 10$ for all faces). Calculate χ^2 statistic.

Solution:

$$\chi^2 = \sum_{i=1}^6 \frac{(O_i - E_i)^2}{E_i} = \frac{(10-10)^2}{10} + \frac{(8-10)^2}{10} + \frac{(12-10)^2}{10} + \frac{(15-10)^2}{10} + \frac{(7-10)^2}{10} + \frac{(8-10)^2}{10} = 4$$

Problem 7: Test of Independence

Question: Calculate χ^2 statistic for the contingency table below ($n = 100$):

	Yes	No
A	20	30
B	25	25

Solution:

1. Calculate expected frequencies:

$$E_{A,\text{Yes}} = \frac{(50)(45)}{100} = 22.5$$

$$E_{A,\text{No}} = \frac{(50)(55)}{100} = 27.5$$

$$E_{B,\text{Yes}} = \frac{(50)(45)}{100} = 22.5$$

$$E_{B,\text{No}} = \frac{(50)(55)}{100} = 27.5$$

2. Calculate χ^2 :

$$\begin{aligned}\chi^2 &= \frac{(20-22.5)^2}{22.5} + \frac{(30-27.5)^2}{27.5} + \frac{(25-22.5)^2}{22.5} + \frac{(25-27.5)^2}{27.5} \\ &= 0.2778 + 0.2273 + 0.2778 + 0.2273 = 1.0102\end{aligned}$$

Problem 8: Goodness-of-Fit Test

Question: Given observed counts [90, 30, 40, 10] and expected ratio 9 : 3 : 3 : 1 ($n = 170$), calculate χ^2 statistic.

Solution:

1. Calculate expected frequencies:

$$E_1 = \frac{9}{16} \times 170 = 95.625$$

$$E_2 = \frac{3}{16} \times 170 = 31.875$$

$$E_3 = \frac{3}{16} \times 170 = 31.875$$

$$E_4 = \frac{1}{16} \times 170 = 10.625$$

2. Calculate χ^2 :

$$\begin{aligned}\chi^2 &= \frac{(90 - 95.625)^2}{95.625} + \frac{(30 - 31.875)^2}{31.875} + \frac{(40 - 31.875)^2}{31.875} + \frac{(10 - 10.625)^2}{10.625} \\ &= 0.331 + 0.110 + 2.071 + 0.037 = 2.549\end{aligned}$$

Key Takeaways

- **p-value:** Probability of observing data as extreme as the sample under H_0 . Reject H_0 if $p \leq \alpha$.
- **t-value:** Standardized difference between sample statistic and H_0 parameter. Larger $|t|$ indicates stronger evidence against H_0 .
- **χ^2 statistic:** Measures discrepancy between observed and expected frequencies. Larger χ^2 indicates poorer fit or stronger association.

CHAPTER THREE: NUMERIC DATA

3.1 Introduction to Linear Regression

Linear regression is a powerful tool used to predict a continuous dependent variable based on one or more independent variables. It establishes a linear relationship between the dependent variable (target) and one or more independent variables (predictors). In our case, we are interested in predicting CO₂ emissions from a car based on its engine size, among other potential factors.

X: Independent variable				Y: Dependent variable
	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

Continuous Values

Regression is the process of predicting a continuous value

The general equation of linear regression is:

$$Y = \beta_0 + \beta_1 \cdot X_1$$

Where:

- Y is the dependent variable (CO₂ emissions),
- X_1 is the independent variable (engine size),
- β_0 is the intercept of the line (where the line crosses the Y-axis),
- β_1 is the slope of the line, indicating how much Y (CO₂ emissions) changes for each unit change in X_1 (engine size).

3.2 Types of Linear Regression

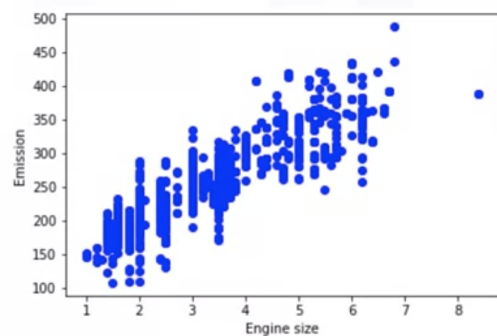
There are two types of linear regression:

- **Simple Linear Regression:** Uses a single independent variable (e.g., engine size) to predict the dependent variable (e.g., CO₂ emissions).
- **Multiple Linear Regression:** Uses more than one independent variable (e.g., engine size, number of cylinders) to predict the dependent variable.

In this example, we will focus on simple linear regression, where the goal is to predict CO₂ emissions using just engine size.

How does linear regression work?

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION, COMB	CO ₂ EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?



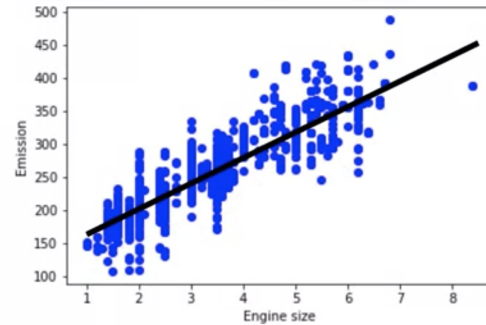
How Linear Regression Works

To understand how linear regression works, we begin by visualizing the relationship between the independent variable (engine size) and the dependent variable (CO₂ emissions). A scatter plot can help show how the two variables are related. In our case, we might observe that as the engine size increases, CO₂ emissions tend to increase as well, which suggests a positive linear relationship.

Scatter Plot: The plot shows individual data points representing various cars, with engine size on the X-axis and CO₂ emissions on the Y-axis. If the points seem to form a straight line or roughly follow a straight line, it indicates a linear relationship between the two variables.

With linear regression, we attempt to find the best-fitting straight line through this scatter of data points. The equation of the line can then be used to predict the CO₂ emissions for a car given its engine size.

	ENGINE SIZE	CYLINDERS	FUEL CONSUMPTION_COMB	CO2 EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?



Example: Predicting CO₂ Emissions for a Car with Engine Size 2.4 L

Let's assume that after plotting the data and fitting a regression line, we are ready to make predictions. Suppose we have a new car with an engine size of 2.4 L. Using our regression model, we can predict its CO₂ emissions.

Fitted Line (Equation): The equation of our line might look like:

$$\text{CO}_2 \text{ Emission} = \theta_0 + \theta_1 \cdot \text{Engine Size}$$

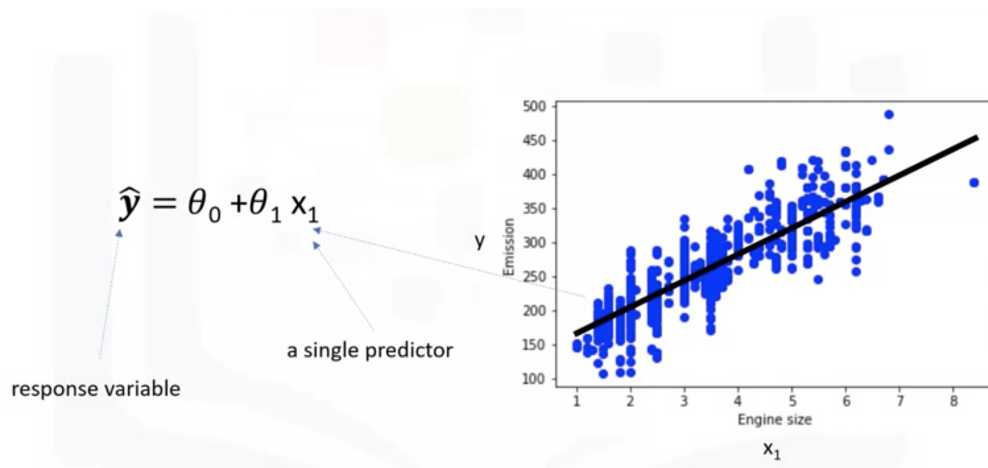
Theta Parameters:

$$\theta_0 = 92.94 \quad (\text{intercept}), \quad \theta_1 = 43.98 \quad (\text{slope}).$$

Using the engine size of 2.4 L, we can substitute the values into the equation:

$$\text{CO}_2 \text{ Emission} = 92.94 + 43.98 \cdot 2.4 = 92.94 + 105.55 = 198.49 \text{ g/km}.$$

So, we predict that the CO₂ emissions for this car with an engine size of 2.4 L is approximately 198.49 g/km.



Fitting the Line: Finding Theta 0 and Theta 1

The key challenge in linear regression is to find the best-fitting line by determining the slope (θ_1) and intercept (θ_0).

1. Calculate the Mean of X and Y

We need the average values of both the engine size (X) and the CO₂ emissions (Y) to help in the calculation of the slope and intercept.

2. Slope (Theta 1)

The slope can be calculated using the formula:

$$\theta_1 = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}$$

Where:

- X_i and Y_i are the individual values for engine size and CO₂ emissions,
- $\bar{X}$ and $\bar{Y}$ are the averages of engine size and CO₂ emissions.

3. Intercept (Theta 0)

The intercept can be found using the formula:

$$\theta_0 = \bar{Y} - \theta_1 \cdot \bar{X}$$

Estimating the parameters

	ENGINE SIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267

$$\hat{y} = \theta_0 + \theta_1 x_1$$

$$\theta_1 = \frac{\sum_{i=1}^s (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^s (x_i - \bar{x})^2}$$

$$\bar{x} = (2.0 + 2.4 + 1.5 + \dots) / 9 = 3.03$$

$$\bar{y} = (196 + 221 + 136 + \dots) / 9 = 226.22$$

$$\theta_1 = \frac{(2.0 - 3.03)(196 - 226.22) + (2.4 - 3.03)(221 - 226.22) + \dots}{(2.0 - 3.03)^2 + (2.4 - 3.03)^2 + \dots}$$

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

Estimating the parameters

	ENGINE SIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267

$$\hat{y} = \theta_0 + \theta_1 x_1$$

$$\theta_1 = \frac{\sum_{i=1}^s (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^s (x_i - \bar{x})^2}$$

$$\bar{x} = (2.0 + 2.4 + 1.5 + \dots) / 9 = 3.03$$

$$\bar{y} = (196 + 221 + 136 + \dots) / 9 = 226.22$$

$$\theta_1 = \frac{(2.0 - 3.03)(196 - 226.22) + (2.4 - 3.03)(221 - 226.22) + \dots}{(2.0 - 3.03)^2 + (2.4 - 3.03)^2 + \dots}$$

$$\theta_1 = 39$$

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

$$\theta_0 = 226.22 - 39 * 3.03$$

$$\theta_0 = 125.74$$

$$\hat{y} = 125.74 + 39x_1$$

Once the slope and intercept are computed, we can form the equation of the line and use it to predict CO₂ emissions for new data points.

How to find the best fit?

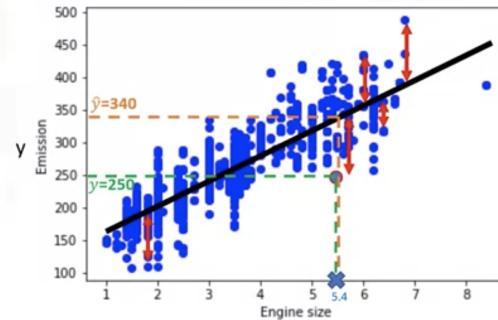
$x_1 = 5.4$ independent variable
 $y = 250$ actual Co2 emission of x1

$$\hat{y} = \theta_0 + \theta_1 x_1$$

$\hat{y} = 340$ the predicted emission of x1

$$\begin{aligned} \text{Error} &= y - \hat{y} \\ &= 250 - 340 \\ &= -90 \end{aligned}$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Residual Error: How Good Is the Fit?

The line we fit to the data might not perfectly pass through all the data points. This leads to residual error—the difference between the observed value and the predicted value from the line.

For example, if the actual CO₂ emissions of a car with an engine size of 5.4 L is 250 g/km, but the model predicts 340 g/km, then the residual error is:

$$\text{Residual Error} = 340 - 250 = 90 \text{ g/km}$$

The Mean Squared Error (MSE) is a common measure used to assess how well the model fits the data. The goal in linear regression is to minimize this error by adjusting the parameters θ_0 and θ_1 to find the best fit line.

The formula for MSE is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Where:

- Y_i is the actual value,
- $\hat{Y}_i$ is the predicted value,
- n is the number of data points.

Conclusion: Using Linear Regression for Prediction

Once the parameters θ_0 and θ_1 are determined, we can use them to make predictions for new data points. This makes linear regression a very useful tool, especially for tasks like predicting CO₂ emissions from car models, based on easily measurable variables such as engine size.

In practice, linear regression is:

- **Fast** to compute,
- **Easy to implement** using libraries like scikit-learn in Python,
- **Highly interpretable**, meaning we can understand how each feature (such as engine size) impacts the predictions.

This makes linear regression an excellent starting point for any predictive modeling task where the target variable is continuous.

Applications in Real Life

Beyond predicting CO₂ emissions in cars, linear regression is widely used in:

- **Sales forecasting**: Predicting future sales based on past data.
- **Housing price prediction**: Estimating house prices based on features like size, location, and number of rooms.
- **Healthcare**: Estimating patient outcomes based on various health metrics.

By using simple linear regression, we can model and make predictions in many different fields, helping businesses, researchers, and policymakers make data-driven decisions.

3.2.1 Multivariate Linear Regression

Multivariate Linear Regression is an extension of simple linear regression that models the relationship between two or more independent variables (predictors) and a single dependent variable (response) using a linear equation. It is used when we want to predict a continuous output based on multiple input features.

Mathematical Model

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon$$

- Y : Dependent variable (target/output)
- $X_1, X_2, \dots, X_n$: Independent variables (features)
- β_0 : Intercept
- $\beta_1, \beta_2, \dots, \beta_n$: Coefficients for each independent variable
- ε : Error term (residual)

Assumptions

1. **Linearity**: Relationship between independent and dependent variables is linear.
2. **Independence**: Observations are independent of each other.
3. **Homoscedasticity**: Constant variance of residuals.
4. **Normality**: Residuals are normally distributed.
5. **No multicollinearity**: Independent variables should not be highly correlated.

Use Cases

- Predicting house prices based on size, location, number of bedrooms, etc.
- Forecasting sales based on advertising spend, season, and region.

- Estimating student performance based on study time, attendance, and prior grades.

Steps to Perform Multivariate Linear Regression

1. Collect data
2. Preprocess data (handle missing values, encode categorical variables, scale features)
3. Split dataset into training and testing sets
4. Fit the model to the training data
5. Evaluate the model using metrics (e.g., R^2 , RMSE)
6. Interpret coefficients

Matrix Formulation of Linear Regression

A multiple linear regression model with $p - 1$ predictors and n observations can be written as:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_{p-1} x_{i(p-1)} + \varepsilon_i$$

In matrix form, this becomes:

$$\mathbf{y} = \mathbf{X}\beta + \varepsilon$$

Components

- **Response vector** ($\mathbf{y}$, $n \times 1$):

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

- **Design matrix** ($\mathbf{X}$, $n \times p$):

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1(p-1)} \\ 1 & x_{21} & x_{22} & \cdots & x_{2(p-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{n(p-1)} \end{bmatrix}$$

- **Coefficient vector** (β , $p \times 1$):

$$\beta = \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_{p-1} \end{bmatrix}$$

- **Error vector** (ε , $n \times 1$):

$$\varepsilon = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}$$

3.2.2 Ordinary Least Squares (OLS) Estimation

OLS estimates β by minimizing the residual sum of squares (RSS):

$$\text{RSS} = (\mathbf{y} - \mathbf{X}\beta)^\top (\mathbf{y} - \mathbf{X}\beta)$$

To minimize, take the derivative with respect to β and set to zero:

$$\frac{\partial \text{RSS}}{\partial \beta} = -2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\beta) = 0$$

Solving gives the OLS estimator:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Assumptions for Validity

- $\mathbf{X}^\top \mathbf{X}$ is invertible (i.e., $\mathbf{X}$ has full column rank, no perfect multicollinearity).
- Errors $\varepsilon \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$ for classical inference.

Predictions and Residuals

- Predicted values:

$$\hat{\mathbf{y}} = \mathbf{X} \hat{\beta}$$

- Residuals:

$$\hat{\varepsilon} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X} \hat{\beta}$$

Numerical Example

Suppose we have the following data with 3 observations and 2 predictors:

Observation	x_1	x_2	y
1	1	2	6
2	2	1	8
3	3	3	12

Numerical Example (Matrix Approach to Linear Regression)

Given 3 observations with 2 predictors:

Observation	x_1	x_2	y
1	1	2	6
2	2	1	8
3	3	3	12

Step 1: Define Matrices

Design matrix $\mathbf{X}$:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 3 & 3 \end{bmatrix}$$

Response vector $\mathbf{y}$:

$$\mathbf{y} = \begin{bmatrix} 6 \\ 8 \\ 12 \end{bmatrix}$$

Step 2: Compute $\mathbf{X}^\top \mathbf{X}$

First, compute $\mathbf{X}^\top$:

$$\mathbf{X}^\top = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 1 & 3 \end{bmatrix}$$

Now compute:

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 1 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 3 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 6 \\ 6 & 14 & 13 \\ 6 & 13 & 14 \end{bmatrix}$$

Step 3: Compute $\mathbf{X}^\top \mathbf{y}$

$$\mathbf{X}^\top \mathbf{y} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 2 & 1 & 3 \end{bmatrix} \begin{bmatrix} 6 \\ 8 \\ 12 \end{bmatrix} = \begin{bmatrix} 6 + 8 + 12 \\ 6 + 16 + 36 \\ 12 + 8 + 36 \end{bmatrix} = \begin{bmatrix} 26 \\ 58 \\ 56 \end{bmatrix}$$

Step 4: Compute $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

Given:

$$(\mathbf{X}^\top \mathbf{X})^{-1} = \begin{bmatrix} 2.333 & -1.000 & -1.000 \\ -1.000 & 0.833 & 0.000 \\ -1.000 & 0.000 & 0.833 \end{bmatrix}$$

Then:

$$\hat{\beta} = \begin{bmatrix} 2.333 & -1.000 & -1.000 \\ -1.000 & 0.833 & 0.000 \\ -1.000 & 0.000 & 0.833 \end{bmatrix} \begin{bmatrix} 26 \\ 58 \\ 56 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$$

Final Regression Equation

$$\hat{y} = 2 + 1 \cdot x_1 + 2 \cdot x_2$$

Step 5: Predictions and Residuals

- Observation 1: $\hat{y} = 2 + 1 \cdot 1 + 2 \cdot 2 = 2 + 1 + 4 = 7$, Actual $y = 6$ Residual = -1
- Observation 2: $\hat{y} = 2 + 1 \cdot 2 + 2 \cdot 1 = 2 + 2 + 2 = 6$, Actual $y = 8$ Residual = $+2$
- Observation 3: $\hat{y} = 2 + 1 \cdot 3 + 2 \cdot 3 = 2 + 3 + 6 = 11$, Actual $y = 12$ Residual = $+1$

3.2.3 R-squared and Adjusted R-squared

1. R-squared (R^2)

Definition:

R-squared measures the proportion of the variance in the dependent variable y that is explained by the model.

$$R^2 = 1 - \frac{\text{SSR}}{\text{SST}} = \frac{\text{SSE}}{\text{SST}}$$

Where:

$$\text{SSR} = \sum (y_i - \hat{y}_i)^2 \quad (\text{Sum of Squared Residuals})$$

$$\text{SST} = \sum (y_i - \bar{y})^2 \quad (\text{Total Sum of Squares})$$

$$\text{SSE} = \sum (\hat{y}_i - \bar{y})^2 \quad (\text{Explained Sum of Squares})$$

Interpretation:

- $R^2 = 1$: perfect fit
- $R^2 = 0$: model explains no variance (same as the mean)
- $R^2 < 0$: model is worse than using the mean (can happen if no intercept is used)

2. Adjusted R-squared

Definition:

Adjusted R^2 penalizes the R-squared value based on the number of predictors to discourage overfitting.

$$\bar{R}^2 = 1 - \left(\frac{(1 - R^2)(n - 1)}{n - p} \right)$$

Where:

- n = number of observations
- p = number of parameters (including intercept)

Why Use It?

- Adjusted R^2 can decrease when unnecessary predictors are added.
- It gives a more honest measure when comparing models with different numbers of predictors.

3. Numerical Example

From the model: $\hat{y} = 2 + 1x_1 + 2x_2$ considering we got different predicted $\hat{y}$ as 7 6 and 11

Observation	y	$\hat{y}$	Residual ($y - \hat{y}$)
1	6	7	-1
2	8	6	+2
3	12	11	+1

Step 1: SSR (Sum of Squared Residuals)

$$\text{SSR} = (-1)^2 + 2^2 + 1^2 = 1 + 4 + 1 = 6$$

Step 2: SST (Total Sum of Squares)

$$\bar{y} = \frac{6 + 8 + 12}{3} = \frac{26}{3} \approx 8.67$$

$$\text{SST} = (6 - 8.67)^2 + (8 - 8.67)^2 + (12 - 8.67)^2 = (-2.67)^2 + (-0.67)^2 + (3.33)^2 \approx 18.65$$

Step 3: Compute R^2

$$R^2 = 1 - \frac{6}{18.65} \approx 0.678$$

So, about 67.8% of the variation in y is explained by the model.

Step 4: Adjusted R^2

$$n = 3, \quad p = 3$$

$$\bar{R}^2 = 1 - \left(\frac{(1 - 0.678)(3 - 1)}{3 - 3} \right) \Rightarrow \text{undefined (division by zero)}$$

This occurs because Adjusted R^2 requires at least 1 degree of freedom. With only 3 data points and 3 parameters, it's not defined.

4. Summary

Metric	Formula	Interpretation
R^2	$1 - \frac{\text{SSR}}{\text{SST}}$	Proportion of variance explained
Adjusted R^2	$1 - \frac{(1 - R^2)(n - 1)}{n - p}$	Adjusted for number of predictors

When to Use: Always compute both when comparing models with different numbers of predictors.

Python Code: R-squared and Adjusted R-squared

```
import numpy as np

# Design matrix with intercept and two predictors
X = np.array([
    [1, 1, 2], # Intercept, x1=1, x2=2
    [1, 2, 1], # Intercept, x1=2, x2=1
    [1, 3, 3]  # Intercept, x1=3, x2=3
])

# Response variable
y = np.array([6, 8, 12]).reshape(-1, 1)

# OLS Estimation:  $\beta_{\text{hat}} = (X^T X)^{-1} X^T y$ 
XtX = X.T @ X
XtX_inv = np.linalg.inv(XtX)
Xty = X.T @ y
beta_hat = XtX_inv @ Xty

# Predicted values and residuals
y_hat = X @ beta_hat
residuals = y - y_hat

# Compute R-squared
SSR = np.sum(residuals ** 2) # Sum of Squared Residuals
SST = np.sum((y - np.mean(y)) ** 2) # Total Sum of Squares
R_squared = 1 - SSR / SST

# Compute Adjusted R-squared
n = len(y) # number of observations
p = X.shape[1] # number of parameters (including intercept)
if n > p:
    adj_R_squared = 1 - ((1 - R_squared) * (n - 1)) / (n - p)
else:
    adj_R_squared = None # Not defined if degrees of freedom <= 0
```



```
# Print Results
print("Estimated Coefficients (beta_hat):\n", beta_hat)
print("Predicted y (y_hat):\n", y_hat.flatten())
print("Residuals:\n", residuals.flatten())
print("R-squared:", round(R_squared, 4))
print("Adjusted R-squared:", adj_R_squared if adj_R_squared
      is not None else "Not defined (n = p)")
```

Multivariate Linear Regression Example

We want to predict a student's final exam score (Y) based on:

- Hours studied (X_1)
- Number of practice exams taken (X_2)

Sample Data

Student	Hours (X_1)	Practice Exams (X_2)	Final Score (Y)
1	2	1	60
2	3	2	75
3	4	2	85
4	5	3	95
5	6	4	100

Step 1: Matrix Setup

Design matrix $\mathbf{X}$ and response vector $\mathbf{Y}$:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 3 & 2 \\ 1 & 4 & 2 \\ 1 & 5 & 3 \\ 1 & 6 & 4 \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} 60 \\ 75 \\ 85 \\ 95 \\ 100 \end{bmatrix}$$

Step 2: Compute $\mathbf{X}^\top \mathbf{X}$

$$\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 5 & 20 & 12 \\ 20 & 90 & 53 \\ 12 & 53 & 34 \end{bmatrix}$$

Step 3: Compute $\mathbf{X}^\top \mathbf{Y}$

$$\mathbf{X}^\top \mathbf{Y} = \begin{bmatrix} 415 \\ 1690 \\ 1025 \end{bmatrix}$$

Step 4: Calculate Regression Coefficients

Using $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$:

$$\hat{\beta} = \begin{bmatrix} 35 \\ 10 \\ 5 \end{bmatrix}$$

Final regression equation:

$$\hat{Y} = 35 + 10X_1 + 5X_2$$

Step 5: Model Evaluation

- Mean of Y : $\bar{Y} = 83$
- Total Sum of Squares (SST): 1030
- Residual Sum of Squares (SSE): 250
- Regression Sum of Squares (SSR): 780

Step 6: Calculate R^2 and Adjusted R^2

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{250}{1030} \approx 0.7573 \quad (75.73\%)$$

$$R_{adj}^2 = 1 - \left(\frac{n-1}{n-p-1} \right) (1 - R^2) = 1 - \left(\frac{4}{2} \right) (0.2427) \approx 0.5146 \quad (51.46\%)$$

Where:

- $n = 5$ observations
- $p = 2$ predictors

Step 7: Predictions and Residuals

Student	Actual Y	Predicted $\hat{Y}$	Residual
1	60	$35+10(2)+5(1)=60$	0
2	75	$35+10(3)+5(2)=75$	0
3	85	$35+10(4)+5(2)=85$	0
4	95	$35+10(5)+5(3)=100$	-5
5	100	$35+10(6)+5(4)=115$	-15

Interpretation

- Both hours studied and practice exams have positive effects on final scores
- The model explains 75.73% of variance in exam scores
- Adjusted R^2 accounts for the small sample size, showing 51.46% explanatory power
- The model fits perfectly for the first 3 observations but has larger residuals for the last 2

Multicollinearity and Variance Inflation Factors (VIF)

3.2.4 Definition of Multicollinearity

Multicollinearity occurs when two or more predictor variables in a regression model are highly linearly related. This can cause the coefficient estimates to be unstable and their variances to be inflated, making it difficult to assess the importance of individual predictors.

Problems Caused

- Coefficients become sensitive to small changes in the model or data.
- Interpretation of the coefficients becomes unreliable.

- Increases standard errors of the coefficients.

Variance Inflation Factor (VIF)

The Variance Inflation Factor for the j -th predictor is defined as:

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

Where R_j^2 is the R-squared from regressing predictor j on all the other predictors.

Interpretation of VIF

VIF Value	Interpretation
1	No multicollinearity
1 to 5	Moderate correlation, acceptable
5 to 10	High correlation, potential problem
greater than 10	Serious multicollinearity issue

Multicollinearity and Variance Inflation Factors (VIF)

Multicollinearity occurs when predictor variables in a regression model are highly correlated. This can lead to unreliable estimates of regression coefficients. A common diagnostic tool for detecting multicollinearity is the **Variance Inflation Factor (VIF)**.

Interpretation

- Variables **x1** and **x2** have very high VIF values, indicating strong multicollinearity.
- Variable **x3** has a low VIF, showing no significant correlation with other predictors.

Guidelines

- **VIF greater than 5:** Potential multicollinearity.
- **VIF greater than 10:** Serious multicollinearity problem.

Solutions

- Remove or combine highly correlated predictors.
- Use dimensionality reduction techniques like Principal Component Analysis (PCA).

3.3 Non-Parametric Regression: Nadaraya-Watson Kernel Regression

Kernel regression is a non-parametric approach used to estimate the relationship between variables. The Nadaraya-Watson kernel regression is a popular method that provides a smooth estimate of the conditional expectation of the response variable.

Derivation of the Estimator

The Nadaraya-Watson estimator for non-parametric regression can be derived as follows:

Given a dataset $\{(x_i, y_i)\}_{i=1}^n$, the goal is to estimate the conditional expectation $\mathbb{E}[Y|X = x]$. The Nadaraya-Watson estimator for $\mathbb{E}[Y|X = x]$ is given by:

$$\hat{m}(x) = \frac{\sum_{i=1}^n K_h(x - x_i) y_i}{\sum_{i=1}^n K_h(x - x_i)}$$

where: - $\hat{m}(x)$ is the estimated value of $\mathbb{E}[Y|X = x]$, - $K_h(\cdot)$ is the kernel function with bandwidth h , - x_i and y_i are the data points, and - h is the bandwidth parameter that controls the smoothness of the estimate.

The kernel function $K_h(\cdot)$ is usually a symmetric function such as the Gaussian kernel:

$$K_h(x - x_i) = \frac{1}{h} K\left(\frac{x - x_i}{h}\right)$$

where $K(u)$ is a kernel function, such as the Gaussian kernel:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}u^2\right)$$

Thus, the Nadaraya-Watson estimator can be written as:

$$\hat{m}(x) = \frac{\sum_{i=1}^n \frac{1}{h} K\left(\frac{x-x_i}{h}\right) y_i}{\sum_{i=1}^n \frac{1}{h} K\left(\frac{x-x_i}{h}\right)}$$

This equation is the basic formulation of the Nadaraya-Watson kernel regression estimator.

Rules for Setting the Appropriate Bandwidth Size

The bandwidth parameter h plays a crucial role in determining the smoothness of the estimated regression function. A smaller bandwidth results in a less smooth estimate that is more sensitive to the data points, while a larger bandwidth produces a smoother estimate.

1. Rule of Thumb: One common method for selecting the bandwidth is the rule of thumb, which is based on minimizing the Mean Squared Error (MSE). A typical heuristic for bandwidth selection is:

$$h_{\text{opt}} \approx \left(\frac{4\hat{\sigma}^5}{3n} \right)^{1/5}$$

where: - $\hat{\sigma}^2$ is the sample variance of the response variable, - n is the number of data points.

This rule assumes that the data is roughly normally distributed and that the kernel is Gaussian.

2. Cross-validation: Cross-validation is another commonly used method to select the optimal bandwidth. In this method, the data is split into training and test sets, and the bandwidth that minimizes the cross-validation error is chosen. The cross-validation error is given by:

$$CV(h) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{m}_{-i}(x_i))^2$$

where $\hat{m}_{-i}(x_i)$ is the Nadaraya-Watson estimate computed without using the i -th data point.

3. Plug-in Methods: Another approach for bandwidth selection is the plug-in method, which estimates the optimal bandwidth by plugging an estimate of the smoothness of the underlying function into a theoretical formula. This method is more complex but can provide better results when the data is noisy.

4. Heuristic Bandwidth Selection: In practice, a heuristic selection process might be used based on trial and error. For example: - If the data is highly variable, a larger bandwidth is chosen to smooth out fluctuations. - If the data is relatively smooth, a smaller bandwidth might be used to capture finer details of the relationship between the variables.

Conclusion

The Nadaraya-Watson kernel regression is a powerful non-parametric method for estimating the conditional expectation of the response variable. The choice of bandwidth h is critical in determining the smoothness of the estimate and can be selected using rules of thumb, cross-validation, or other methods depending on the nature of the data.

3.4 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a fundamental **unsupervised** dimensionality reduction technique widely used in statistics, machine learning, and data visualization. It transforms high-dimensional data into a lower-dimensional space while preserving as much variance as possible.

Intuition Behind PCA

PCA identifies new axes (**principal components**) in the data that maximize variance. These axes are linear combinations of the original features and are orthogonal (uncorrelated) to each other.

- **First Principal Component (PC1):** Direction of maximum variance
- **Second Principal Component (PC2):** Orthogonal to PC1, captures the next highest variance

Why PCA?

- **Visualization:** Reduce data to 2D/3D for plotting
- **Noise Reduction:** Remove low-variance (noisy) dimensions
- **Efficiency:** Speeds up machine learning models
- **Multicollinearity Removal:** Produces uncorrelated features

Mathematical Foundations

Standardization

Since PCA is sensitive to scale, data must be standardized:

$$X_{\text{std}} = \frac{X - \mu}{\sigma}$$

where:

- μ = mean of each feature
- σ = standard deviation

Covariance Matrix

The covariance matrix $\mathbf{C}$ measures how features vary together:

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X}$$

- If $C_{ij} = 0$, features i and j are uncorrelated
- If $C_{ij} > 0$, they vary together
- If $C_{ij} < 0$, they vary inversely

Eigenvalue Decomposition

PCA solves:

$$\mathbf{C}\mathbf{v} = \lambda\mathbf{v}$$

where:

- $\mathbf{v}$ = eigenvector (direction of PC)
- λ = eigenvalue (magnitude of variance along PC)

Key Properties:

- Eigenvectors are **orthogonal** (uncorrelated)
- Eigenvalues rank PCs by importance (higher λ = more variance)

Selecting Principal Components

- Sort eigenvalues in descending order
- Choose top k eigenvectors (PCs) that capture most variance
- **Explained Variance Ratio:**

$$\text{Explained Variance} = \frac{\lambda_i}{\sum \lambda}$$

Projection (Dimensionality Reduction)

Transform data into the new subspace:

$$\mathbf{X}_{\text{PCA}} = \mathbf{X}_{\text{std}} \times \mathbf{W}_k$$

where $\mathbf{W}_k$ is the matrix of top k eigenvectors.

Principal Component Analysis (PCA) Steps

1. **Standardize the data:**

$$\mathbf{X}_{\text{std}} = \frac{\mathbf{X} - \mu}{\sigma}$$

(Mean-center and scale to unit variance)

2. **Compute covariance matrix:**

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}_{\text{std}}^{\top} \mathbf{X}_{\text{std}}$$

3. **Perform eigendecomposition:**

$$\mathbf{C}\mathbf{v} = \lambda\mathbf{v}$$

(Find eigenvalues λ and eigenvectors $\mathbf{v}$)

4. **Select principal components:**

- Sort eigenvalues in descending order
- Choose top k eigenvectors (loading vectors) $\mathbf{W}_k$

5. **Transform the data:**

$$\mathbf{X}_{\text{PCA}} = \mathbf{X}_{\text{std}} \mathbf{W}_k$$

(Project onto new subspace)

Applications of PCA

- **Data Compression:** Reduce storage & computation costs
- **Visualization:** Plot high-dimensional data in 2D/3D
- **Noise Reduction:** Discard low-variance components
- **Feature Extraction:** Improve supervised learning

Limitations & Caveats

- **Interpretability Loss:** PCs are linear combinations
- **Scaling Sensitivity:** Must standardize data first
- **Linearity Assumption:** Nonlinear methods may be better
- **Information Loss:** Dropping too many PCs loses variance

Practical Example (Python)

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
```

```

import matplotlib.pyplot as plt

# Load data
data = load_iris()
X = data.data
y = data.target

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Plot
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
plt.xlabel('PC1 (Explained Variance: {:.2f}\%)'
           .format(pca.explained_variance_ratio_[0]*100))
plt.ylabel('PC2 (Explained Variance: {:.2f}\%)'
           .format(pca.explained_variance_ratio_[1]*100))
plt.title('PCA of Iris Dataset')
plt.show()

```

When to Use PCA?

- High-dimensional data (e.g., images, genomics)
- Multicollinearity exists among features
- Need visualization in 2D/3D
- Want to speed up ML models

Conclusion

PCA is a **powerful, versatile** method for reducing dimensionality while preserving key patterns. Understanding its **mathematical foundations** (covariance, eigenvalues) helps in applying it effectively.

Dimension Reduction

In pattern recognition, **Dimension Reduction** is defined as:

- It is a process of converting a data set having vast dimensions into a data set with lesser dimensions.
- It ensures that the converted data set conveys similar information concisely.

Example

Consider the following example:

The following graph shows two dimensions x_1 and x_2 .

- x_1 represents the measurement of several objects in cm.
- x_2 represents the measurement of several objects in inches.

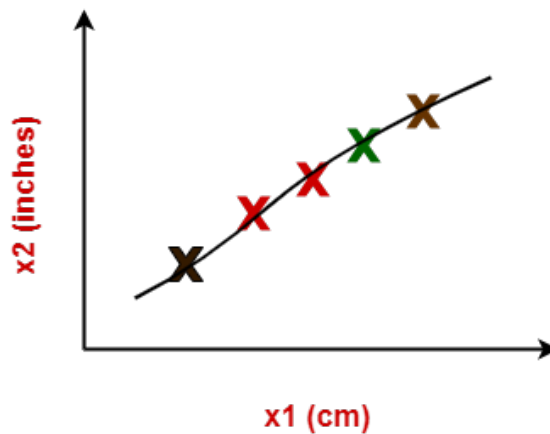


Figure 3.1: Original 2D data

In machine learning:

- Using both these dimensions convey similar information.
- Also, they introduce a lot of noise in the system.
- So, it is better to use just one dimension.

Using dimension reduction techniques:

- We convert the dimensions of data from 2 dimensions (x_1 and x_2) to 1 dimension (z_1).
- It makes the data relatively easier to explain.

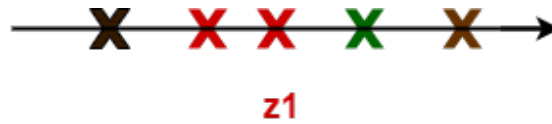


Figure 3.2: Data after dimension reduction

Benefits

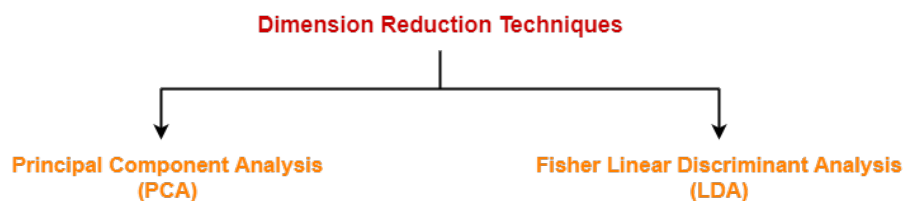
Dimension reduction offers several benefits such as:

- It compresses the data and thus reduces the storage space requirements.
- It reduces the time required for computation since less dimensions require less computation.
- It eliminates the redundant features.
- It improves the model performance.

Dimension Reduction Techniques

The two popular and well-known dimension reduction techniques are:

- Principal Component Analysis (PCA)
- Fisher Linear Discriminant Analysis (LDA)



Principal Component Analysis

Principal Component Analysis is a well-known dimension reduction technique.

- It transforms the variables into a new set of variables called as principal components.
- These principal components are linear combination of original variables and are orthogonal.

- The first principal component accounts for most of the possible variation of original data.
- The second principal component does its best to capture the variance in the data.
- There can be only two principal components for a two-dimensional data set.

PCA Algorithm

The steps involved in PCA Algorithm are as follows:

1. Step-01: Get data.
2. Step-02: Compute the mean vector (μ).
3. Step-03: Subtract mean from the given data.
4. Step-04: Calculate the covariance matrix.
5. Step-05: Calculate the eigen vectors and eigen values of the covariance matrix.
6. Step-06: Choosing components and forming a feature vector.
7. Step-07: Deriving the new data set.

Practice Problems Based on Principal Component Analysis

Problem-01

Given data = $\{2, 3, 4, 5, 6, 7; 1, 5, 3, 6, 7, 8\}$.

Compute the principal component using PCA Algorithm.

OR Consider the two dimensional patterns (2, 1), (3, 5), (4, 3), (5, 6), (6, 7), (7, 8).

Compute the principal component using PCA Algorithm.

OR Compute the principal component of following data-

- CLASS 1

$$- X = 2, 3, 4$$

$$- Y = 1, 5, 3$$

- CLASS 2

$$- X = 5, 6, 7$$

$$- Y = 6, 7, 8$$

Solution

We use the above discussed PCA Algorithm:

Step-01:

Get data.

The given feature vectors are-

$$x_1 = (2, 1)$$

$$x_2 = (3, 5)$$

$$x_3 = (4, 3)$$

$$x_4 = (5, 6)$$

$$x_5 = (6, 7)$$

$$x_6 = (7, 8)$$

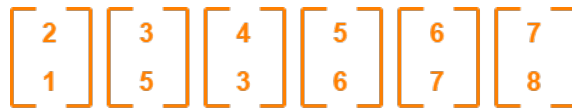


Figure 3.3: Original data points

Step-02:

Calculate the mean vector (μ) .

$$\text{Mean vector } (\mu) = \left(\frac{2+3+4+5+6+7}{6}, \frac{1+5+3+6+7+8}{6} \right) = (4.5, 5)$$

Thus,

$$\text{Mean vector } (\mu) = \begin{bmatrix} 4.5 \\ 5 \end{bmatrix}$$

Figure 3.4: Mean vector

Step-03:

Subtract mean vector (μ) from the given feature vectors.

$$x_1 - \mu = (2 - 4.5, 1 - 5) = (-2.5, -4)$$

$$x_2 - \mu = (3 - 4.5, 5 - 5) = (-1.5, 0)$$

$$x_3 - \mu = (4 - 4.5, 3 - 5) = (-0.5, -2)$$

$$x_4 - \mu = (5 - 4.5, 6 - 5) = (0.5, 1)$$

$$x_5 - \mu = (6 - 4.5, 7 - 5) = (1.5, 2)$$

$$x_6 - \mu = (7 - 4.5, 8 - 5) = (2.5, 3)$$

Feature vectors (x_i) after subtracting mean vector (μ) are:

$$\begin{bmatrix} -2.5 \\ -4 \end{bmatrix} \begin{bmatrix} -1.5 \\ 0 \end{bmatrix} \begin{bmatrix} -0.5 \\ -2 \end{bmatrix} \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} \begin{bmatrix} 1.5 \\ 2 \end{bmatrix} \begin{bmatrix} 2.5 \\ 3 \end{bmatrix}$$

Figure 3.5: Data after centering

Step-04:

Calculate the covariance matrix.

Covariance matrix is given by:

$$C = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)(x_i - \mu)^T$$

Now,

$$\begin{aligned}
m_1 &= \begin{bmatrix} -2.5 \\ -4 \end{bmatrix} \begin{bmatrix} -2.5 & -4 \end{bmatrix} = \begin{bmatrix} 6.25 & 10 \\ 10 & 16 \end{bmatrix} \\
m_2 &= \begin{bmatrix} -1.5 \\ 0 \end{bmatrix} \begin{bmatrix} -1.5 & 0 \end{bmatrix} = \begin{bmatrix} 2.25 & 0 \\ 0 & 0 \end{bmatrix} \\
m_3 &= \begin{bmatrix} -0.5 \\ -2 \end{bmatrix} \begin{bmatrix} -0.5 & -2 \end{bmatrix} = \begin{bmatrix} 0.25 & 1 \\ 1 & 4 \end{bmatrix} \\
m_4 &= \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} \begin{bmatrix} 0.5 & 1 \end{bmatrix} = \begin{bmatrix} 0.25 & 0.5 \\ 0.5 & 1 \end{bmatrix} \\
m_5 &= \begin{bmatrix} 1.5 \\ 2 \end{bmatrix} \begin{bmatrix} 1.5 & 2 \end{bmatrix} = \begin{bmatrix} 2.25 & 3 \\ 3 & 4 \end{bmatrix} \\
m_6 &= \begin{bmatrix} 2.5 \\ 3 \end{bmatrix} \begin{bmatrix} 2.5 & 3 \end{bmatrix} = \begin{bmatrix} 6.25 & 7.5 \\ 7.5 & 9 \end{bmatrix}
\end{aligned}$$

Now,

$$\text{Covariance matrix} = \frac{m_1 + m_2 + m_3 + m_4 + m_5 + m_6}{6}$$

On adding the above matrices and dividing by 6, we get:

$$C = \begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix}$$

Step-05:

Calculate the eigen values and eigen vectors of the covariance matrix.

λ is an eigen value for a matrix M if it is a solution of the characteristic equation $|M - \lambda I| = 0$.

So, we have:

$$\begin{vmatrix} 2.92 - \lambda & 3.67 \\ 3.67 & 5.67 - \lambda \end{vmatrix} = 0$$

From here,

$$\begin{aligned}(2.92 - \lambda)(5.67 - \lambda) - (3.67 \times 3.67) &= 0 \\ 16.56 - 2.92\lambda - 5.67\lambda + \lambda^2 - 13.47 &= 0 \\ \lambda^2 - 8.59\lambda + 3.09 &= 0\end{aligned}$$

Solving this quadratic equation, we get $\lambda = 8.22, 0.38$

Thus, two eigen values are $\lambda_1 = 8.22$ and $\lambda_2 = 0.38$.

Clearly, the second eigen value is very small compared to the first eigen value. So, the second eigen vector can be left out.

Eigen vector corresponding to the greatest eigen value is the principal component for the given data set. So, we find the eigen vector corresponding to eigen value λ_1 .

We use the following equation to find the eigen vector:

$$MX = \lambda X$$

where:

- M = Covariance Matrix
- X = Eigen vector
- λ = Eigen value

Substituting the values in the above equation, we get:

$$\begin{bmatrix} 2.92 & 3.67 \\ 3.67 & 5.67 \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} = 8.22 \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$$

Solving these, we get:

$$2.92X_1 + 3.67X_2 = 8.22X_1$$

$$3.67X_1 + 5.67X_2 = 8.22X_2$$

On simplification, we get:

$$5.3X_1 = 3.67X_2 \quad (1)$$

$$3.67X_1 = 2.55X_2 \quad (2)$$

From (1) and (2), $X_1 = 0.69X_2$

From (2), the eigen vector is:

$$X = \begin{bmatrix} 0.69 \\ 1 \end{bmatrix}$$

Thus, principal component for the given data set is:

$$\begin{bmatrix} 0.69 \\ 1 \end{bmatrix}$$

Lastly, we project the data points onto the new subspace as:

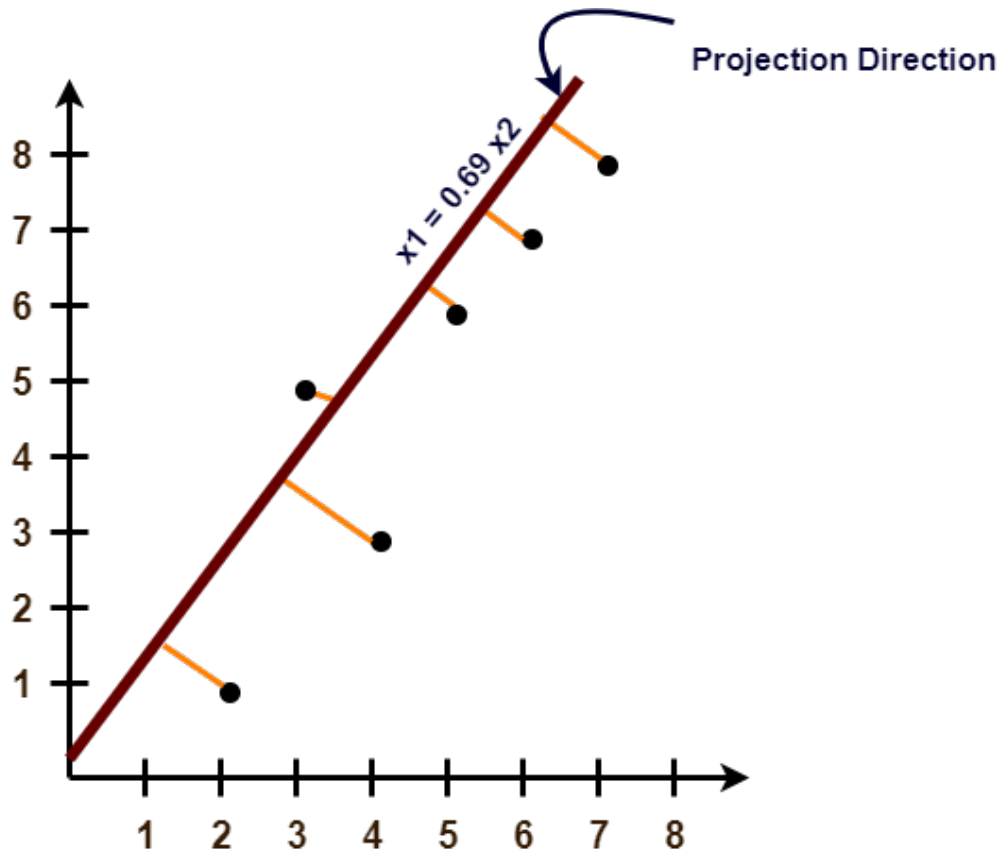


Figure 3.6: Data projection on principal component

Problem-02:

Use PCA Algorithm to transform the pattern (2, 1) onto the eigen vector in the previous question.

Solution

The given feature vector is (2, 1).

The feature vector gets transformed to:

Transpose of Eigen vector $\times$ (Feature Vector – Mean Vector)

$$\begin{bmatrix} 0.69 & 1 \end{bmatrix} \begin{bmatrix} 2 - 4.5 \\ 1 - 5 \end{bmatrix} = \begin{bmatrix} 0.69 & 1 \end{bmatrix} \begin{bmatrix} -2.5 \\ -4 \end{bmatrix} = -5.725$$

3.5 Numericals

1. Simple Linear Regression and MSE Calculation

Given the dataset:

Table 3.1: Data points

x	y
1	2
2	4
3	5
4	4
5	5

Step 1: Calculate Regression Coefficients

The regression line is given by $\hat{y} = mx + b$, where:

$$m = \frac{n \sum(xy) - \sum x \sum y}{n \sum(x^2) - (\sum x)^2}, \quad b = \frac{\sum y - m \sum x}{n}$$

Compute required sums:

$$\sum x = 1 + 2 + 3 + 4 + 5 = 15$$

$$\sum y = 2 + 4 + 5 + 4 + 5 = 20$$

$$\sum xy = (1 \cdot 2) + (2 \cdot 4) + (3 \cdot 5) + (4 \cdot 4) + (5 \cdot 5) = 2 + 8 + 15 + 16 + 25 = 66$$

$$\sum x^2 = 1^2 + 2^2 + 3^2 + 4^2 + 5^2 = 1 + 4 + 9 + 16 + 25 = 55$$

$$n = 5$$

Calculate slope m :

$$m = \frac{(5 \cdot 66) - (15 \cdot 20)}{(5 \cdot 55) - (15)^2} = \frac{330 - 300}{275 - 225} = \frac{30}{50} = 0.6$$

Calculate intercept b :

$$b = \frac{20 - (0.6 \cdot 15)}{5} = \frac{20 - 9}{5} = \frac{11}{5} = 2.2$$

Regression equation: $\hat{y} = 0.6x + 2.2$

Step 2: Predict $\hat{y}$ Values

Using the regression equation:

$$x = 1 : \hat{y} = 0.6(1) + 2.2 = 2.8$$

$$x = 2 : \hat{y} = 0.6(2) + 2.2 = 3.4$$

$$x = 3 : \hat{y} = 0.6(3) + 2.2 = 4.0$$

$$x = 4 : \hat{y} = 0.6(4) + 2.2 = 4.6$$

$$x = 5 : \hat{y} = 0.6(5) + 2.2 = 5.2$$

Step 3: Calculate Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n} \sum (y - \hat{y})^2$$

Table 3.2: Residual calculations

x	y	$\hat{y}$	Residual $(y - \hat{y})$	Squared Residual $(y - \hat{y})^2$
1	2	2.8	-0.8	0.64
2	4	3.4	0.6	0.36
3	5	4.0	1.0	1.00
4	4	4.6	-0.6	0.36
5	5	5.2	-0.2	0.04
Sum				2.40

$$\text{MSE} = \frac{2.40}{5} = 0.48$$

Final Results

- Regression line: $\hat{y} = 0.6x + 2.2$
- Mean Squared Error: $\text{MSE} = 0.48$

2. Problem Statement

Given the following data:

- $X_1 = [1, 2, 3, 4, 5]$
- $X_2 = [1, 1, 2, 2, 3]$
- $Y = [2, 3, 4, 5, 6]$

Fit the multivariate linear regression model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

Solution

Step 1: Design Matrix and Response Vector

Construct the design matrix X and response vector $\mathbf{y}$:

$$X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 3 & 2 \\ 1 & 4 & 2 \\ 1 & 5 & 3 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix}$$

Step 2: Compute $X^T X$

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 1 & 2 & 2 & 3 \end{bmatrix}$$

$$X^T X = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 1 & 2 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 3 & 2 \\ 1 & 4 & 2 \\ 1 & 5 & 3 \end{bmatrix} = \begin{bmatrix} \sum 1 & \sum X_1 & \sum X_2 \\ \sum X_1 & \sum X_1^2 & \sum X_1 X_2 \\ \sum X_2 & \sum X_1 X_2 & \sum X_2^2 \end{bmatrix} = \begin{bmatrix} 5 & 15 & 9 \\ 15 & 55 & 32 \\ 9 & 32 & 19 \end{bmatrix}$$

Step 3: Compute $X^T \mathbf{y}$

$$X^T \mathbf{y} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 1 & 2 & 2 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{bmatrix} = \begin{bmatrix} \sum Y \\ \sum X_1 Y \\ \sum X_2 Y \end{bmatrix} = \begin{bmatrix} 20 \\ 70 \\ 41 \end{bmatrix}$$

Step 4: Compute $(X^T X)^{-1}$

First, calculate the determinant of $X^T X$:

$$\begin{aligned} \det(X^T X) &= 5 \begin{vmatrix} 55 & 32 \\ 32 & 19 \end{vmatrix} - 15 \begin{vmatrix} 15 & 32 \\ 9 & 19 \end{vmatrix} + 9 \begin{vmatrix} 15 & 55 \\ 9 & 32 \end{vmatrix} \\ &= 5(55 \cdot 19 - 32 \cdot 32) - 15(15 \cdot 19 - 32 \cdot 9) + 9(15 \cdot 32 - 55 \cdot 9) \\ &= 5(1045 - 1024) - 15(285 - 288) + 9(480 - 495) \\ &= 5(21) - 15(-3) + 9(-15) \\ &= 105 + 45 - 135 = 15 \end{aligned}$$

Next, find the adjugate matrix:

$$\begin{aligned} \text{Cofactor matrix } C &= \begin{bmatrix} +(55 \cdot 19 - 32 \cdot 32) & -(15 \cdot 19 - 32 \cdot 9) & +(15 \cdot 32 - 55 \cdot 9) \\ -(15 \cdot 19 - 32 \cdot 9) & +(5 \cdot 19 - 9 \cdot 9) & -(5 \cdot 32 - 9 \cdot 15) \\ +(15 \cdot 32 - 55 \cdot 9) & -(5 \cdot 32 - 9 \cdot 15) & +(5 \cdot 55 - 15 \cdot 15) \end{bmatrix} \\ &= \begin{bmatrix} 21 & 3 & -15 \\ 3 & 14 & -25 \\ -15 & -25 & 50 \end{bmatrix} \\ \text{Adjugate } \text{adj}(X^T X) &= C^T = \begin{bmatrix} 21 & 3 & -15 \\ 3 & 14 & -25 \\ -15 & -25 & 50 \end{bmatrix} \quad (\text{symmetric}) \end{aligned}$$

Then compute the inverse:

$$(X^T X)^{-1} = \frac{1}{15} \begin{bmatrix} 21 & 3 & -15 \\ 3 & 14 & -25 \\ -15 & -25 & 50 \end{bmatrix} = \begin{bmatrix} 21/15 & 3/15 & -15/15 \\ 3/15 & 14/15 & -25/15 \\ -15/15 & -25/15 & 50/15 \end{bmatrix} = \begin{bmatrix} 7/5 & 1/5 & -1 \\ 1/5 & 14/15 & -5/3 \\ -1 & -5/3 & 10/3 \end{bmatrix}$$

Step 5: Compute Coefficients β

$$\beta = (X^T X)^{-1} (X^T \mathbf{y}) = \begin{bmatrix} 7/5 & 1/5 & -1 \\ 1/5 & 14/15 & -5/3 \\ -1 & -5/3 & 10/3 \end{bmatrix} \begin{bmatrix} 20 \\ 70 \\ 41 \end{bmatrix}$$

Calculate each component:

$$\begin{aligned} \beta_0 &= \left(\frac{7}{5}\right)(20) + \left(\frac{1}{5}\right)(70) + (-1)(41) \\ &= 28 + 14 - 41 = 1 \end{aligned}$$

$$\begin{aligned} \beta_1 &= \left(\frac{1}{5}\right)(20) + \left(\frac{14}{15}\right)(70) + \left(-\frac{5}{3}\right)(41) \\ &= 4 + \frac{980}{15} - \frac{205}{3} \\ &= 4 + \frac{196}{3} - \frac{205}{3} \\ &= 4 - \frac{9}{3} = 4 - 3 = 1 \end{aligned}$$

$$\begin{aligned} \beta_2 &= (-1)(20) + \left(-\frac{5}{3}\right)(70) + \left(\frac{10}{3}\right)(41) \\ &= -20 - \frac{350}{3} + \frac{410}{3} \\ &= -20 + \frac{60}{3} = -20 + 20 = 0 \end{aligned}$$

Final Model

$$\boxed{Y = 1 + 1 \cdot X_1 + 0 \cdot X_2}$$

Verification

X_1	X_2	Actual Y	Predicted $\hat{Y}$	Result
1	1	2	$1 + 1 \cdot 1 = 2$	Match
2	1	3	$1 + 1 \cdot 2 = 3$	Match
3	2	4	$1 + 1 \cdot 3 = 4$	Match
4	2	5	$1 + 1 \cdot 4 = 5$	Match
5	3	6	$1 + 1 \cdot 5 = 6$	Match

Conclusion

The regression coefficients are $\beta_0 = 1$, $\beta_1 = 1$, and $\beta_2 = 0$. The model simplifies to $Y = 1 + X_1$, indicating X_2 is not significant for predicting Y in this dataset.

Verification

X_1	X_2	Actual Y	Predicted $\hat{Y}$
1	1	2	2
2	1	3	3
3	2	4	4
4	2	5	5
5	3	6	6

Perfect fit with residual sum of squares (SSE) = 0.

3. Goodness of Fit: R-squared and Adjusted R-squared

Calculations

- Total Sum of Squares (SST):

$$SST = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$\text{Mean } \bar{y} = \frac{2+3+4+5+6}{5} = 4$$

$$SST = (2-4)^2 + (3-4)^2 + (4-4)^2 + (5-4)^2 + (6-4)^2 = 4 + 1 + 0 + 1 + 4 = 10$$

- Residual Sum of Squares (SSE):

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = 0 \quad (\text{from verification})$$

- Regression Sum of Squares (SSR):

$$SSR = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2 = (2-4)^2 + (3-4)^2 + (4-4)^2 + (5-4)^2 + (6-4)^2 = 10$$

- R-squared (Coefficient of Determination):

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{0}{10} = 1$$

$$\text{or } R^2 = \frac{\text{SSR}}{\text{SST}} = \frac{10}{10} = 1$$

- Adjusted R-squared:

$$\bar{R}^2 = 1 - \left(\frac{\text{SSE}/(n-k-1)}{\text{SST}/(n-1)} \right) = 1 - \left(\frac{0/(5-2-1)}{10/(5-1)} \right) = 1 - \left(\frac{0}{2.5} \right) = 1$$

where:

- $n = 5$ (number of observations)
- $k = 2$ (number of independent variables)

Results

$$R^2 = 1.0$$

$$\text{Adjusted } R^2 = 1.0$$

Interpretation

- $R^2 = 1$ indicates 100% of the variance in Y is explained by the model
- Adjusted $R^2 = 1$ confirms the model is not overfitting
- Both metrics achieve their maximum value due to the perfect fit
- X_2 is redundant as it contributes no explanatory power ($\beta_2 = 0$)

Conclusion

The model $Y = 1 + X_1$ perfectly explains the data with both R-squared and Adjusted R-squared equal to 1.0. This indicates an excellent fit where X_1 completely determines Y , while X_2 provides no additional explanatory power.

Part 2: Non-parametric Regression: Nadaraya-Watson Estimator

Problem Statement

Given a new dataset to demonstrate non-parametric regression:

- $X = [1, 2, 3, 4, 5]$
- $Y = [2, 4, 1, 5, 3]$

Estimate $\hat{y}$ at $x = 2.5$ using the Nadaraya-Watson kernel regression with Gaussian kernel:

$$\hat{y}(x) = \frac{\sum_{i=1}^n K_h(x - X_i) Y_i}{\sum_{i=1}^n K_h(x - X_i)}$$

where $K_h(u) = \exp\left(-\frac{u^2}{2h^2}\right)$ and bandwidth $h = 1$.

Numerical Calculation

Step 1: Compute Distances

Compute distances from query point $x_q = 2.5$ to all X_i :

$$d_1 = |2.5 - 1| = 1.5$$

$$d_2 = |2.5 - 2| = 0.5$$

$$d_3 = |2.5 - 3| = 0.5$$

$$d_4 = |2.5 - 4| = 1.5$$

$$d_5 = |2.5 - 5| = 2.5$$

Step 2: Compute Kernel Weights

Gaussian kernel: $K_h(u) = \exp\left(-\frac{u^2}{2h^2}\right)$ with $h = 1$:

$$w_1 = \exp\left(-\frac{(1.5)^2}{2 \times 1^2}\right) = \exp\left(-\frac{2.25}{2}\right) = e^{-1.125} \approx 0.3247$$

$$w_2 = \exp\left(-\frac{(0.5)^2}{2}\right) = e^{-0.125} \approx 0.8825$$

$$w_3 = \exp\left(-\frac{(0.5)^2}{2}\right) = e^{-0.125} \approx 0.8825$$

$$w_4 = \exp\left(-\frac{(1.5)^2}{2}\right) = e^{-1.125} \approx 0.3247$$

$$w_5 = \exp\left(-\frac{(2.5)^2}{2}\right) = \exp\left(-\frac{6.25}{2}\right) = e^{-3.125} \approx 0.0440$$

Step 3: Compute Weighted Sums

Numerator (weighted Y sum):

$$\begin{aligned} \sum w_i Y_i &= (0.3247 \times 2) + (0.8825 \times 4) + (0.8825 \times 1) + (0.3247 \times 5) + (0.0440 \times 3) \\ &= 0.6494 + 3.5300 + 0.8825 + 1.6235 + 0.1320 = 6.8174 \end{aligned}$$

Denominator (sum of weights):

$$\sum w_i = 0.3247 + 0.8825 + 0.8825 + 0.3247 + 0.0440 = 2.4584$$

Step 4: Compute Estimate

$$\hat{y}(2.5) = \frac{\sum w_i Y_i}{\sum w_i} = \frac{6.8174}{2.4584} \approx 2.772$$

Bandwidth Sensitivity Analysis

Table 3.3: Effect of bandwidth on prediction at $x = 2.5$

Bandwidth (h)	Weights	$\hat{y}(2.5)$
0.5	$w = [0.0111, 0.8825, 0.8825, 0.0111, 0.0000]$ $\sum w_i = 1.7872$	$\frac{0.0222+3.5300+0.8825+0.0555}{1.7872} \approx 2.51$
1.0	As above	2.772
2.0	$w = [0.754, 0.939, 0.939, 0.754, 0.457]$ $\sum w_i = 3.843$	$\frac{1.508+3.756+0.939+3.770+1.371}{3.843} \approx 2.93$

Visualization of Kernel Weighting

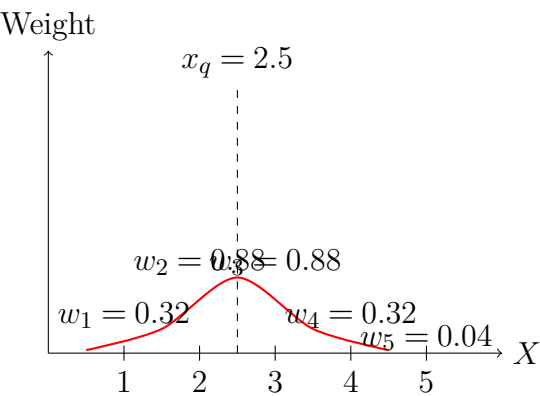


Figure 3.7: Kernel weights at $x_q = 2.5$ (Gaussian kernel, $h = 1$)

Key Observations

1. The Nadaraya-Watson estimator gives $\hat{y}(2.5) \approx 2.772$ for $h = 1$
2. Bandwidth h controls the smoothing:
 - Small h (0.5): More weight to nearest neighbors ($\hat{y} = 2.51$)
 - Large h (2.0): More global averaging ($\hat{y} = 2.93$)
3. The Gaussian kernel gives symmetric weights based on distance
4. Point $x = 2.5$ gets most influence from $X_2 = 2$ and $X_3 = 3$

Comparison with Linear Regression

Method	Prediction at $x = 2.5$	Characteristics
Linear Regression	3.5	Global parametric model
Nadaraya-Watson ($h = 1$)	2.77	Local non-parametric

Conclusion

The Nadaraya-Watson estimator provides a flexible, data-driven approach to regression without assuming a functional form. The bandwidth parameter h critically controls the bias-variance tradeoff:

- Smaller h : Higher variance (more wiggly fit)
- Larger h : Higher bias (smoother fit)

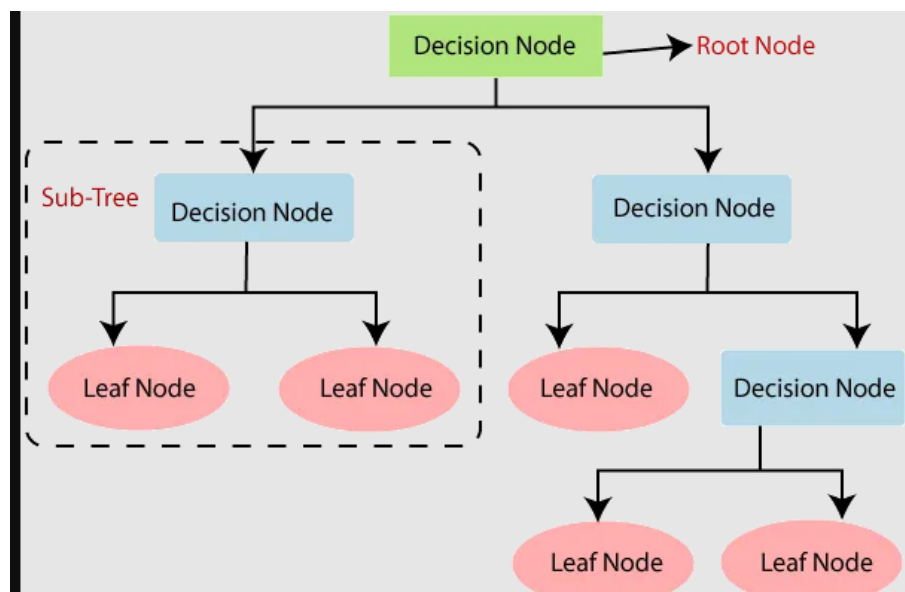
Kernel regression is particularly useful when relationships are non-linear or when no parametric model adequately describes the data.

CHAPTER FOUR: CATEGORICAL AND MIXED-TYPE DATA

4.1 What is a Decision Tree?

A **decision tree** is a type of supervised machine learning algorithm used for both **classification** and **regression** tasks.

It works by splitting the dataset into smaller and smaller subsets based on certain conditions, forming a tree-like structure. The goal is to find the *best splits* that help separate the data points clearly based on the target variable.



Components of a Decision Tree

- **Root Node:** The topmost node. It represents the entire dataset, and the first decision (split) is made here.
- **Internal/Decision Nodes:** These are the points where the dataset is split based on a feature.
- **Branches:** These represent the outcome of a split (e.g., **Yes** or **No**, $\leq$ threshold or $>$ threshold).
- **Leaf/Terminal Nodes:** These are the final outcomes (classes or predicted values).

How Does It Work?

The decision tree algorithm chooses the best feature and split point using a criterion such as:

- **Information Gain** (used in ID3)
- **Gain Ratio** (used in C4.5)
- **Gini Impurity** (used in CART)



It then recursively splits the data until one of the following conditions is met:

- All instances in a node belong to the same class (i.e., the node is **pure**).
- A predefined stopping criterion is satisfied (e.g., maximum depth reached, or minimum number of samples per leaf).

How Splitting Happens in Decision Trees

Decision trees split the data based on impurity measures. Here are the common ones:

1. Gini Impurity (Used in CART)

$$\text{Gini} = 1 - \sum_{i=1}^n p_i^2$$

Where p_i is the probability of class i . Gini measures how often a randomly chosen element would be incorrectly labeled.

2. Entropy (Used in ID3)

$$\text{Entropy} = - \sum_{i=1}^n p_i \log_2 p_i$$

Entropy measures the impurity or randomness in the data.

3. Information Gain

Used to decide the best feature to split on.

$$\text{Information Gain} = \text{Entropy}(\text{Parent}) - \sum \left(\frac{|\text{Child}|}{|\text{Parent}|} \times \text{Entropy}(\text{Child}) \right)$$

4. Mean Squared Error (MSE) — Regression Trees

Used for continuous target variables.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

Algorithms for Building Decision Trees

Algorithm	Key Idea	Use Case
ID3	Uses entropy and information gain	Classification
C4.5	Improvement over ID3; handles continuous data	Classification
CART	Uses Gini (classification) or MSE (regression)	Classification & Regression
CHAID	Uses statistical significance tests	Multi-level categorical splits

Pruning: Avoiding Overfitting

Decision trees can easily overfit the training data. **Pruning** helps by simplifying the model.

- **Pre-pruning (Early Stopping):** Stop growing the tree if a condition is met (e.g., minimum samples, max depth).
- **Post-pruning:** First grow the full tree, then remove nodes that provide little to no improvement.

Pros and Cons of Decision Trees

Advantages

- Easy to understand and visualize
- No need for feature scaling or normalization
- Works with both numerical and categorical data

Disadvantages

- Prone to overfitting
- Can be unstable — small data changes may produce different trees
- Biased toward features with many levels/categories

Real-world Applications

- Medical Diagnosis (e.g., classifying tumors as benign or malignant)
- Credit Scoring
- Customer Churn Prediction
- Fraud Detection

4.1.1 What ID3 Does

ID3 builds a decision tree top-down, starting from the root and recursively selecting the best attribute to split the data. It uses **information gain** to decide which attribute to split on.

Key Concepts

1. Entropy

Entropy is a measure of impurity or disorder in a dataset. It is calculated as:

$$\text{Entropy}(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

where:

- S is a set of examples,
- p_i is the proportion of examples in class i ,
- c is the number of classes.

2. Information Gain

Information Gain measures how much “information” a feature gives us about the class. It is defined as:

$$IG(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

where:

- A is the attribute,
- S_v is the subset of S for which attribute A has value v .

The attribute with the highest information gain is selected to split the node.

ID3 Algorithm Steps

1. Start with the full dataset.
2. Compute entropy of the dataset.
3. For each attribute, compute information gain.
4. Choose the attribute with the highest information gain to split the dataset.
5. Repeat recursively for each subset until:

- All examples in a subset belong to the same class,
- There are no more attributes to split on,
- The subset is empty.

Dataset Summary

Outlook	Temperature	Humidity	Windy	PlayGolf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

We have 14 instances with the target variable **PlayGolf**, which has 9 **Yes** and 5 **No** instances.

1. ID3 (Information Gain with Entropy)

Step 1: Total Dataset Summary

There are 14 records in total.

Play Golf	Count
Yes	9
No	5

Initial Entropy $E(S)$:

$$\begin{aligned}
E(S) &= -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) \\
&= -0.643 \cdot \log_2(0.643) - 0.357 \cdot \log_2(0.357) \approx 0.940
\end{aligned}$$

Step 2: Compute Information Gain for Each Attribute

We now calculate the expected entropy for each attribute split.

Attribute: Outlook

- Sunny: 5 samples (2 Yes, 3 No)

$$E = -\frac{2}{5} \log_2 \left(\frac{2}{5} \right) - \frac{3}{5} \log_2 \left(\frac{3}{5} \right) \approx 0.971$$

- Overcast: 4 samples (4 Yes, 0 No)

$$E = 0$$

- Rainy: 5 samples (3 Yes, 2 No)

$$E = -\frac{3}{5} \log_2 \left(\frac{3}{5} \right) - \frac{2}{5} \log_2 \left(\frac{2}{5} \right) \approx 0.971$$

Information Gain:

$$Gain(S, Outlook) = 0.940 - \left(\frac{5}{14} \cdot 0.971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.971 \right) = 0.940 - (0.693) = \boxed{0.247}$$

Attribute: Temperature

- Hot: 4 samples (2 Yes, 2 No), $E = 1.0$
- Mild: 6 samples (4 Yes, 2 No),

$$E = -\frac{4}{6} \log_2 \left(\frac{4}{6} \right) - \frac{2}{6} \log_2 \left(\frac{2}{6} \right) \approx 0.918$$

- Cool: 4 samples (3 Yes, 1 No), $E \approx 0.811$

$$Gain(S, Temp) = 0.940 - (0.286 + 0.393 + 0.232) = 0.029$$

Attribute: Humidity

- High: 7 samples (3 Yes, 4 No), $E \approx 0.985$
- Normal: 7 samples (6 Yes, 1 No), $E \approx 0.591$

$$Gain(S, Humidity) = 0.940 - \left(\frac{7}{14} \cdot 0.985 + \frac{7}{14} \cdot 0.591 \right) = 0.940 - (0.788) = \boxed{0.152}$$

Attribute: Windy

- False: 8 samples (6 Yes, 2 No), $E \approx 0.811$
- True: 6 samples (3 Yes, 3 No), $E = 1.0$

$$Gain(S, Windy) = 0.940 - \left(\frac{8}{14} \cdot 0.811 + \frac{6}{14} \cdot 1.0 \right) = 0.940 - 0.892 = \boxed{0.048}$$

Best attribute to split on: Outlook (Gain = 0.247)

Step 3: Split on Outlook

Outlook = Sunny

Subset size: 5 samples (2 Yes, 3 No) Entropy = 0.971

Split on **Humidity**:

- High: 3 samples (0 Yes, 3 No), $E = 0 \rightarrow$ No
- Normal: 2 samples (2 Yes, 0 No), $E = 0 \rightarrow$ Yes

Gain = 0.971

Outlook = Overcast

Subset size: 4 samples (4 Yes, 0 No) → Pure Node: Predict Yes

Outlook = Rainy

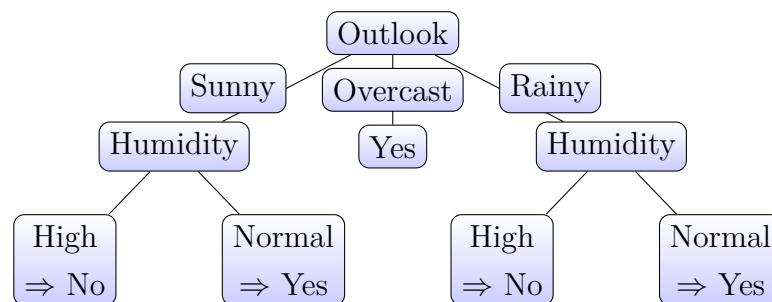
Subset size: 5 samples (3 Yes, 2 No) Entropy = 0.971

Try splitting on **Humidity**:

- High: 2 samples (0 Yes, 2 No), $E = 0 \rightarrow$ No
- Normal: 3 samples (3 Yes, 0 No), $E = 0 \rightarrow$ Yes

Gain = 0.971

Final Decision Tree



Summary of Leaf Nodes

Path	Prediction
Outlook = Overcast	Yes
Outlook = Sunny, Humidity = High	No
Outlook = Sunny, Humidity = Normal	Yes
Outlook = Rainy, Humidity = High	No
Outlook = Rainy, Humidity = Normal	Yes

4.1.2 CART (Gini Index)

Overview

The **CART (Classification and Regression Trees)** algorithm is a decision tree learning method introduced by *Breiman et al. (1986)*. It can be used for both:

- **Classification** (predicting class labels), and
- **Regression** (predicting continuous values).

Unlike algorithms such as ID3 or C4.5, CART produces strictly **binary trees**—each internal node splits into exactly two children.

Key Concepts

1. Gini Impurity (for classification)

CART uses **Gini Impurity** to measure the impurity of a dataset. It is calculated as:

$$Gini(S) = 1 - \sum_{i=1}^c p_i^2$$

Where:

- S is the dataset at a node,
- p_i is the proportion of samples belonging to class i ,
- c is the number of classes.

A lower Gini impurity indicates a purer node.

2. Mean Squared Error (for regression)

For regression tasks, CART uses **Mean Squared Error (MSE)** as the splitting criterion:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2$$

Where:

- y_i is the actual value,
- $\hat{y}$ is the predicted value (typically the mean of values in the node),
- n is the number of samples in the node.

CART Algorithm Steps

1. Start with the full dataset at the root node.
2. For each feature and potential split value, compute the impurity (Gini or MSE).
3. Select the split that **minimizes impurity** in the resulting child nodes.
4. Split the dataset into two child nodes based on this criterion.
5. Recurse on each child node until a stopping condition is met:
 - All samples belong to the same class,
 - Maximum tree depth is reached,
 - Minimum number of samples per node is not met,
 - No further information gain is possible.

Pruning

CART uses a method called **cost-complexity pruning** (also known as *weakest link pruning*):

- First, a fully grown tree is generated.
- Then, nodes are pruned based on a complexity parameter α that balances accuracy with tree size.

Features of CART

- **Tree Type:** Binary decision tree.

- **Splitting Criteria:** Gini impurity (classification), MSE (regression).
- **Supports:** Both categorical and numerical attributes.
- **Output:** Class label or numeric value.
- **Pruning:** Cost-complexity pruning supported.

Step 1: Gini Index of Full Dataset

$$Gini(S) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 0.459$$

Step 1: Dataset Summary

The dataset has 14 instances.

Play Golf	Count
Yes	9
No	5

Gini Index for root:

$$Gini(S) = 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 = 1 - 0.41 - 0.13 = \boxed{0.459}$$

Step 2: Try Splits Using Binary CART

CART evaluates **binary splits only** (e.g., Outlook = Sunny? Yes/No).

We'll compute weighted Gini for each binary split.

Split: Outlook = Sunny?

- Yes branch (Sunny): 5 samples (2 Yes, 3 No),

$$Gini = 1 - (2/5)^2 - (3/5)^2 = 0.48$$

- No branch (Not Sunny): 9 samples (7 Yes, 2 No),

$$Gini = 1 - (7/9)^2 - (2/9)^2 \approx 0.346$$

$$Gini_{split} = \frac{5}{14} \cdot 0.48 + \frac{9}{14} \cdot 0.346 \approx 0.398$$

Split: Outlook = Overcast?

- Yes branch (Overcast): 4 samples (4 Yes), Gini = 0
- No branch: 10 samples (5 Yes, 5 No),

$$Gini = 1 - (5/10)^2 - (5/10)^2 = 0.5$$

$$Gini_{split} = \frac{4}{14} \cdot 0 + \frac{10}{14} \cdot 0.5 = 0.357$$

Split: Humidity = High?

- High: 7 samples (3 Yes, 4 No),

$$Gini = 1 - (3/7)^2 - (4/7)^2 \approx 0.489$$

- Not High: 7 samples (6 Yes, 1 No),

$$Gini = 1 - (6/7)^2 - (1/7)^2 \approx 0.245$$

$$Gini_{split} = \frac{7}{14} \cdot 0.489 + \frac{7}{14} \cdot 0.245 = 0.367$$

Split: Windy = True?

- True: 6 samples (3 Yes, 3 No), Gini = 0.5

- False: 8 samples (6 Yes, 2 No),

$$Gini = 1 - (6/8)^2 - (2/8)^2 = 0.375$$

$$Gini_{split} = \frac{6}{14} \cdot 0.5 + \frac{8}{14} \cdot 0.375 = 0.429$$

Best Split: Outlook = Overcast? (Gini = 0.357)

—

Step 3: First Split - Outlook = Overcast?

Overcast = Yes → 4 samples (4 Yes) → Pure → **Yes**

Overcast = No → 10 samples (5 Yes, 5 No) → Needs further splitting

—

Step 4: Further Split on Outlook Overcast

Now use the remaining 10 samples to test other binary splits.

Try **Humidity = High?** → Gives next best split for this subset.

- High: 5 samples (0 Yes, 5 No) → Gini = 0
- Not High: 5 samples (5 Yes, 0 No) → Gini = 0

$$Gini_{split} = 0 \Rightarrow \text{Perfect split}$$

4.2 Variable Importance: Permutation Tests and Partial Dependence Plots

1. Permutation Feature Importance

Objective

To quantify the importance of each feature by measuring the change in model performance when the feature's values are randomly shuffled, breaking the relationship between that feature and the target.

Algorithm

1. Train the model on training data.
2. Evaluate a baseline performance metric (e.g., accuracy, RMSE):

$$M_{\text{base}} = \text{metric}(y, \hat{y})$$

3. For each feature X_i :
 - Shuffle its values in the validation/test set.
 - Recalculate predictions and evaluate the performance metric M_i .
 - Compute the importance score as:

$$I_i = M_i - M_{\text{base}}$$

Key Points

- **Model-agnostic:** Works with any type of model.
- **Captures feature interactions.**
- **Limitation:** Sensitive to correlated features.

Pros and Cons

- **Pros:** Simple, intuitive, supports any model.

- **Cons:** Computationally expensive; may underestimate importance of correlated features.

2. Partial Dependence Plots (PDP)

Objective

To visualize the marginal effect of one or two features on the predicted outcome by averaging out the effects of all other features.

Mathematical Formulation

Let $f(x)$ be the trained prediction function. The partial dependence of feature subset X_s is given by:

$$PD(X_s) = \mathbb{E}_{X_c}[f(X_s, X_c)]$$

Where:

- X_s : The feature(s) of interest.
- X_c : The complement set of all other features.
- The expectation is taken over the joint distribution of X_c .

Procedure

1. Select a feature X_s and a range of values for it.
2. For each value, substitute it in all rows of the dataset and make predictions.
3. Average the predictions to estimate the marginal effect at that value.

Key Points

- **Visual interpretation:** 1D or 2D plots.
- **Assumes feature independence:** Can be misleading if features are correlated.

- Often used with tree-based models like Random Forests or Gradient Boosted Trees.

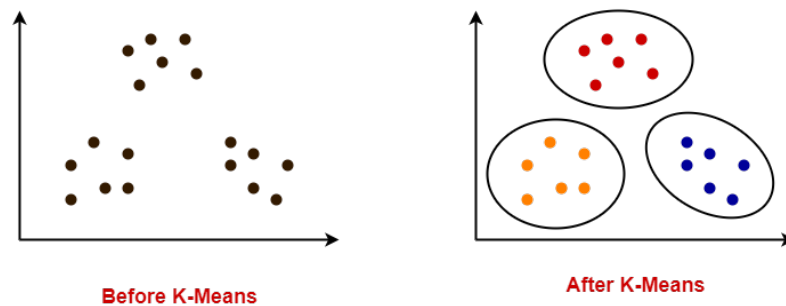
Comparison Table

Aspect	Permutation Importance	Partial Dependence Plot
Type	Quantitative metric	Visual interpretation
Output	Importance score	Plot of average model output
Captures Interactions	Yes	Yes (in 2D plots)
Feature Correlation Issue	Yes (underestimates importance)	Yes (can be misleading)
Model Compatibility	Any (model-agnostic)	Mostly tree-based models
Usefulness	Feature ranking	Understanding effect on prediction
Cost	Medium to High	Medium

Table 4.1: Comparison of Permutation Importance and PDP

4.3 K means clustering

K-Means clustering is an **unsupervised**, iterative clustering technique that partitions a given dataset into k predefined distinct clusters. A cluster is defined as a collection of data points exhibiting certain similarities.



Cluster Properties

The clustering is performed such that:

- Each data point belongs to a cluster whose mean (centroid) is nearest to it.
- Data points within the same cluster exhibit a high degree of similarity.

- Data points from different clusters show a high degree of dissimilarity.

K-Means Clustering Algorithm

The K-Means clustering algorithm involves the following steps:

Step-1: Choose the number of clusters k .

Step-2: Initialize cluster centers:

- Randomly select k data points as the initial cluster centers.
- Try to select them such that they are as far apart as possible.

Step-3: Assign data points to clusters:

- Calculate the distance between each data point and each cluster center.
- Commonly used distance metric: Euclidean distance.

Step-4: Cluster assignment:

- Assign each data point to the cluster with the nearest centroid.

Step-5: Recompute centroids:

- For each cluster, compute the new centroid by taking the mean of all data points assigned to that cluster.

Step-6: Repeat:

- Repeat steps 3 to 5 until one of the following conditions is met:
 - Cluster centers do not change.
 - Data point assignments remain unchanged.
 - A maximum number of iterations is reached.

Advantages

K-Means Clustering offers the following advantages:

- **Efficiency:** Time complexity is $\mathcal{O}(nkt)$, where:
 - n = number of data points,
 - k = number of clusters,
 - t = number of iterations.
- **Fast convergence:** The algorithm often converges quickly, although it may reach a local optimum.
- **Extensibility:** Techniques such as Simulated Annealing or Genetic Algorithms can be used to escape local optima and seek a global optimum.

Disadvantages

Despite its efficiency, K-Means has several limitations:

- Requires the number of clusters k to be specified in advance.
- Sensitive to noisy data and outliers.
- Poor performance with non-convex shaped clusters or clusters of varying density and size.

Given

- Number of clusters: $K = 2$
- Initial centroids:
 - $C_1 = (185, 72)$
 - $C_2 = (170, 56)$
- Data points:

Point	Coordinates
P_1	(185, 72)
P_2	(170, 56)
P_3	(168, 60)
P_4	(179, 68)
P_5	(182, 72)
P_6	(188, 77)

Iteration 1: Euclidean Distance

The Euclidean distance is calculated using:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Point	Coordinates	$d(P, C_1)$	$d(P, C_2)$	Cluster
P_1	(185, 72)	0	$\sqrt{15^2 + 16^2} = 21.93$	C_1
P_2	(170, 56)	21.93	0	C_2
P_3	(168, 60)	$\sqrt{17^2 + 12^2} = 20.80$	$\sqrt{2^2 + 4^2} = 4.47$	C_2
P_4	(179, 68)	$\sqrt{6^2 + 4^2} = 7.21$	$\sqrt{9^2 + 12^2} = 15.00$	C_1
P_5	(182, 72)	3.00	20.00	C_1
P_6	(188, 77)	$\sqrt{3^2 + 5^2} = 5.83$	27.66	C_1

Clusters after Iteration 1:

- C_1 : P_1, P_4, P_5, P_6
- C_2 : P_2, P_3

New Centroids

- For C_1 :

$$\text{Mean} = \left(\frac{185 + 179 + 182 + 188}{4}, \frac{72 + 68 + 72 + 77}{4} \right) = (183.5, 72.25)$$

- For C_2 :

$$\text{Mean} = \left(\frac{170 + 168}{2}, \frac{56 + 60}{2} \right) = (169, 58)$$

Iteration 2: Recalculate Distance

Point	Coordinates	$d(P, C'_1)$	$d(P, C'_2)$	Cluster
P_1	(185, 72)	$\sqrt{1.5^2 + 0.25^2} \approx 1.52$	$\sqrt{16^2 + 14^2} \approx 21.26$	C_1
P_2	(170, 56)	$\sqrt{13.5^2 + 16.25^2} \approx 21.13$	$\sqrt{1^2 + 2^2} \approx 2.24$	C_2
P_3	(168, 60)	$\sqrt{15.5^2 + 12.25^2} \approx 19.71$	$\sqrt{1^2 + 2^2} \approx 2.24$	C_2
P_4	(179, 68)	$\sqrt{4.5^2 + 4.25^2} \approx 6.20$	$\sqrt{10^2 + 10^2} \approx 14.14$	C_1
P_5	(182, 72)	$\sqrt{1.5^2 + 0.25^2} \approx 1.52$	$\sqrt{13^2 + 14^2} \approx 19.10$	C_1
P_6	(188, 77)	$\sqrt{4.5^2 + 4.75^2} \approx 6.55$	$\sqrt{19^2 + 19^2} \approx 26.87$	C_1

Clusters after Iteration 2:

- C_1 : P_1, P_4, P_5, P_6
- C_2 : P_2, P_3

Conclusion

After two iterations, the clusters did not change. Hence, the algorithm has converged. The final clusters are:

- **Cluster 1 (C_1):** (185, 72), (179, 68), (182, 72), (188, 77)
- **Cluster 2 (C_2):** (170, 56), (168, 60)

Problem Statement

Cluster the following eight 2D points using the K-Means algorithm with $K = 3$ and perform 2 iterations.

Data Points:

Point	Coordinates
A_1	(2, 10)
A_2	(2, 5)
A_3	(8, 4)
A_4	(5, 8)
A_5	(7, 5)
A_6	(6, 4)
A_7	(1, 2)
A_8	(4, 9)

Initial Cluster Centers:

- Cluster 1 (C1): $A_1 = (2, 10)$
- Cluster 2 (C2): $A_4 = (5, 8)$
- Cluster 3 (C3): $A_7 = (1, 2)$

Iteration 1: Distance Calculation

Distance formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Point	Coordinates	$d(C1)$	$d(C2)$	$d(C3)$	Cluster
A_1	(2, 10)	0.00	3.61	8.06	$C1$
A_2	(2, 5)	5.00	3.61	3.16	$C3$
A_3	(8, 4)	8.48	5.00	7.28	$C2$
A_4	(5, 8)	3.61	0.00	6.71	$C2$
A_5	(7, 5)	7.62	3.61	6.08	$C2$
A_6	(6, 4)	7.21	4.47	5.39	$C2$
A_7	(1, 2)	8.06	6.71	0.00	$C3$
A_8	(4, 9)	2.24	1.41	7.28	$C2$

Clusters after Iteration 1:

- Cluster 1 (C1): A_1

- Cluster 2 (C2): A_3, A_4, A_5, A_6, A_8
- Cluster 3 (C3): A_2, A_7

New Centroids After Iteration 1

C1: Only $A_1 \rightarrow (2, 10)$

C2:

$$x = \frac{8+5+7+6+4}{5} = 6, \quad y = \frac{4+8+5+4+9}{5} = 6$$

$$\Rightarrow C2 = (6, 6)$$

C3:

$$x = \frac{2+1}{2} = 1.5, \quad y = \frac{5+2}{2} = 3.5$$

$$\Rightarrow C3 = (1.5, 3.5)$$

Iteration 2: Distance Calculation

Point	Coordinates	$d(C1 : 2, 10)$	$d(C2 : 6, 6)$	$d(C3 : 1.5, 3.5)$	Cluster
A_1	(2, 10)	0.00	5.00	6.54	$C1$
A_2	(2, 5)	5.00	4.12	1.80	$C3$
A_3	(8, 4)	8.48	2.83	6.55	$C2$
A_4	(5, 8)	3.61	2.24	5.70	$C2$
A_5	(7, 5)	7.62	1.41	5.70	$C2$
A_6	(6, 4)	7.21	2.00	4.53	$C2$
A_7	(1, 2)	8.06	5.00	1.58	$C3$
A_8	(4, 9)	2.24	3.61	6.40	$C1$

Clusters after Iteration 2:

- Cluster 1 (C1): A_1, A_8
- Cluster 2 (C2): A_3, A_4, A_5, A_6
- Cluster 3 (C3): A_2, A_7

New Centroids After Iteration 2

C1:

$$x = \frac{2+4}{2} = 3, \quad y = \frac{10+9}{2} = 9.5 \Rightarrow C1 = (3, 9.5)$$

C2:

$$x = \frac{8+5+7+6}{4} = 6.5, \quad y = \frac{4+8+5+4}{4} = 5.25 \Rightarrow C2 = (6.5, 5.25)$$

C3:

$$x = \frac{2+1}{2} = 1.5, \quad y = \frac{5+2}{2} = 3.5 \Rightarrow C3 = (1.5, 3.5)$$

Final Answer

After 2 iterations, the final cluster centers are:

- Cluster 1: (3, 9.5)
- Cluster 2: (6.5, 5.25)
- Cluster 3: (1.5, 3.5)

4.4 Clustering Mixed-Type Data: K-Means and Distance Metrics

Handling **mixed-type data** (numerical + categorical variables) requires specialized approaches since standard K-means and Euclidean distance are incompatible with categorical features.

Data Preprocessing & Distance Metrics

One-Hot Encoding + Euclidean Distance

- **Approach:** Convert categorical variables to binary columns
- **Issues:**
 - High dimensionality and sparsity distort distance
 - Centroids become non-interpretable (fractional values)

- Suitable only for low-cardinality categorical features

4.4.1 Gower Distance

Ideal unified metric for mixed data:

- **Numerical:** Manhattan distance normalized by range
- **Categorical:** Dice similarity (1 if same, 0 if different)

$$d_{\text{Gower}}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\sum_{k=1}^p w_k \delta_{ijk}}{\sum_{k=1}^p w_k}$$

where δ_{ijk} is feature-specific distance and w_k is optional weight.

Advantages: Natural mixed-type handling, normalized $[0,1]$ distances

Limitation: $O(n^2)$ computational complexity

Other Metrics

- **Hamming Distance:** Mismatch count for categorical features
- **Weighted Combinations:** Custom blends (e.g., Euclidean + Hamming)

Clustering Algorithms

4.4.2 K-Prototypes

Hybrid of K-Means (numerical) and K-Modes (categorical):

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sum_{k \in \text{num}} (x_{ik} - x_{jk})^2 + \gamma \sum_{k \in \text{cat}} \mathbb{1}(x_{ik} \neq x_{jk})$$

where γ balances feature-type contributions.

Implementation:

```
from kmodes.kprototypes import KPrototypes
# Preprocess: Scale numericals, encode categoricals as integers
kp = KPrototypes(n_clusters=3, init='Cao')
clusters = kp.fit_predict(data, categorical=[0, 2])
```


K-Medoids with Gower Distance

PAM algorithm using actual data points as centroids:

- Robust to noise
- Compatible with any distance matrix

Implementation:

```
import gower
from sklearn_extra.cluster import KMedoids
dist_matrix = gower.gower_matrix(data)
kmed = KMedoids(n_clusters=3, metric="precomputed")
labels = kmed.fit_predict(dist_matrix)
```

FAMD + K-Means

Dimensionality reduction for mixed data:

- FAMD projects data into continuous latent space
- Preserves variance in both feature types

Implementation:

```
from prince import FAMD
from sklearn.cluster import KMeans

famd = FAMD(n_components=5).fit(data)
kmeans = KMeans(n_clusters=3).fit(famd.row_coordinates_)
```

Practical Considerations

4.4.3 Preprocessing

- Scale numerical features (e.g., StandardScaler)
- Encode categoricals as integers or one-hot

Balancing Feature Types

- Use γ parameter in K-Prototypes
- Apply weights in Gower distance

Cluster Validation

- Silhouette Score with Gower distance
- Inspect cluster characteristics (categorical modes)

Limitations

- K-Prototypes assumes spherical clusters
- Gower distance scales poorly for large datasets
- No universal solution - requires experimentation

Method Comparison

Table 4.2: Summary of Clustering Methods for Mixed-Type Data

Method	Distance Metric	Algorithm	Scalability
K-Prototypes	Hybrid (Euclidean + Dissimilarity)	Custom	High
Gower + K-Medoids	Gower	PAM	Low
One-Hot + K-Means	Euclidean	K-Means	Medium
FAMD + K-Means	Euclidean (latent space)	K-Means	Medium

Recommendations

- **K-Prototypes:** Best for efficiency and implementation simplicity
- **Gower + K-Medoids:** Preferred for accuracy in small-to-medium datasets
- Always validate with domain knowledge and multiple metrics

4.5 Logistic Regression

What is Logistic Regression?

Logistic Regression is a supervised machine learning algorithm used primarily for classification tasks.

Unlike Linear Regression, which predicts continuous values, Logistic Regression estimates the probability that a given input belongs to a particular class.

It is especially useful for binary classification problems where the target variable has only two classes:

- Yes / No
- 0 / 1
- True / False

Types of Logistic Regression

Type	Description
Binomial Logistic Regression	Target variable has two categories (e.g., Yes/No)
Multinomial Logistic Regression	Target has 3+ unordered categories (e.g., cat/dog/sheep)
Ordinal Logistic Regression	Target has 3+ ordered categories (e.g., Low/Med/High)

Assumptions of Logistic Regression

- **Independent Observations:** Each sample is independent.
- **Binary Output (for binary case):** Target variable should be 0 or 1.
- **Linearity of Logit:** Linear relationship between predictors and log-odds.
- **No Outliers:** Outliers can distort parameter estimation.
- **Sufficient Sample Size:** Needed for stability and accuracy.

The Sigmoid Function

The sigmoid function transforms any real-valued number into a value between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\lim_{z \rightarrow \infty} \sigma(z) = 1, \quad \lim_{z \rightarrow -\infty} \sigma(z) = 0$$

Decision Rule:

$$\sigma(z) \geq 0.5 \Rightarrow \text{Class 1}; \quad \text{Else Class 0}$$

Mathematical Derivation

Step 1: Linear Function

$$z = w \cdot X + b$$

Where:

- X = feature vector
- w = weights
- b = bias/intercept

Step 2: Apply Sigmoid

$$p(X) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

This gives the probability of belonging to Class 1.

Logistic Regression as Log-Odds

Odds:

$$\text{Odds} = \frac{p(x)}{1 - p(x)} = e^z$$

Log-Odds (Logit):

$$\log \left(\frac{p(x)}{1-p(x)} \right) = w \cdot X + b$$

Likelihood Function

Likelihood:

$$L(w, b) = \prod_{i=1}^n p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}$$

Log-Likelihood:

$$\log L(w, b) = \sum_{i=1}^n [y_i \log(p(x_i)) + (1 - y_i) \log(1 - p(x_i))]$$

Gradient for Optimization (Gradient Ascent)

$$\frac{\partial J}{\partial w_j} = \sum_{i=1}^n (y_i - p(x_i)) x_{ij}$$

Weight Update Rule:

$$w_j := w_j + \alpha \cdot \frac{\partial J}{\partial w_j}$$

Softmax in Multiclass Logistic Regression

Given K classes, the softmax function:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Final probability for class c :

$$P(y = c \mid x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{k=1}^K e^{w_k \cdot x + b_k}}$$

Logistic vs Linear Regression

Aspect	Linear Regression	Logistic Regression
Output	Continuous value	Probability (0 to 1)
Type of Problem	Regression	Classification
Estimation Technique	Least Squares	Maximum Likelihood
Model	Best-fit line	Sigmoid curve (S-shaped)
Output Range	$(-\infty, +\infty)$	$(0, 1)$
Assumptions	Linearity between X and Y	Linearity between X and log-odds

Common Terminologies

Term	Description
Independent Variables (X)	Input features or predictors
Dependent Variable (Y)	Target class label (binary or multiclass)
Odds	$\frac{P}{1-P}$
Logit	$\log\left(\frac{P}{1-P}\right)$
Coefficients	Weights showing effect of input features
Intercept	Constant term in the logit equation
MLE	Maximum Likelihood Estimation for parameter learning

Evaluation Metrics

Metric	Formula	Use Case
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	General performance
Precision	$\frac{TP}{TP+FP}$	Importance of positive class
Recall	$\frac{TP}{TP+FN}$	Catching all positive cases
F1-Score	$2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Trade-off between P and R
AUC-ROC	Area under ROC Curve	Probabilistic ranking quality

Example: Numerical Calculation

Consider a dataset with a single feature:

x	y
2	0
4	0
6	1
8	1

Let the initial parameters be:

$$\theta_0 = -4, \quad \theta_1 = 1$$

Prediction for $x = 6$

Step 1: Compute z

$$z = \theta_0 + \theta_1 x = -4 + 1 \cdot 6 = 2$$

Step 2: Apply sigmoid function

$$h_{\theta}(x) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-2}} \approx \frac{1}{1 + 0.1353} \approx 0.88$$

Interpretation: There is an 88% probability the class is 1 $\Rightarrow$ **Predict 1**

Prediction for $x = 2$

Step 1: Compute z

$$z = \theta_0 + \theta_1 x = -4 + 1 \cdot 2 = -2$$

Step 2: Apply sigmoid function

$$h_{\theta}(x) = \frac{1}{1 + e^{-(-2)}} = \frac{1}{1 + e^2} \approx \frac{1}{1 + 7.389} \approx 0.119$$

Interpretation: About 11.9% chance the class is 1 $\Rightarrow$ **Predict 0**

Applications

Logistic Regression is widely used in various domains due to its simplicity and interpretability. Some common applications include:

- **Email Spam Detection**
- **Disease Prediction (e.g., Diabetes)**
- **Credit Risk Scoring**
- **Marketing Campaign Success Prediction**

Advantages and Disadvantages

Advantages

- Simple and fast to implement.
- Provides a probabilistic interpretation of the outcome.
- Works well with linearly separable data.

Disadvantages

- Assumes a linear decision boundary, which may not hold for complex datasets.
- May underperform when the relationship between features and target is non-linear or intricate.

Overview

① Introduction of Time Series

- Categories and Terminologies
- White Noise and Random Walk
- Time Series Analysis

② ARIMA Models

- AR Process
- MA Process
- ARMA Models
- ARIMA Models

③ ARIMA Modeling: A Toy Problem

Time Series



- A time series is a sequential set of data points, measured typically over successive times.
- Time series analysis comprises methods for analyzing time series data in order to extract meaningful statistics and other characteristics of the data.

Categories and Terminologies

- **Time-domain vs. Frequency-domain**
 - Time-domain approach: how does what happened today affect what will happen tomorrow?
These approaches view the investigation of **lagged relationships** as most important, e.g. autocorrelation analysis.
 - Frequency-domain approach: what is the economic cycle through periods of expansion and recession?
These approaches view the investigation of **cycles** as most important, e.g. spectral analysis and wavelet analysis.
- This lecture will focus on time-domain approaches.

Categories and Terminologies (cont.)

- **univariate** vs. **multivariate**

A time series containing records of a single variable is termed as univariate, but if records of more than one variable are considered then it is termed as multivariate.

- **linear** vs. **non-linear**

A time series model is said to be linear or non-linear depending on whether the current value of the series is a linear or non-linear function of past observations.

- **discrete** vs. **continuous**

In a continuous time series observations are measured at every instance of time, whereas a discrete time series contains observations measured at discrete points in time.

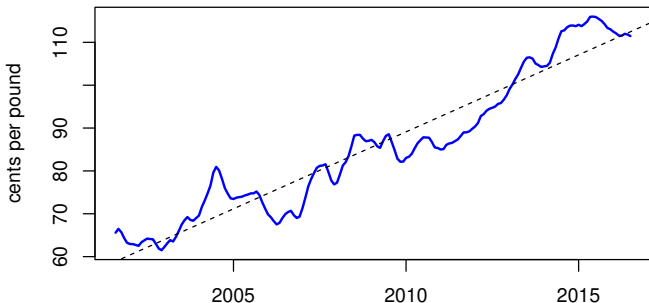
- This lecture will focus on univariate, linear, discrete time series.

Components of a Time Series

- In general, a time series is affected by four components, i.e. **trend**, **seasonal**, **cyclical** and **irregular** components.

- **Trend**

The general tendency of a time series to increase, decrease or stagnate over a long period of time.

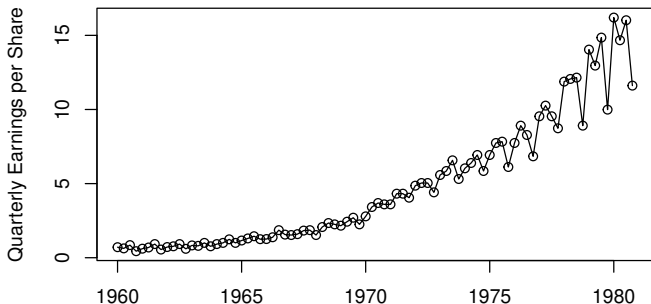


The price of chicken: monthly whole bird spot price, Georgia docks, US cents per pound, August 2001 to July 2016, with fitted linear trend line.

Components of a Time Series (cont.)

- In general, a time series is affected by four components, i.e. **trend**, **seasonal**, **cyclical** and **irregular** components.
 - **Seasonal variation**

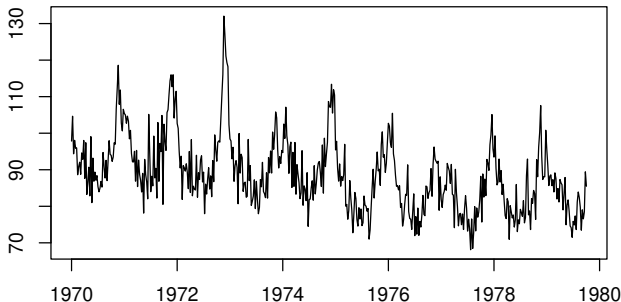
This component explains fluctuations within a year during the season, usually caused by climate and weather conditions, customs, traditional habits, etc.



Johnson & Johnson quarterly earnings per share, 84 quarters, 1960-I to 1980-IV.

Components of a Time Series (cont.)

- In general, a time series is affected by four components, i.e. **trend**, **seasonal**, **cyclical** and **irregular** components.
 - **Cyclical variation**
This component describes the medium-term changes caused by circumstances, which repeat in cycles. The duration of a cycle extends over longer period of time.



Average weekly cardiovascular mortality in Los Angeles County. There are 508 six-day smoothed averages obtained by filtering daily values over the 10 year period 1970-1979.

Components of a Time Series (cont.)

- In general, a time series is affected by four components, i.e. **trend**, **seasonal**, **cyclical** and **irregular** components.
 - **Irregular variation**

Irregular or random variations in a time series are caused by unpredictable influences, which are not regular and also do not repeat in a particular pattern.

These variations are caused by incidences such as war, strike, earthquake, flood, revolution, etc.

There is no defined statistical technique for measuring random fluctuations in a time series.

Combination of Four Components

- Considering the effects of these four components, two different types of models are generally used for a time series.

- Additive Model

$$Y(t) = T(t) + S(t) + C(t) + I(t)$$

Assumption: These four components are independent of each other.

- Multiplicative Model

$$Y(t) = T(t) \times S(t) \times C(t) \times I(t)$$

Assumption: These four components of a time series are not necessarily independent and they can affect one another.

Time Series Example: White Noise

- **White Noise**

- A simple time series could be a collection of **uncorrelated** random variables, $\{w_t\}$, with zero mean $\mu = 0$ and finite variance σ_w^2 , denoted as $w_t \sim wn(0, \sigma_w^2)$.

- **Gaussian White Noise**

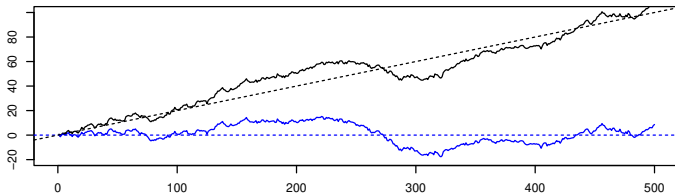
- A particular useful white noise is Gaussian white noise, wherein the w_t are **independent normal random variables** (with mean 0 and variance σ_w^2), denoted as $w_t \sim \text{iid } \mathcal{N}(0, \sigma_w^2)$.

- White noise time series is of great interest because if the stochastic behavior of all time series could be explained in terms of the white noise model, then classical statistical methods would suffice.

Time Series Example: Random Walk

- A random walk is the process by which randomly-moving objects wander away from where they started.
- Consider a simple 1-D process:
 - The value of the time series at time t is the value of the series at time $t - 1$ plus a completely random movement determined by w_t . More generally, a constant drift factor δ is introduced.

$$X_t = \delta + X_{t-1} + w_t = \delta t + \sum_{i=1}^t w_i$$



Time Series Analysis

- The procedure of using known data values to fit a time series with suitable model and estimating the corresponding parameters. It comprises methods that attempt to understand the nature of the time series and is often useful for future forecasting and simulation.
- There are several ways to build time series forecasting models, but this lecture will focus on **stochastic process**.
 - We assume a time series can be defined as a collection of **random variables** indexed according to the order they are obtained in time, $X_1, X_2, X_3, \dots$. t will typically be discrete and vary over the integers $t = 0, \pm 1, \pm 2, \dots$.
 - Note that the collection of random variables $\{X_t\}$ is referred to as a **stochastic process**, while the observed values are referred to as a **realization** of the stochastic process.

Measures of Dependence

- A complete description of a time series, observed as a collection of n random variables at arbitrary time points $t_1, t_2, \dots, t_n$, for any positive integer n , is provided by the **joint distribution function**, evaluated as the probability that the values of the series are jointly less than the n constants, $c_1, c_2, \dots, c_n$; i.e.,

$$F_{t_1, t_2, \dots, t_n}(c_1, c_2, \dots, c_n) = \Pr(X_{t_1} \leq c_1, X_{t_2} \leq c_2, \dots, X_{t_n} \leq c_n).$$

- Unfortunately, these multidimensional distribution functions cannot usually be written easily.
- Therefore some informative descriptive measures can be useful, such as **mean function** and more.

Measurement Functions

- **Mean function**

- The mean function is defined as

$$\mu_t = \mu_{X_t} = E[X_t] = \int_{-\infty}^{\infty} x f_t(x) dx,$$

provided it exists, where E denotes the usual expected value operator.

- Clearly for white noise series, $\mu_{w_t} = E[w_t] = 0$ for all t .
- For random walk with drift ($\delta \neq 0$),

$$\mu_{X_t} = E[X_t] = \delta t + \sum_{i=1}^t E[w_i] = \delta t$$

Autocovariance for Time Series

- Lack of independence between adjacent values in time series X_s and X_t can be numerically assessed.
- **Autocovariance Function**
 - Assuming the variance of X_t is finite, the autocovariance function is defined as the second moment product

$$\gamma(s, t) = \gamma_X(s, t) = \text{cov}(X_s, X_t) = E[(X_s - \mu_s)(X_t - \mu_t)],$$

for all s and t .

- Note that $\gamma(s, t) = \gamma(t, s)$ for all time points s and t .
- The autocovariance measures the **linear dependence** between two points on the same series observed at different times.
 - Very smooth series exhibit autocovariance functions that stay large even when the t and s are far apart, whereas choppy series tend to have autocovariance functions that are nearly zero for large separations.

Autocorrelation for Time Series

- **Autocorrelation Function (ACF)**

- The autocorrelation function is defined as

$$\rho(s, t) = \frac{\gamma(s, t)}{\sqrt{\gamma(s, s)\gamma(t, t)}}$$

- According to Cauchy-Schwarz inequality

$$|\gamma(s, t)|^2 \leq \gamma(s, s)\gamma(t, t),$$

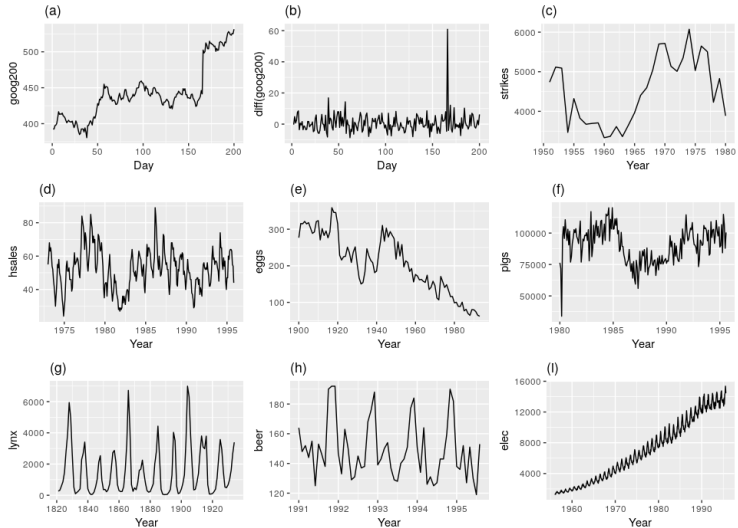
it's easy to show that $-1 \leq \rho(s, t) \leq 1$.

- ACF measures the **linear predictability** of X_t using only X_s .
 - If we can predict X_t perfectly from X_s through a linear relationship, then ACF will be either $+1$ or -1 .

Stationarity of Stochastic Process

- Forecasting is difficult as time series is non-deterministic in nature, i.e. we cannot predict with certainty what will occur in the future.
- But the problem could be a little bit easier if the time series is **stationary**: you simply predict its statistical properties will be the same in the future as they have been in the past!
 - A stationary time series is one whose statistical properties such as mean, variance, autocorrelation, etc. are all **constant over time**.
- Most statistical forecasting methods are based on the assumption that the time series can be rendered approximately stationary after mathematical transformations.

Which of these are stationary?



Strict Stationarity

- There are two types of stationarity, i.e. **strictly stationary** and **weakly stationary**.
- **Strict Stationarity**
 - The time series $\{X_t, t \in \mathbb{Z}\}$ is said to be strictly stationary if the **joint distribution** of $(X_{t_1}, X_{t_2}, \dots, X_{t_k})$ is the **same** as that of $(X_{t_1+h}, X_{t_2+h}, \dots, X_{t_k+h})$.
 - In other words, strict stationarity means that the joint distribution only depends on the “difference” h , not the time $(t_1, t_2, \dots, t_k)$.
- However in most applications this stationary condition is too strong.

Weak Stationarity

- **Weak Stationarity**

- The time series $\{X_t, t \in \mathbb{Z}\}$ is said to be weakly stationary if

- ① $E[X_t^2] < \infty, \quad \forall t \in \mathbb{Z};$

- ② $E[X_t] = \mu, \quad \forall t \in \mathbb{Z};$

- ③ $\gamma_X(s, t) = \gamma_X(s + h, t + h), \quad \forall s, t, h \in \mathbb{Z}.$

- In other words, a weakly stationary time series $\{X_t\}$ must have **three features**: finite variation, constant first moment, and that the second moment $\gamma_X(s, t)$ only depends on $|t - s|$ and not depends on s or t .

- Usually the term *stationary* means **weakly stationary**, and when people want to emphasize a process is stationary in the strict sense, they will use **strictly stationary**.

Remarks on Stationarity

- Strict stationarity does not assume finite variance thus strictly stationary does **NOT** necessarily imply weakly stationary.
 - Processes like i.i.d Cauchy is strictly stationary but not weakly stationary.
- A **nonlinear function** of a strictly stationary time series is still strictly stationary, but this is not true for weakly stationary.
- Weak stationarity usually does not imply strict stationarity as higher moments of the process may depend on time t .
- If time series $\{X_t\}$ is Gaussian (i.e. the distribution functions of $\{X_t\}$ are all multivariate Gaussian), then weakly stationary also implies strictly stationary. This is because a multivariate Gaussian distribution is fully characterized by its first two moments.

Autocorrelation for Stationary Time Series

- Recall that the autocovariance $\gamma_X(s, t)$ of stationary time series depends on s and t only through $|s - t|$, thus we can rewrite notation $s = t + h$, where h represents the time shift.

$$\gamma_X(t + h, t) = \text{cov}(X_{t+h}, X_t) = \text{cov}(X_h, X_0) = \gamma(h, 0) = \gamma(h)$$

- Autocovariance Function of Stationary Time Series**

$$\gamma(h) = \text{cov}(X_{t+h}, X_t) = E[(X_{t+h} - \mu)(X_t - \mu)]$$

- Autocorrelation Function of Stationary Time Series**

$$\rho(h) = \frac{\gamma(t + h, t)}{\sqrt{\gamma(t + h, t + h)\gamma(t, t)}} = \frac{\gamma(h)}{\gamma(0)}$$

Partial Autocorrelation

- Another important measure is called partial autocorrelation, which is the correlation between X_s and X_t with the linear effect of “everything in the middle” removed.
- **Partial Autocorrelation Function (PACF)**
 - For a stationary process X_t , the PACF (denoted as ϕ_{hh}), for $h = 1, 2, \dots$ is defined as

$$\phi_{11} = \text{corr}(X_{t+1}, X_t) = \rho_1$$

$$\phi_{hh} = \text{corr}(X_{t+h} - \hat{X}_{t+h}, X_t - \hat{X}_t), \quad h \geq 2$$

where $\hat{X}_{t+h}$ and $\hat{X}_t$ is defined as:

$$\hat{X}_{t+h} = \beta_1 X_{t+h-1} + \beta_2 X_{t+h-2} + \dots + \beta_{h-1} X_{t+1}$$

$$\hat{X}_t = \beta_1 X_{t+1} + \beta_2 X_{t+2} + \dots + \beta_{h-1} X_{t+h-1}$$

- If X_t is Gaussian, then ϕ_{hh} is actually conditional correlation

$$\phi_{hh} = \text{corr}(X_t, X_{t+h} | X_{t+1}, X_{t+2}, \dots, X_{t+h-1})$$

ARIMA Models

- ARIMA is an acronym that stands for **A**uto-**R**egressive **I**ntegrated **M**oving **A**verage. Specifically,
 - **AR** *Autoregression*. A model that uses the dependent relationship between an observation and some number of lagged observations.
 - **I** *Integrated*. The use of differencing of raw observations in order to make the time series stationary.
 - **MA** *Moving Average*. A model that uses the dependency between an observation and a residual error from a moving average model applied to lagged observations.
- Each of these components are explicitly specified in the model as a parameter.
- Note that **AR** and **MA** are two widely used linear models that work on stationary time series, and **I** is a preprocessing procedure to “stationarize” time series if needed.

Notations

- A standard notation is used of $ARIMA(p, d, q)$ where the parameters are substituted with integer values to quickly indicate the specific ARIMA model being used.
 - **p** The number of lag observations included in the model, also called the **lag order**.
 - **d** The number of times that the raw observations are differenced, also called the **degree of differencing**.
 - **q** The size of the moving average window, also called the **order of moving average**.
- A value of 0 can be used for a parameter, which indicates to not use that element of the model.
- In other words, ARIMA model can be configured to perform the function of an ARMA model, and even a simple AR, I, or MA model.

Autoregressive Models

- **Intuition**

- Autoregressive models are based on the idea that current value of the series, X_t , can be explained as a **linear combination** of p past values, $X_{t-1}, X_{t-2}, \dots, X_{t-p}$, together with a **random error** in the same series.

- **Definition**

- An autoregressive model of order p , abbreviated $AR(p)$, is of the form

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \dots + \phi_p X_{t-p} + w_t = \sum_{i=1}^p \phi_i X_{t-i} + w_t$$

where X_t is stationary, $w_t \sim wn(0, \sigma_w^2)$, and $\phi_1, \phi_2, \dots, \phi_p$ ($\phi_p \neq 0$) are model parameters. The hyperparameter p represents the length of the “direct look back” in the series.

Backshift Operator

- Before we dive deeper into the AR process, we need some new notations to simplify the representations.
- **Backshift Operator**
 - The backshift operator is defined as

$$BX_t = X_{t-1}.$$

It can be extended, $B^2X_t = B(BX_t) = B(X_{t-1}) = X_{t-2}$, and so on. Thus,

$$B^kX_t = X_{t-k}$$

- We can also define an inverse operator (*forward-shift operator*) by enforcing $B^{-1}B = 1$, such that

$$X_t = B^{-1}BX_t = B^{-1}X_{t-1}.$$

Autoregressive Operator of AR Process

- Recall the definition for $AR(p)$ process:

$$X_t = \phi_1 X_{t-1} + \phi_2 X_{t-2} + \cdots + \phi_p X_{t-p} + w_t$$

By using the backshift operator we can rewrite it as:

$$\begin{aligned} X_t - \phi_1 X_{t-1} - \phi_2 X_{t-2} - \cdots - \phi_p X_{t-p} &= w_t \\ (1 - \phi_1 B - \phi_2 B^2 - \cdots - \phi_p B^p) X_t &= w_t \end{aligned}$$

- The **autoregressive operator** is defined as:

$$\phi(B) = 1 - \phi_1 B - \phi_2 B^2 - \cdots - \phi_p B^p = 1 - \sum_{j=1}^p \phi_j B^j,$$

then the $AR(p)$ can be rewritten more concisely as:

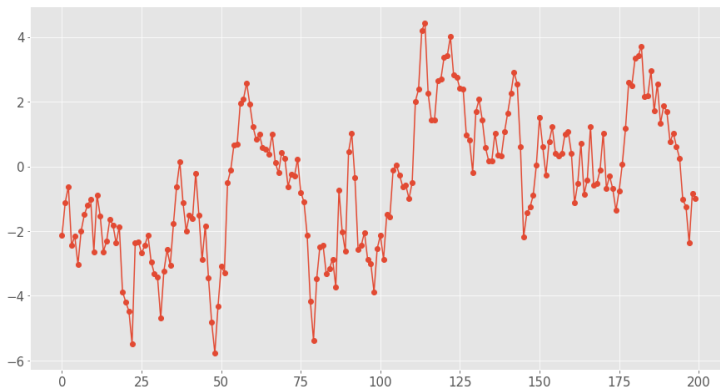
$$\boxed{\phi(B)X_t = w_t}$$

AR Example: $AR(0)$ and $AR(1)$

- The simplest AR process is $AR(0)$, which has no dependence between the terms. In fact, $AR(0)$ is essentially **white noise**.
- $AR(1)$ can be given by $X_t = \phi_1 X_{t-1} + w_t$.
 - Only the previous term in the process and the noise term contribute to the output.
 - If $|\phi_1|$ is close to 0, then the process still looks like white noise.
 - If $\phi_1 < 0$, X_t tends to oscillate between positive and negative values.
 - If $\phi_1 = 1$ then the process is equivalent to random walk, which is not stationary as the variance is dependent on t (and infinite).

AR Examples: AR(1) Process

- Simulated AR(1) Process $X_t = 0.9X_{t-1} + w_t$:

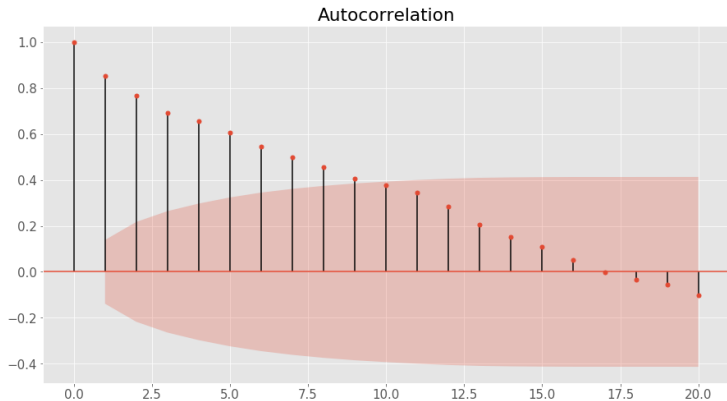


- Mean** $E[X_t] = 0$
- Variance** $\text{Var}(X_t) = \frac{\sigma_w^2}{(1 - \phi_1^2)}$

AR Examples: AR(1) Process

- Autocorrelation Function (ACF)**

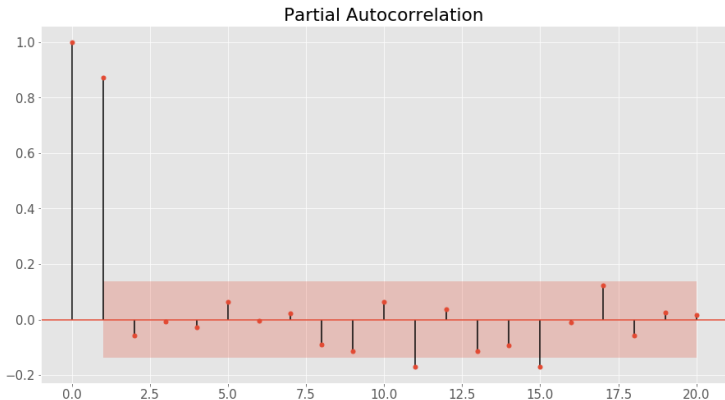
$$\rho_h = \phi_1^h$$



AR Examples: AR(1) Process

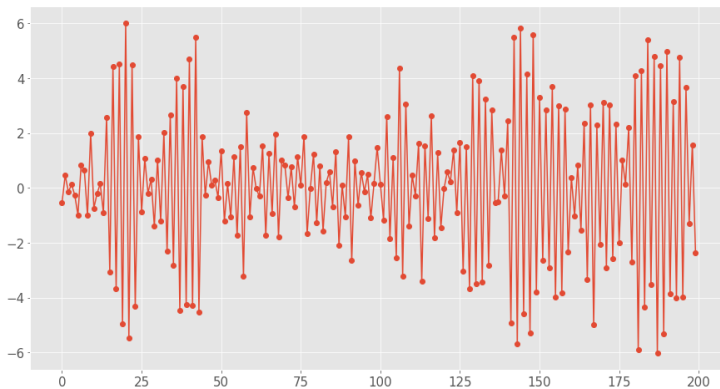
- Partial Autocorrelation Function (PACF)

$$\phi_{11} = \rho_1 = \phi_1 \quad \phi_{hh} = 0, \forall h \geq 2$$



AR Examples: AR(1) Process

- Simulated AR(1) Process $X_t = -0.9X_{t-1} + w_t$:

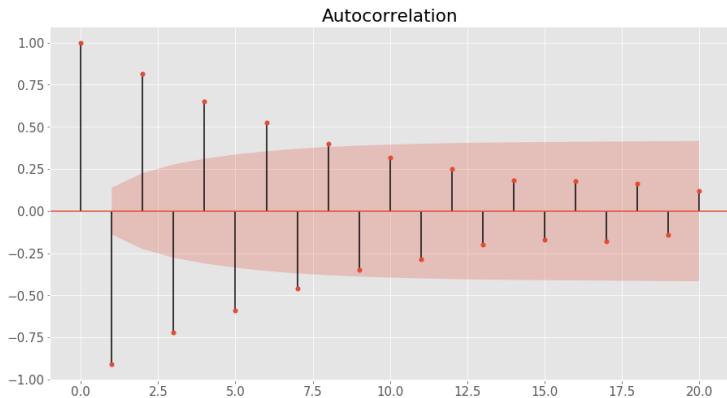


- Mean** $E[X_t] = 0$
- Variance** $\text{Var}(X_t) = \frac{\sigma_w^2}{(1 - \phi_1^2)}$

AR Examples: AR(1) Process

- Autocorrelation Function (ACF)**

$$\rho_h = \phi_1^h$$

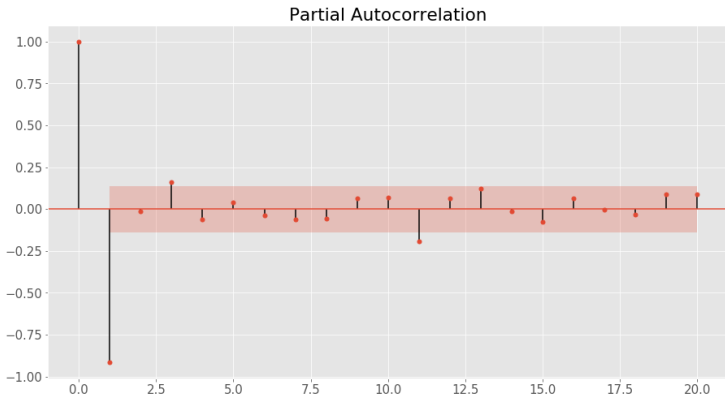


AR Examples: AR(1) Process

- Partial Autocorrelation Function (PACF)

$$\phi_{11} = \rho_1 = \phi_1$$

$$\phi_{hh} = 0, \forall h \geq 2$$



Stationarity of AR(1)

- We can iteratively expand $AR(1)$ representation as:

$$\begin{aligned}X_t &= \phi_1 X_{t-1} + w_t \\&= \phi_1(\phi_1 X_{t-2} + w_{t-1}) + w_t = \phi_1^2 X_{t-2} + \phi_1 w_{t-1} + w_t \\&\vdots \\&= \phi_1^k X_{t-k} + \sum_{j=0}^{k-1} \phi_1^j w_{t-j}\end{aligned}$$

- Note that if $|\phi_1| < 1$ and $\sup_t \text{Var}(X_t) < \infty$, we have:

$$X_t = \sum_{j=0}^{\infty} \phi_1^j w_{t-j}$$

This representation is called the stationary solution.

AR Problem: Explosive AR Process

- We've seen $AR(1)$: $X_t = \phi_1 X_{t-1} + w_t$ while $|\phi_1| \leq 1$.
- What if $|\phi_1| > 1$? Intuitively the time series will “explode”.
- However, technically it still can be **stationary**, because we expand the representation differently and get:

$$X_t = - \sum_{j=1}^{\infty} \phi_1^{-j} w_{t+j}$$

- But clearly this is not useful because we need the future (w_{t+j}) to predict now (X_t).
- We use the concept of *causality* to describe time series that is **not only stationary but also NOT future-dependent**.

General AR(p) Process

- An important property of $AR(p)$ models in general is
 - When $h > p$, theoretical partial autocorrelation function is 0:
$$\phi_{hh} = \text{corr}(X_{t+h} - \hat{X}_{t+h}, X_t - \hat{X}_t) = \text{corr}(w_{t+h}, X_t - \hat{X}_t) = 0.$$
 - When $h \leq p$, ϕ_{pp} is **not zero** and $\phi_{11}, \phi_{22}, \dots, \phi_{h-1, h-1}$ are **not necessarily zero**.
- In fact, **identification of an AR model is often best done with the PACF.**

AR Models: Parameters Estimation

- Note that p is like a hyperparameter for the $AR(p)$ process, thus fitting an $AR(p)$ model presumes p is known and only focusing on estimating **coefficients**, i.e. $\phi_1, \phi_2, \dots, \phi_p$.
- There are many feasible approaches:
 - **Method of moments** estimator (e.g. Yule-Walker estimator)
 - **Maximum Likelihood Estimation (MLE)** estimator
 - **Ordinary Least Squares (OLS)** estimator
- If the observed series is short or the process is far from stationary, then substantial differences in the parameter estimations from various approaches are expected.

Moving Average Models (MA)

- The name might be misleading, but **moving average models** should not be confused with the **moving average smoothing**.
- **Motivation**
 - Recall that in AR models, current observation X_t is regressed using the previous observations $X_{t-1}, X_{t-2}, \dots, X_{t-p}$, plus an error term w_t at current time point.
 - One problem of AR model is the ignorance of correlated noise structures (which is unobservable) in the time series.
 - In other words, the imperfectly predictable terms in current time, w_t , and previous steps, $w_{t-1}, w_{t-2}, \dots, w_{t-q}$, are **also informative** for predicting observations.

Moving Average Models (MA)

- **Definition**

- A moving average model of order q , or $MA(q)$, is defined to be

$$X_t = w_t + \theta_1 w_{t-1} + \theta_2 w_{t-2} + \cdots + \theta_q w_{t-q} = w_t + \sum_{j=1}^q \theta_j w_{t-j}$$

where $w_t \sim wn(0, \sigma_w^2)$, and $\theta_1, \theta_2, \dots, \theta_q$ ($\theta_q \neq 0$) are parameters.

- Although it looks like a regression model, the difference is that the w_t is not observable.
- Contrary to AR model, finite MA model is **always stationary**, because the observation is just a weighted moving average over past forecast errors.

Moving Average Operator

- **Moving Average Operator**
 - Equivalent to autoregressive operator, we define moving average operator as:

$$\theta(B) = 1 + \theta_1 B + \theta_2 B^2 + \cdots + \theta_q B^q,$$

where B stands for backshift operator, thus $B(w_t) = w_{t-1}$.

- Therefore the moving average model can be rewritten as:

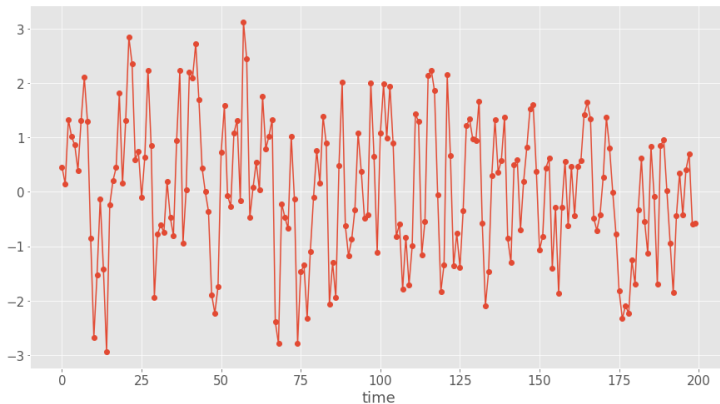
$$X_t = w_t + \theta_1 w_{t-1} + \theta_2 w_{t-2} + \cdots + \theta_q w_{t-q}$$

$$X_t = (1 + \theta_1 B + \theta_2 B^2 + \cdots + \theta_q B^q) w_t$$

$$\boxed{X_t = \theta(B) w_t}$$

MA Examples: MA(1) Process

- Simulated MA(1) Process $X_t = w_t + 0.8 \times w_{t-1}$:

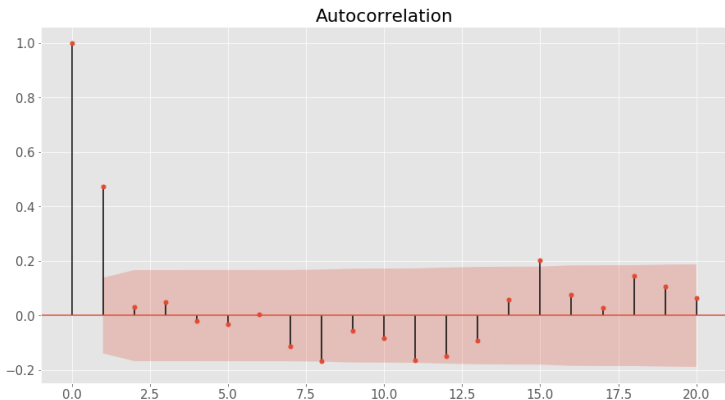


- Mean** $E[X_t] = 0$
- Variance** $\text{Var}(X_t) = \sigma_w^2(1 + \theta_1^2)$

MA Examples: MA(1) Process

- Autocorrelation Function (ACF)**

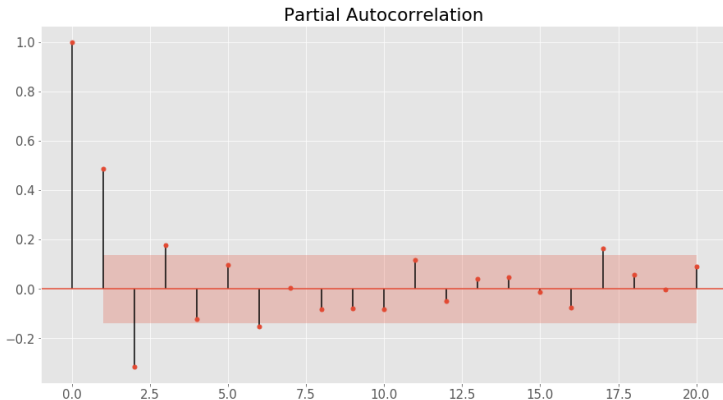
$$\rho_1 = \frac{\theta_1}{1 + \theta_1^2} \quad \rho_h = 0, \forall h \geq 2$$



MA Examples: MA(1) Process

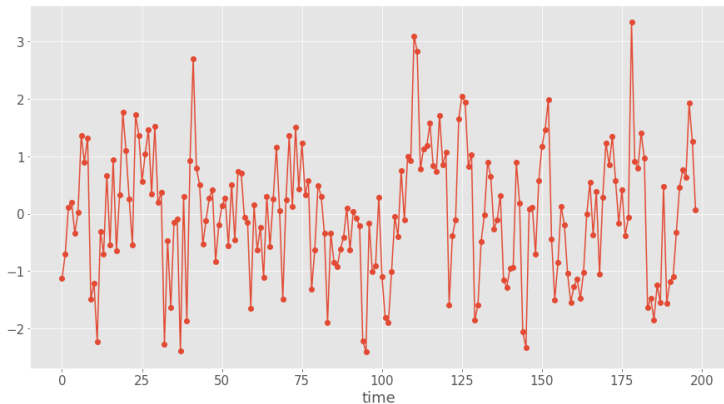
- Partial Autocorrelation Function (PACF)

$$\phi_{hh} = -\frac{(-\theta_1)^h(1 - \theta_1^2)}{1 - \theta_1^{2(h+1)}}, \quad h \geq 1$$



MA Examples: MA(2) Process

- Simulated $MA(2)$ Process $X_t = w_t + 0.5 \times w_{t-1} + 0.3 \times w_{t-2}$:

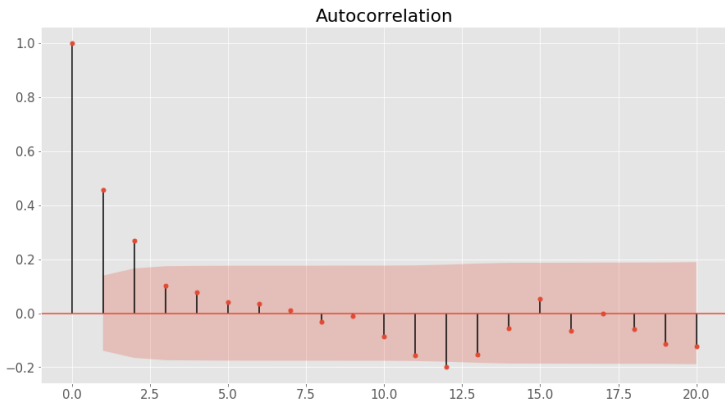


- Mean** $E[X_t] = 0$
- Variance** $\text{Var}(X_t) = \sigma_w^2(1 + \theta_1^2 + \theta_2^2)$

MA Examples: MA(2) Process

- Autocorrelation Function (ACF)**

$$\rho_1 = \frac{\theta_1 + \theta_1\theta_2}{1 + \theta_1^2 + \theta_2^2} \quad \rho_2 = \frac{\theta_2}{1 + \theta_1^2 + \theta_2^2} \quad \rho_h = 0, \forall h \geq 3$$



General MA(q) Process

- An important property of $MA(q)$ models in general is that there are **nonzero autocorrelations** for the first q lags, and $\rho_h = 0$ for all lags $h > q$.
- In other words, ACF provides a considerable amount of information about the order of the dependence q for $MA(q)$ process.
- Identification of an MA model is often best done with the ACF rather than the PACF.

MA Problem: Non-unique MA Process

- Consider the following two $MA(1)$ models:
 - $X_t = w_t + 0.2w_{t-1}, \quad w_t \sim \text{iid } \mathcal{N}(0, 25),$
 - $Y_t = v_t + 5v_{t-1}, \quad v_t \sim \text{iid } \mathcal{N}(0, 1)$
- Note that both of them have $\text{Var}(X_t) = 26$ and $\rho_1 = \frac{5}{26}$.
- In fact, these two $MA(1)$ processes are essentially **the same**. However, since we can only observe X_t (Y_t) but not noise terms w_t (v_t), we cannot distinguish them.
- Conventionally, we define the concept *invertibility* and always choose the invertible representation from multiple alternatives.
 - Simply speaking, for $MA(1)$ models the invertibility condition is $|\theta_1| < 1$.

Comparisons between AR and MA

- Recall that we have seen for $AR(1)$ process, if $|\phi_1| < 1$ and $\sup_t \text{Var}(X_t) < \infty$,

$$X_t = \sum_{j=0}^{\infty} \phi_1^j w_{t-j}$$

- In fact, all **causal** $AR(p)$ processes can be represented as $MA(\infty)$; In other words, **infinite moving average processes are finite autoregressive processes**.
- All **invertible** $MA(q)$ processes can be represented as $AR(\infty)$.
i.e. **finite moving average processes are infinite autoregressive processes**.

MA Models: Parameters Estimation

- A well-known fact is that parameter estimation for MA model is more difficult than AR model.
 - One reason is that the lagged error terms are not observable.
- We can still use **method of moments** estimators for MA process, but we won't get the optimal estimators with Yule-Walker equations.
- In fact, since MA process is nonlinear in the parameters, we need iterative non-linear fitting instead of linear least squares.
- From a practical point of view, **modern scientific computing software packages** will handle most of the details after given the correct configurations.

ARMA Models

- Autoregressive and moving average models can be combined together to form ARMA models.
- **Definition**
 - A time series $\{x_t; t = 0, \pm 1, \pm 2, \dots\}$ is $ARMA(p, q)$ if it is stationary and

$$X_t = w_t + \sum_{i=1}^p \phi_i X_{t-i} + \sum_{j=1}^q \theta_j w_{t-j},$$

- where $\phi_p \neq 0$, $\theta_q \neq 0$, and $\sigma_w^2 > 0$, $w_t \sim wn(0, \sigma_w^2)$.
- With the help of AR operator and MA operator we defined before, the model can be rewritten more concisely as:

$$\boxed{\phi(B)X_t = \theta(B)w_t}$$

ARMA Problems: Redundant Parameters

- You may have observed that if we multiply a same factor on both sides of the equation, it still holds.

$$\eta(B)\phi(B)X_t = \eta(B)\theta(B)w_t$$

- For example, consider a white noise process $X_t = w_t$ and $\eta(B) = (1 - 0.5B)$:

$$(1 - 0.5B)X_t = (1 - 0.5B)w_t$$

$$X_t = 0.5X_{t-1} - 0.5w_{t-1} + w_t$$

- Now it looks exactly like a $ARMA(1, 1)$ process!
- If we were unaware of parameter redundancy, we might claim the data are correlated when in fact they are not.

Choosing Model Specification

- Recall we have discussed that ACF and PACF can be used for determining ARIMA model hyperparameters p and q .

	$AR(p)$	$MA(q)$	$ARMA(p, q)$
ACF	Tails off	Cuts off after lag q	Tails off
PACF	Cuts off after lag p	Tails off	Tails off

- Other criteria can be used for choosing p and q too, such as AIC (Akaike Information Criterion), AICc (corrected AIC) and BIC (Bayesian Information Criterion).
- Note that the selection for p and q is not unique.

“Stationarize” Nonstationary Time Series

- One limitation of ARMA models is the **stationarity** condition.
- In many situations, time series can be thought of as being composed of two components, **a non-stationary trend series** and **a zero-mean stationary series**, i.e. $X_t = \mu_t + Y_t$.
- **Strategies**

- **Detrending**: Subtracting with an estimate for trend and deal with residuals.

$$\hat{Y}_t = X_t - \hat{\mu}_t$$

- **Differencing**: Recall that random walk with drift is capable of representing trend, thus we can model trend as a stochastic component as well.

$$\mu_t = \delta + \mu_{t-1} + w_t$$

$$\nabla X_t = X_t - X_{t-1} = \delta + w_t + (Y_t - Y_{t-1}) = \delta + w_t + \nabla Y_t$$

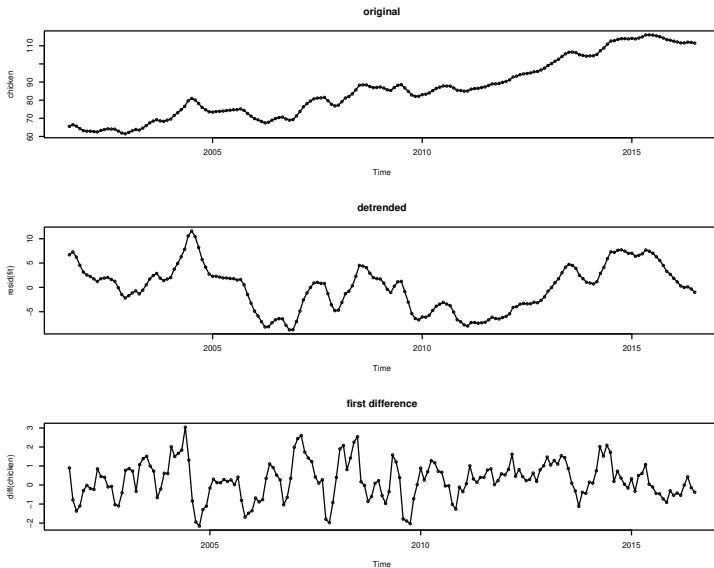
∇ is defined as the first difference and it can be extended to higher orders.

Differencing

- One advantage of differencing over detrending for trend removal is that no parameter estimation is required.
- In fact, differencing operation can be repeated.
 - The first difference eliminates a linear trend.
 - A second difference, i.e. the difference of first difference, can eliminate a quadratic trend.
- Recall the backshift operator $X_t = BX_{t-1}$:

$$\begin{aligned}\nabla X_t &= X_t - X_{t-1} = X_t - BX_t = (1 - B)X_t \\ \nabla^2 X_t &= \nabla(\nabla X_t) = \nabla(X_t - X_{t-1}) \\ &= (X_t - X_{t-1}) - (X_{t-1} - X_{t-2}) \\ &= X_t - 2X_{t-1} + X_{t-2} = X_t - 2BX_t + B^2X_t \\ &= (1 - 2B + B^2)X_t = (1 - B)^2X_t\end{aligned}$$

Detrending vs. Differencing



From ARMA to ARIMA

- **Order of Differencing**

- Differences of order d are defined as

$$\nabla^d = (1 - B)^d,$$

where $(1 - B)^d$ can be expanded algebraically for higher integer values of d .

- **Definition**

- A process X_t is said to be $ARIMA(p, d, q)$ if

$$\nabla^d X_t = (1 - B)^d X_t$$

is $ARMA(p, q)$.

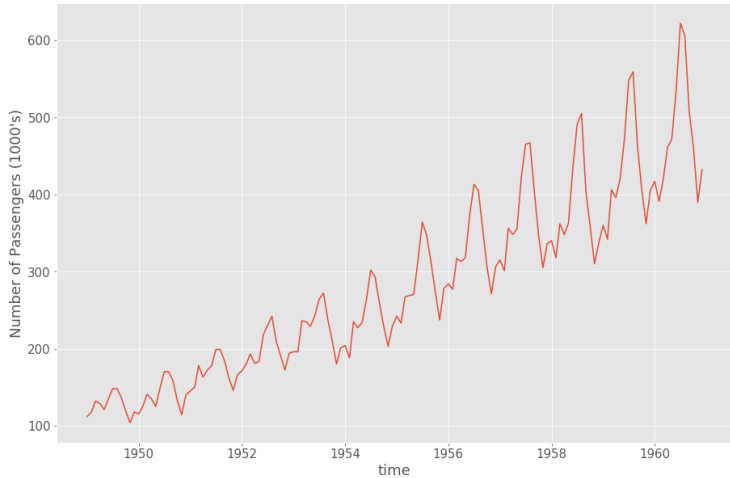
- In general, $ARIMA(p, d, q)$ model can be written as:

$$\phi(B)(1 - B)^d X_t = \theta(B)w_t$$

Box-Jenkins Methods

- As we have seen ARIMA models have numerous parameters and hyper parameters, Box and Jenkins suggests an iterative three-stage approach to estimate an ARIMA model.
- **Procedures**
 - ① **Model identification:** Checking stationarity and seasonality, performing differencing if necessary, choosing model specification $ARIMA(p, d, q)$.
 - ② **Parameter estimation:** Computing coefficients that best fit the selected ARIMA model using *maximum likelihood estimation* or *non-linear least-squares estimation*.
 - ③ **Model checking:** Testing whether the obtained model conforms to the specifications of a stationary univariate process (i.e. the residuals should be independent of each other and have constant mean and variance). If failed go back to step 1.
- Let's go through a concrete example together for this procedure.

Air Passenger Data

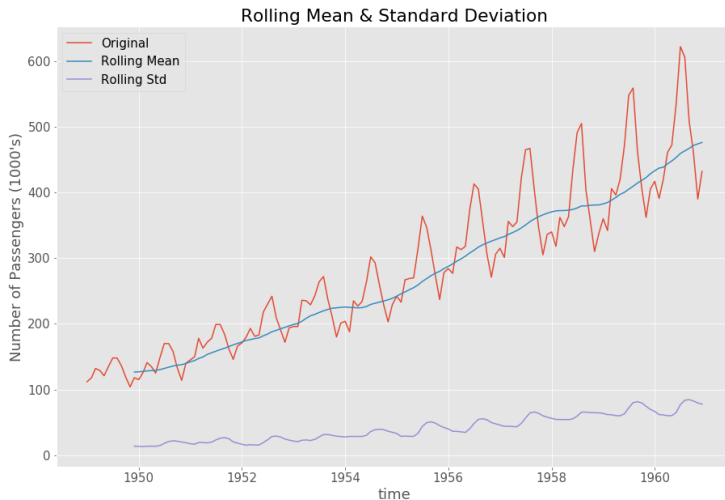


Monthly totals of a US airline passengers, from 1949 to 1960

Model Identification

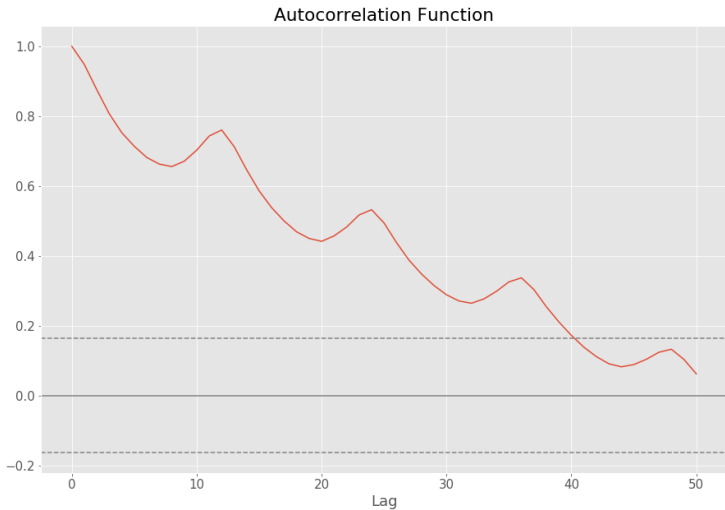
- As with any data analysis, we should construct a time plot of the data, and inspect the graph for any anomalies.
- The most important thing in this phase is to determine if the time series is **stationary** and if there is any **significant seasonality** that needs to be handled.
- **Test Stationarity**
 - Recall the definition, if the mean or variance changes over time then it's non-stationary, thus an intuitive way is to plot **rolling statistics**.
 - We can also make an **autocorrelation plot**, as non-stationary time series often shows very slow decay.
 - A well-established statistical test called **augmented Dickey-Fuller Test** can help. The null hypothesis is *the time series is non-stationary*.

Stationarity Test: Rolling Statistics



Rolling statistics with sliding window of 12 months

Stationarity Test: ACF Plot



Autocorrelation with varying lags

Stationarity Test: ADF Test

- Results of Augmented Dickey-Fuller Test

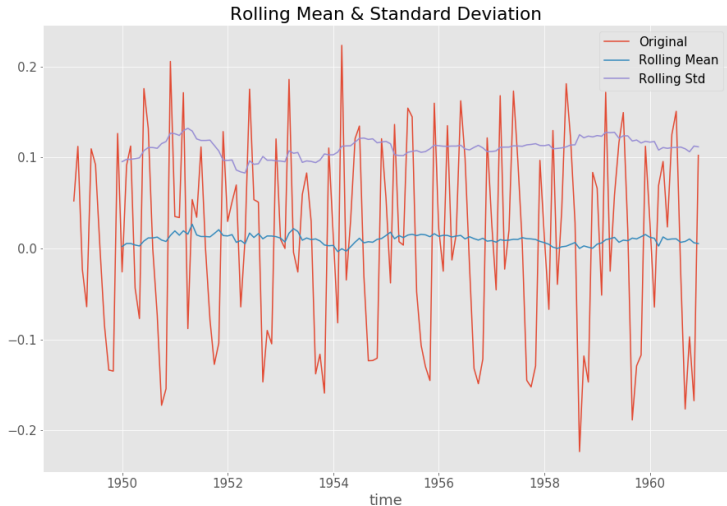
Item	Value
Test Statistic	0.815369
p-value	0.991880
#Lags Used	13.000000
Number of Observations Used	130.000000
Critical Value (1%)	-3.481682
Critical Value (5%)	-2.884042
Critical Value (10%)	-2.578770

- The test statistic is a negative number.
- The more negative it is, the stronger the rejection of the null hypothesis.

Stationarize Time Series

- As all previous methods show that the initial time series is non-stationary, it's necessary to perform **transformations** to make it stationary for ARMA modeling.
 - **Detrending**
 - **Differencing**
 - **Transformation**: Applying arithmetic operations like log, square root, cube root, etc. to stationarize a time series.
 - **Aggregation**: Taking average over a longer time period, like weekly/monthly.
 - **Smoothing**: Removing rolling average from original time series.
 - **Decomposition**: Modeling trend and seasonality explicitly and removing them from the time series.

Stationarized Time Series: ACF Plot



First order differencing over logarithm of passengers

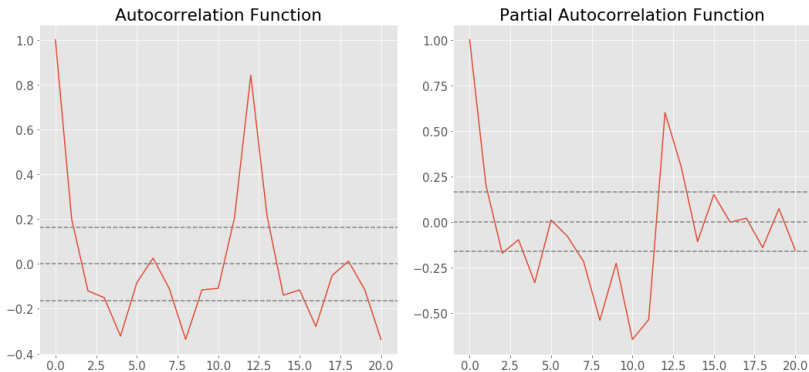
Stationarized Time Series: ADF Test

- Results of Augmented Dickey-Fuller Test

Item	Value
Test Statistic	-2.717131
p-value	0.071121
#Lags Used	14.000000
Number of Observations Used	128.000000
Critical Value (1%)	-3.482501
Critical Value (5%)	-2.884398
Critical Value (10%)	-2.578960

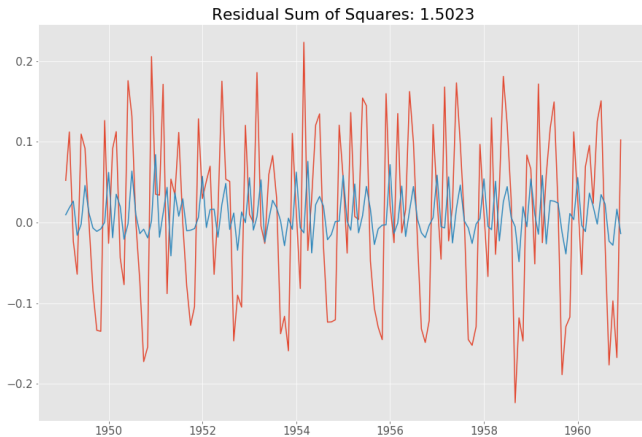
- From the ACF plot, we can see that the mean and std variations have much smaller variations with time.
- Also, the ADF test statistic is less than the 10% critical value, indicating the time series is stationary with 90% confidence.

Choosing Model Specification



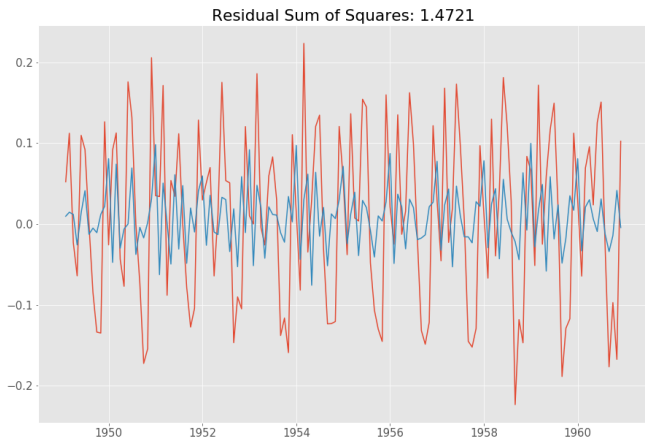
- Firstly we notice an obvious peak at $h = 12$, because for simplicity we didn't model the cyclical effect.
- It seems $p = 2$, $q = 2$ is a reasonable choice. Let's see three models, $AR(2)$, $MA(2)$ and $ARMA(2,2)$.

AR(2): Predicted on Residuals

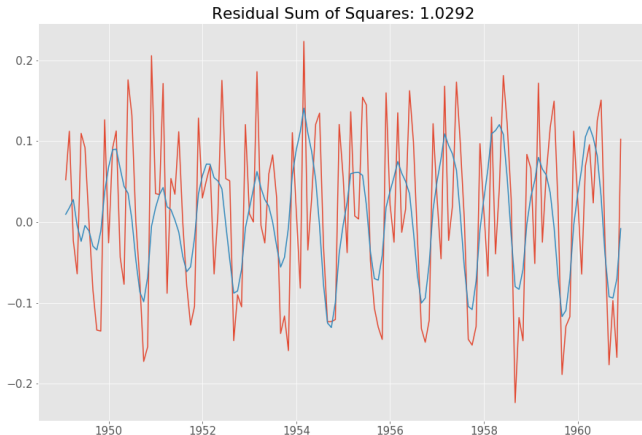


- RSS is a measure of the discrepancy between the data and the estimation model.
 - A small RSS indicates a tight fit of the model to the data.

$MA(2)$: Predicted on Residuals



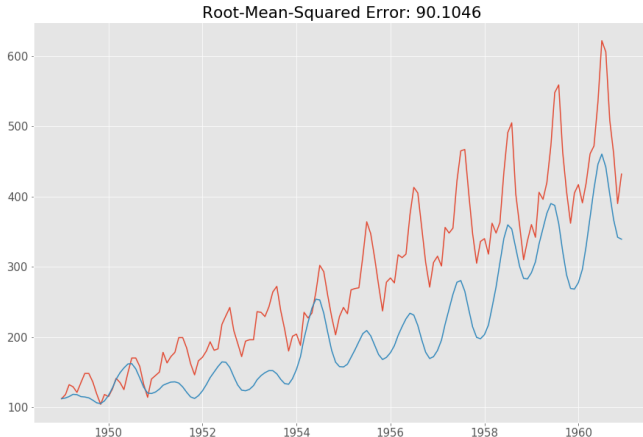
$ARMA(2, 2)$: Predicted on Residuals



- Here we can see that the $AR(2)$ and $MA(2)$ models have almost the same RSS but combined is significantly better.

Forecasting

- The last step is to reverse the transformations we've done to get the prediction on original scale.



SARIMA: Seasonal ARIMA Models

- One problem in the previous model is the lack of seasonality, which can be addressed in a generalized version of ARIMA model called **seasonal ARIMA**.

- **Definition**

- A seasonal ARIMA model is formed by including additional seasonal terms in the ARIMA models, denoted as

$$ARIMA(p, d, q)(P, D, Q)_m,$$

i.e.

$$\phi(B^m) \phi(B) (1 - B^m)^D (1 - B)^d X_t = \theta(B^m) \theta(B) w_t$$

where m represents the number of observations per year.

- The seasonal part of the model consists of terms that are similar to the non-seasonal components, but involve backshifts of the seasonal period.